

Succinct Garbled Circuits with Low-Depth Garbling Algorithms

Hanjun Li
CMU
hanjunl@andrew.cmu.edu

Huijia Lin
University of Washington
rachel@cs.washington.edu

George Lu
UT Austin
gclu@cs.utexas.edu

Abstract

We study the problem of constructing Boolean garbling schemes that are both **succinct**—with garbled circuit size significantly smaller than the original circuit—and have **low-depth** garbling algorithms, where the garbling process runs in parallel time logarithmic in the circuit size. Prior schemes achieve one but not the other, unless relying on indistinguishability obfuscation (iO), which is prohibitively inefficient, relies on a combination of multiple assumptions, and achieves only polynomial garbling depth $\text{poly}(\lambda, \log |C|)$.

We resolve this tension by presenting the first garbling schemes that are both succinct and admit garbling algorithms in NC^1 , based only on standard group and lattice assumptions. Our main results include:

- **One-bit-per-gate garbling** with logarithmic garbling depth based on DDH or RLWE and the existence of a local PRG.
- Succinct **Privacy-free garbling** of size linear in the circuit depth D (and sublinear in the circuit size $|C|$), based on DDH or RLWE.
- **Reusable, fully succinct garbling** with logarithmic garbling depth, based on decomposable LWE.

The DDH-based one-bit-per-gate scheme has tunably small inverse polynomial correctness and privacy errors, which can be made negligible at the cost of increasing garbling depth to $\text{poly}(\lambda)$.

As further extension, we also obtain the first attribute-based encryption schemes with *succinct keys* and *low-depth* key generation.

At a conceptual level, our constructions are derived from a **unified framework** that subsumes all prior approaches to succinct garbling. It identifies the common source of high-depth garbling, and provides a general methodology for reducing garbling depth without sacrificing succinctness, applicable across different techniques and assumptions.

Contents

1	Introduction	3
1.1	Related and Concurrent Works	6
2	Technical Overview	7
3	Definitions	10
3.1	Garbled Circuits with Algebraic Output Labels	13
4	A General Template for Low-Depth Succinct Garbling	16
4.1	Garbling Depth	17
5	Non-Reusable Succinct Privacy-Free Garbling	18
5.1	Preliminaries	18
5.2	Instantiation Under Groups	20
5.3	Instantiation Under Lattices	31
6	Non-Reuseable Partially Succinct Garbling	38
6.1	Instantiation Under Groups	38
6.2	Instantiation Under Lattices	42
7	Reusable Succinct Privacy-Free Garbling	44
7.1	Preliminaries	44
7.2	Construction	47
7.3	Garbler Depth Complexity	49
7.3.1	Log-Depth Preimage Sampling	49
7.3.2	Log-Depth Key-Homomorphic RAM Encoding	50
7.4	Succinctness and Decomposability	50
7.5	Correctness	50
7.6	Security	51
8	Reusable Fully Succinct Garbling	54
8.1	Preliminaries	55
8.2	Construction	56
8.3	Garbler Depth Complexity	57
8.4	Succinctness and Decomposability	57
8.5	Correctness	57
8.6	Security	57
9	Attribute-Based Encryption	59
9.1	Definitions	60
9.2	Construction	60
9.3	Key Succinctness and Generation Depth	62
9.4	Correctness	62
9.5	Security	63
10	Acknowledgments	67

1 Introduction

A Boolean garbling scheme transforms a Boolean circuit C and an input x into garbled versions, $(\hat{C}, \text{gk}) \leftarrow \text{GarbC}(1^\lambda, C)$ and $\hat{x} \leftarrow \text{Garblnp}(\text{gk}, x)$, from which the output can be recovered $\text{Eval}(C, \hat{C}, \hat{x}) = y$ without revealing any other information about the input. Garbling schemes are a foundational primitive in cryptography with wide-ranging applications, including multiparty computation [Yao86, BMR90, IPS08, GS17, GS18, BL18], parallel cryptography [AIK04, AIK06], and verifiable computation [GGP10, AIK10] etc; see [App17] for a survey.

Given their central role, a huge amount of work has focused on improving the efficiency of garbled circuits. Two aspects have received particular attention: succinctness, namely minimizing the size of the garbled circuit and garbled inputs; and low-depth garbling, namely reducing the depth, and thus the parallel-time, of the garbling algorithms. Surprisingly, almost all known constructions achieve either succinctness or low-depth garbling but not both, except for schemes relying on indistinguishability obfuscation (iO). In this work, we resolve this tension by presenting an array of *succinct* and *low-depth* garbling schemes, based on standard group and lattice assumptions, that are significantly simpler and more efficient than iO-based approaches.

Succinctness. Yao’s original construction [Yao82] and subsequent optimizations [BMR90, NPS99, KS08, PSSW09, KMR14, GLNP15, ZRE15, RR21] remain the most computationally efficient general-purpose garbling schemes, owing to their reliance on fast symmetric-key cryptography. They also achieve extremely low garbling depth. The main limitation is their communication complexity: the current state of the art, due to Rosulek and Roy [RR21], devotes $1.5\lambda + 5$ bits to garbling an AND gate, while XOR and NOT gates are free under the random oracle model.

This $\Omega(\lambda)$ -bits-per-gate barrier motivated the study of *succinct* garbled circuits, where the bit-length of the garbled circuit $|\hat{C}|$ is asymptotically smaller than the description size of the original circuit C . Since a circuit C with $|C|$ gates requires at least $|C| \log |C|$ bits to describe, succinct garbling demands $|\hat{C}| < |C| \log |C|$ ¹. Two main approaches have emerged:

- *Fully succinct garbling schemes.* These achieve garbled circuit size independent of $|C|$. Constructions either rely on iO [KLW15, BCG⁺18], or on combinations of homomorphic encryption (HE) and attribute-based encryption (ABE) [GKP⁺13, BGG⁺14, HLL23]. In addition to optimal asymptotic size, these schemes are *reusable* [GKP⁺13], meaning the same garbled circuit can be used to evaluate arbitrarily many input encodings. However, their concrete efficiency is poor. iO-based constructions are prohibitively expensive, while FHE-plus-ABE approaches are more efficient in a relative sense, they are still far from practical.
- *One-bit-per-gate garbling schemes.* A more recent line of work [LWYY25, ILL25b, MORS25b] constructs garbled circuits of size $|\hat{C}| = |C| + o(|C|)$, achieving essentially one bit per gate. These schemes are significantly more efficient than fully succinct ones and have already demonstrated promising concrete performance [ILL25b, LWYY25]. This improvement stems from their reliance on comparatively lightweight techniques, such as homomorphic secret sharing (HSS) [BGI16] and FHE without bootstrapping [GSW13], rather than heavy tools like iO or the non-black-box combination of FHE and ABE. A further advantage is flexibility in assumptions: one-bit-per-gate constructions can be based on a diverse set of algebraic assumptions, including RLWE, NTRU, DDH over prime-order or Paillier groups, and DCR, in contrast to the narrow reliance of fully succinct approaches on iO or lattice-based primitives.

Low-depth garbling. A major drawback of existing succinct garbling schemes in either of these categories is that their garbling algorithms have high depth, growing polynomially with the circuit depth D (namely $\text{poly}(\lambda, D)$). The only exception is constructions based on iO, which achieve garbling depth $\text{poly}(\lambda)$ but at prohibitive concrete cost.

In contrast, a low-depth garbling scheme is one where the garbling depth depends only logarithmically on the circuit size, that is, $\text{poly}(\lambda, \log |C|)$. Such schemes enable highly parallelizable garbling, which are crucial in applications where garbling depth directly translates into round complexity and latency. Yao’s garbled circuits exemplify this property: all gates can be garbled in parallel, yielding depth $\text{poly}(\lambda)$ under one-way functions, $O(\log \lambda)$ assuming PRGs in NC^1 , and even NC^0 when combined with an NC^0 -garbling scheme for NC^1 circuits [IK02, AIK04].

¹Note that C and $\hat{C}$ are syntactically different objects. C is a circuit, while $\hat{C}$ is a binary string. Adopting standard notation, $|C|$ denotes the number of gates in the original circuit C , while $|\hat{C}|$ denotes the bit-length of the garbled circuit.

This contrast raises the central questions of our work:

*Can we construct Boolean garbling schemes that are simultaneously succinct and low-depth, without
relying on iO?
Can succinct garbling be in NC^1 , which is unknown even assuming iO?*

Our Results. We give a comprehensive and affirmative answer to the above question by enhancing existing non-iO-based succinct garbling schemes to achieve low – indeed NC^1 – garbling depth while preserving succinctness. Concretely: 1) by enhancing the schemes of [ILL25b], we obtain one-bit-per-gate garbled circuits with NC^1 garbling, based on DDH or RLWE, and the existence of local Pseudorandom Generators (PRG) [Gol00]; and 2) by enhancing the “leveled” version of the scheme in [GKP⁺13, BGG⁺14], we obtain a *fully succinct, reusable* garbling scheme with NC^1 garbling, based on decomposable LWE [AMR25]. To the best of our knowledge, our succinct garbling schemes are the first to achieve NC^1 garbling depth; this level of parallel efficiency was not attained even by prior iO-based schemes, which have $\text{poly}(\lambda)$ depth.

A key intermediate step towards these constructions is the design of new *privacy-free* garbling schemes that retain various degrees of succinctness while enjoying NC^1 garbling, based on the same assumptions. These schemes are of independent interest. Unlike standard garbling, privacy-free garbling [JKO13, FNO15] requires only computational integrity, not privacy: the input x is public, the evaluation algorithm takes it as input explicitly, and the scheme guarantees *authenticity*, namely that it is computationally hard to produce a valid label for an incorrect output (with respect to an authentication key). Privacy-free garbling and the closely related notion of partial garbling [IW14] have found many applications, including efficient zero-knowledge arguments for non-algebraic statements [JKO13, FNO15], CDS and PSM protocols, constrained PRFs [ILL25a], and blockchain applications [Lin25]. Yet in this setting as well, no prior construction achieved both succinctness and low-depth: the only known succinct privacy-free garbling [ILL25a] had depth proportional to the circuit depth. Our work fills this gap.

In essence, our constructions are summarized in the following informal theorems.

Theorem 1.1 (One-Bit-Per-Gate, NC^1 -Garbling-Depth, Garbled Circuits, Informal). *Assume DDH over prime-order subgroup of $\mathbb{Z}_p^*$, or DDH over Paillier groups, or RLWE, and the existence of constant-local PRGs. The constructions in Section 6 are garbling schemes for Boolean circuits with the following properties:*

- One-bit-per-gate: *The size of the garbled circuit is $|\hat{C}| = |C| + \text{poly}(\lambda) \cdot (D + o(|C|))^2$, where $|C|$ is the number of gates and D is the depth of the circuit.*
- NC^1 garbling: *The depth of the garbling algorithm is $O(\log \lambda + \log |C|)$.*

The instantiation based on DDH over subgroup of $\mathbb{Z}_p^$ has a $1/\text{poly}(\lambda)$ correctness and privacy errors. The errors can be made negligible, at the price of increasing the garbling depth to $\text{poly}(\lambda)$. Moreover, assuming the same assumptions without local PRGs, the constructions in Section 5 yield privacy-free garbling schemes for Boolean circuits, with garbled circuit size $\text{poly}(\lambda) \cdot (D + o(|C|))$ and garbling algorithm depth $O(\log \lambda + \log |C|)$.*

We note that achieving NC^1 garbling requires the ability to evaluate iterated basic operations of the underlying algebraic structure, such as repeated group operations or ring multiplications, in NC^1 . For this reason, we assume DDH over subgroups of $\mathbb{Z}_p^*$. When instantiated under DDH over general groups, our schemes are still low-depth, but with depth complexity $\text{poly}(\lambda)$ rather than NC^1 .

Theorem 1.2 (Depth-Succinct, NC^1 -Garbling-Depth, Reusable Garbled Circuits, Informal). *Assume decomposable LWE. The construction in Section 8 is a reusable garbling scheme for Boolean circuits with the following properties:*

- Depth-succinctness: *The size of the garbled circuit is $\text{poly}(\lambda, D)$, where D is the circuit depth.*
- NC^1 garbling: *The depth of the garbling algorithm is $O(\log \lambda + \log |C|)$.*

²The exact sublinear factors in $|C|$ varies per construction. Refer to the formal statements in Section 6 for the exact parameters.

The decomposable LWE assumption was recently introduced in [AMR25]; it can be viewed as a circular-security assumption for a natural lattice-based public-key encryption scheme and is implied by the hardness of the succinct LWE problem of [Wee24]. See [Assumption 7.5](#) for the formal definition and [AMR25] for further discussion.

Theorems 1.1 and 1.2 improve upon prior one-bit-per-gate garbling [LWYY25, ILL25b, MORS25b] and succinct reusable garbling [GKP⁺13, BGG⁺14, QWW18] respectively by reducing the garbling depth from $\text{poly}(\lambda, D)$ to $\text{poly}(\log \lambda, \log |C|)$ while maintaining the same asymptotic garbling size and underlying assumptions (albeit restricted to integer groups for DDH).

Extending Theorem 1.2: ABE with Low-Depth and Succinct Key Generation. We further extend the above theorem to obtain the first attribute-based encryption (ABE) with succinct keys and low-depth key generation [Section 9](#).

Reusable garbled circuits and attribute-based encryption (ABE) are closely connected. Existing reusable garbled circuits [GKP⁺13, BGG⁺14, QWW18] are typically constructed by combining homomorphic encryption (HE) and ABE, in such a way that the depth of the garbling algorithm mirrors the depth of the ABE key generation, and the garbled circuit size grows with the size of the ABE secret key. Since all prior ABE schemes with succinct secret keys [BGG⁺14, HLL23] had high-depth key generation, the corresponding reusable garbled circuits also had high-depth garbling. We show that the same techniques we use to reduce garbling depth also apply to ABE, yielding the first ABE scheme with both succinct keys and low-depth key generation.

Theorem 1.3 (ABE with Low-Depth and Succinct Key Generation, Informal). *Assume decomposable LWE. The construction in [Section 9](#) is an ABE scheme for Boolean circuits with the following properties:*

- Depth-succinct keys: *For a depth- D circuit C , the secret key has size $\text{poly}(\lambda, D)$.*
- NC¹ key generation: *The depth of the key generation algorithm is $O(\log \lambda + \log |C|)$.*

Unified Framework. Our constructions are obtained through a unified framework (described in [Section 4](#)) that provides a general method for reducing garbling depth without sacrificing succinctness. This framework extends and abstracts the approach of [ILL25b], which combined homomorphic secret sharing (HSS) with algebraic HMAC (aHMAC) [ILL25a] to realize one-bit-per-gate garbled circuits. Unlike that specialized setting, our framework operates at a higher level of abstraction and encompasses both one-bit-per-gate garbling and lattice-based reusable garbling [GKP⁺13, BGG⁺14], despite their very different underlying tools (HSS and aHMAC in the former, HE and ABE in the latter).

The framework highlights a common source of high garbling depth in all prior succinct garbling schemes. Conceptually, it views garbling as a two-party protocol between the garbler and the evaluator, who jointly maintain secret shares of every wire value. Evaluation proceeds via local homomorphic computations on each party’s shares (corresponding to the garbling and evaluation algorithms), together with one-way communication from the garbler to the evaluator (the garbled circuit itself). In this perspective, the “secret sauce” behind succinctness is the use of homomorphic techniques—HSS, aHMAC, HE, ABE—that reduce communication and thereby garbled circuit size.

The conflict with low-depth garbling arises because these homomorphic techniques typically require *symmetric* (but not identical) local computation by both the garbler and the evaluator, which results in high depth at both ends. Our key idea is to break this symmetry by introducing *periodic share refreshing*. Specifically, the garbler’s share of an intermediate value is re-sampled from a fixed distribution, while the evaluator’s share is updated accordingly. As a result, the garbler’s computation becomes shallow, given all refreshed shares.

A challenge is that naively refreshing all intermediate shares incurs communication linear in $|C|$, destroying succinctness. To overcome this, we compress shares of intermediate values into compact shares of a *hash* and refresh only the latter, which incurs small communication and preserves succinctness. To implement this, we adapt the recent algebraic compression techniques of [AMR25, Wee24], originally developed for succinct garbled RAM and ABE with succinct ciphertexts, to the garbling setting, and provide new group-based instantiation.

Open Questions The garbling size of our schemes in [Theorem 1.1](#) and [Theorem 1.2](#) all exceed $D \cdot \text{poly}(\lambda)$. Under stronger circular-security variants of the assumptions, prior works can reduce the garbled circuit size further, down to $|\hat{C}| = |C| + \text{poly}(\lambda)$ for one-bit-per-gate garbling [LWYY25, ILL25b, MORS25b] and $\text{poly}(\lambda)$ for reusable

garbling [HLL23]. Our schemes, however, do not currently admit analogous variants, and we leave closing this gap as an interesting open question.

Another natural question is whether the depth of our schemes can be further reduced to NC^0 using the standard composition technique of [IK02, AIK04]. This technique combines an NC^1 garbling algorithm for general circuits with an NC^0 garbling algorithm for NC^1 circuits. While this approach would indeed reduce the garbling depth to NC^0 , it comes at the cost of losing succinctness, since existing NC^0 schemes are not succinct. Whether succinct garbling can be constructed in NC^0 is another extremely interesting question.

1.1 Related and Concurrent Works

This work focuses on garbling schemes for Boolean circuits. Garbling schemes for other computational models have also been studied, and in some cases achieve notions of succinctness and low-depth garbling simultaneously. We describe and compare with these below.

Comparison with Succinct-Time Garbled RAM [AMR25]. The work of [AMR25] (AMR) targets minimizing the *time* to garble RAM programs, whereas our work aims to reduce the *depth* (parallel time) for garbling Boolean circuits. Based on circular LWE and decomposable LWE, AMR shows how to garble a RAM program P in time $\text{poly}(\lambda, |P|, \log T)$, logarithmic in the time complexity T of P . By contrast, garbling a circuit necessarily takes time linear in its size $|C|$, simply to read its description. Instead, we focus on reducing the parallel-time: our garbling algorithms run in logarithmic parallel time (NC^1), while the AMR garbling algorithm has depth $\text{poly}(\lambda)$.

In addition, our constructions provide advantages beyond parallel-time. Our non-reusable schemes (Theorem 1.1) can be based on group assumptions as well as lattices, whereas AMR relies solely on lattice assumptions. Our reusable schemes (Theorem 1.2) achieve reusability, while the AMR construction does not. Despite these differences, our work is technically connected with [AMR25], as mentioned above and elaborated in later sections.

Comparison with Succinct Bounded-Integer Garbling [BLLL23]. The work of [BLLL23] presents a garbling scheme for arithmetic circuits (with addition and multiplication gates) over integers from a bounded range $[2^\ell]$, where ℓ is the bit length of the values. Their scheme achieves communication of $(O(\ell) + \text{poly}(\lambda))$ bits per gate using linearly homomorphic encryption based on DCR. As in Yao’s garbled circuits, their garbling algorithm samples fresh “keys” for each gate, so all gates can be garbled in parallel, yielding NC^1 garbling depth.

Follow-up work [MORS24] improved the communication to $\ell + \text{poly}(\lambda)$ bits per gate using HSS techniques, but at the cost of losing low-depth garbling. However, these schemes do not yield succinct garbling for Boolean circuits: when $\ell = 1$, the communication remains $\text{poly}(\lambda)$ bits per gate.

Comparison with Attribute-Based LFE from DCR [MORS25a]. The concurrent work of [MORS25a] presents an attribute-based laconic function evaluation (AB-LFE) scheme from DCR. Our works share some technical similarities - both works present new group-based instantiations of linear-hashing reminiscent of the lattice-based constructions of [Wee24, AMR25] and leverage the homomorphic properties of such a hash to achieve some notion of succinctness. However, beyond this shared technical core, our techniques and results building upon this hash largely diverge. [MORS25a] focuses on achieving a notion of sender time-succinctness in a two message computation setting corresponding to (AB)-LFE, while we focus on retaining size-succinctness in the setting of garbled circuits while reducing the depth complexity of the garbler.

We note that the constructions of neither paper do not directly imply the other - the AB-LFE scheme of [MORS25a] is not directly low-depth and our group-based garbling scheme does not achieve any notion of laconicity. However, due to the usage of essentially equivalent underlying hash compression, it seems likely the techniques can be applied in a fairly direct manner when relevant (e.g. modifying the [MORS25a] AB-LFE to additionally be low-depth using ideas presented in this work).

2 Technical Overview

The starting point of our ideas will be the high level view of succinct garbling in [MORS24, MORS25b, ILL25b], where a garbling scheme is constructed as a 2-party protocol where the garbler and evaluator jointly compute ‘shares’ of each wire w_i in the circuit $\llbracket w_i \rrbracket$. This view captures both privacy-free garbling (where the input is not hidden from the evaluator, who only seeks to produce an authenticated ‘label’ of the output) as well as the traditional notion of private garbling. In these protocol, only the garbler sends messages, which represents the garbled circuit. This roughly breaks down a garbling protocol into the following steps:

- $\text{Enc}(x) \rightarrow \llbracket x \rrbracket$: Given an input x , the garbler generates an encoding $\llbracket x \rrbracket_G, \llbracket x \rrbracket_E$ of the input, and $\llbracket x \rrbracket_E$ is sent to the evaluator as the garbled input.
- $\text{HGate}(\llbracket w_1 \rrbracket, \llbracket w_2 \rrbracket, \text{op}) \rightarrow \llbracket \text{op}(w_1, w_2) \rrbracket$: With encodings of input wires, the garbler and evaluator can jointly generate encoding of the output of a gate. Any communication this may require on the garbler’s part is part of the garbled circuit.
- $\text{Dec}(\llbracket C(x) \rrbracket) \rightarrow \text{dk}$: Finally, given an encoding of the output wire, the garbler sends some decoding information which allows the evaluator to recover the output of the garbled circuit.

Zooming in a little further, in these prior schemes[ILL25b, MORS25b], the garbler’s $\llbracket w_i \rrbracket_G = k_i$ and evaluator’s $\llbracket w_i \rrbracket_E = \sigma_i = k_i + s \cdot w_i$ shares form an additive secret sharing of $s \cdot w_i$, for some garbling secret s . To advance the computation with some gate $\text{op}(w_1, w_2)$, the garbler publishes some cryptographic material which enables the garbler and evaluator to run HGate to obtain additive shares of $s' \cdot \text{op}(w_1, w_2)$ either completely *noninteractively* (in the case of privacy-free garbling) or with only a single bit of communication (in the setting of private garbling).³ Importantly, this cryptographic material is reusable, and given shares of every depth j wire, enables the garbler and evaluator to obtain shares of every depth $j + 1$ wire, enabling succinctness. Moreover, this HGate protocol works on *any* additive sharing of $s \cdot w$.

Rekeying. This latter observation yields the following simple idea - the garbler ‘fixes’ their shares k_i of the additive encoding of all the circuit wires before performing any homomorphic evaluation. Since the garbler already has their ‘shares’ of every wire in the circuit, they can in parallel run HGate for every wire of C , albeit obtaining a ‘different’ additive share of the output wire k'_i . To ensure the evaluator can proceed with their computation, the garbler simply sends them $\text{rk} = (k_i - k'_i)$, which allows the evaluator to realign or ‘rekey’ their additive share to be consistent with the garbler’s independently sampled shares.

We formalize and abstract this idea through defining a ‘rekeying’ protocol, which takes the following form:

- $\text{RekeySamp}(1^\lambda) \rightarrow k$: The garbler can sample a fresh share k
- $\text{RekeyGen}(k'_i, k_i) \rightarrow \text{rk}$: Given a share k'_i (which will be the output of HGate) and a ‘target share’ k_i (which will be independently sampled), the garbler computes a rekeying key.
- $\text{RekeyEval}(\sigma'_i, \text{rk}) \rightarrow \sigma_i$: When delivered to the evaluator, the rekey rk can be used in conjunction with a share σ'_i to produce a share σ_i .

So long as (k'_i, σ'_i) and (k_i, σ_i) encode the same value, this will preserve correctness of HGate, and hence the garbling scheme.

Hash Compression. Unfortunately, directly applying this straightforward method of rekeying as described above removes any hope for succinctness, as the garbler will need to send the evaluator a key for every wire of the circuit. Instead, what we need is a *succinct* rekeying operation where a short rekeying key can re-encode many different wire encodings.

³[ILL25b] also describe an ‘unlevelled’ variant where $s = s'$ which relies on circular security which achieves depth-independent succinctness. Since our scheme will incur depth-dependent factors for other reasons, we will look at their leveled construction

One way to realize this is rather than attempting to rekeying individual wire values, we can instead rekey a short ‘hash’ of the wire values. For this to be useful with prior gate evaluation procedures, which operates directly on encodings of wire values, there needs to exist non-interactive procedures for both converting an encoding of wires to an encoding of the hash of said wires as well as the reverse. A bit more formally, our hashing scheme should support the following protocols⁴:

- $\text{Compress}(\llbracket x \rrbracket) \rightarrow \llbracket \text{Hash}(x) \rrbracket$: Given an encoding of x , which we recall takes the form $\{k_i\}$ for the garbler and $\{k_i + s \cdot w_i\}$ for the evaluator, they should respectively be able to compute shares k_{Hash} and $(k_{\text{Hash}} + s \cdot \text{Hash}(\{w_i\}))$.
- $\text{Expand}(\llbracket \text{Hash}(x) \rrbracket) \rightarrow \llbracket x \rrbracket$: Conversely, given shares k_{Hash} and $(k_{\text{Hash}} + s \cdot \text{Hash}(\{w_i\}))$, the garbler and evaluator should each compute shares $\{k_i\}$, $\{k_i + s' \cdot w_i\}$ respectively.

This enables the garbler and evaluator to alternate between encodings of the wires (to compute HGate) and encodings of hash of wires (to run Rekey).

It turns out by virtue of the additive structure of the encodings, any linear hash function $\text{Hash}(\mathbf{x}) = \mathbf{x}^\top \cdot \mathbf{u}$ can be ‘compressed’ while supporting the share format. Thus, the main challenge will be supporting the Expand operation. Much like how succinct gate evaluation HGate is constructed in prior works, this will rely on a publishing some cryptographic ‘hints’ dependent on garbling secrets s, s' which allow the garbler and evaluator to compute $\llbracket x \rrbracket$ from $\llbracket \text{Hash}(x) \rrbracket$ while simultaneously hiding s, s' . Inspired by the hashing and reconstruction technique recently introduced by [AMR25, Wee25] for constructing succinct-time RAM garbling and ABE with succinct ciphertext from lattices, given coefficients $\mathbf{u} = (u_1, \dots, u_N)^\top$ to a linear hash, we can convert an encoding of $\text{Hash}(\mathbf{x}) = \mathbf{x}^\top \cdot \mathbf{u}$ into an encoding of $\mathbf{x}$ by publishing a random vector $\mathbf{t} = (t_1, \dots, t_N)^\top$ as well as the matrix $\mathbf{W} = s \cdot \mathbf{t} \cdot \mathbf{u}^\top - s' \cdot \mathbf{I}_N$, embedding s' in the diagonal of $\mathbf{W}$ masked by term $s \cdot t_i \cdot u_i$. Now, given a sharing of the form $(k, k + s \cdot \text{Hash}(\mathbf{x}))$, the garbler and evaluator can recover a share of x_i by first multiplying their respective sharings by t_i

$$t_i \cdot \llbracket \text{Hash}(\mathbf{x}) \rrbracket_G = t_i \cdot k \quad , \quad t_i \cdot \llbracket \text{Hash}(\mathbf{x}) \rrbracket_E = t_i \cdot (k + s \cdot (\mathbf{u}^\top \cdot \mathbf{x})) = t_i \cdot k + s \cdot t_i \cdot \mathbf{u}^\top \cdot \mathbf{x}.$$

This latter term of the evaluator’s encoding is in fact exactly the i^{th} row of $\mathbf{W}$ multiplied by $\mathbf{x}$, which allows the evaluator to compute

$$t_i \cdot \llbracket \text{Hash}(\mathbf{x}) \rrbracket_E - \langle \mathbf{w}_i, \mathbf{x} \rangle = t_i \cdot k + s \cdot t_i \cdot \mathbf{u}^\top \cdot \mathbf{x} - (s \cdot (t_i \cdot \mathbf{u}^\top) \cdot \mathbf{x} - s' \cdot \mathbf{e}_i^\top \cdot \mathbf{x}) = t_i \cdot k + s' \cdot x_i.$$

where $\mathbf{e}_i$ is the i^{th} canonical basis vector. The garbler and evaluator can now use

$$t_i \cdot k = \llbracket x_i \rrbracket_G, t_i \cdot k + s' \cdot x_i = \llbracket x_i \rrbracket_E$$

as their new encoding of x_i .

While this gives us the desired functionality, directly giving out $\mathbf{W}$ clearly compromises any sort of secrecy of s, s' . Instead, we will need to give out cryptographic ‘encryption’ of $\mathbf{W}$ which still enables us to perform the above computation. We discuss how this is instantiated in various contexts below:

Group-based Construction: In both prime-order and Paillier groups, the high level idea is instead of publishing $\mathbf{t}, \mathbf{W}$, the garbler instead publishes $g^{\mathbf{t}}, g^{\mathbf{W}}$. Intuitively, this allows us to use the secrecy of these terms in the exponent to act as an ElGamal-esque encryption of the secret s' . Since the garbler and evaluator only perform linear operations, this can be directly translated to computing $(g^{\mathbf{t}})^{\llbracket \text{Hash}(\mathbf{x}) \rrbracket}$, $(g^{\mathbf{W}})^{\mathbf{x}}$, which gives an appropriate additive sharing *in the exponent* of $\mathbb{G}$. Since rekeying is a linear operation, this can be done directly in the exponent of the group! However, while such sharings in the exponent of the group are sufficient for rekeying, such an encoding is insufficient for HGate. Thus, our Expand operation will need to recover a sharing over $\mathbb{Z}$, which we achieve using the techniques of distributed discrete log DDLog protocol [BGI16]. This technique comes with two caveats (in prime-order groups): it incurs some inverse polynomial correctness error, and only works if the additive shares have some bounded polynomial difference. The former will simply reflect as an inverse polynomial correctness error in our ‘base’ garbling protocol, which can be amplified through generic techniques. With respect to the latter, notice while we can always

⁴In our real scheme, we refer to Expand as HRead which also has the syntax to ‘locally’ extract/read a single wire value from the hash.

maintain our initial input $\mathbf{x}$ and the shares of the input encoding $\llbracket \mathbf{x} \rrbracket$, have bounded distance, the term of $s' \cdot \mathbf{x}$ in $g^{\mathbf{W}}$ could be as large as the group order p itself. However, rather than giving out encryptions of s' directly, we can instead bit-decompose s' , using $\mathbf{W}$ to encode the bits of s' which are only ‘repacked’ after the DDLog returns the encoding to $\mathbb{Z}$.

At a glance, it may not be clear that this hashing has accomplished anything, as although the hash itself is now shorter than the original input, the garbler also has to give out $\text{ct}_{s,s'} = g^t, g^{\mathbf{W}}$, which are at least as long as the input of Hash (and in fact quadratically longer than said input). However, an important distinction between these two quantities is that $\text{ct}_{s,s'}$ are *reusable* (within the same circuit depth of a garbling scheme). Thus, by fixing a compression factor K our garbling scheme pays a one-time cost of $K^2 \cdot D$ to reduce the re-keying communication by a factor of K .

We can see this can be asymptotically minimized by setting $K = \left(\frac{|C|}{D}\right)^{\frac{1}{3}}$, incurring $D + |C|^{\frac{2}{3}} D^{\frac{1}{3}}$ total communication (ignoring $\text{poly}(\lambda)$ factors). We also note that by assuming a circular variant of power-DDH, we can reuse the public parameters across layers, incurring a one-time cost of K^2 and rather than $K^2 \cdot D$, meaning our total communication can be brought down to $D + |C|^{\frac{2}{3}}$.

Lattice-based Construction In the setting of lattices, this same hashing idea can be executed by simply publishing $\mathbf{t}, \mathbf{u}$ in the clear and an “encryption” as simply a noisy version of $\mathbf{W} + \varepsilon$. In fact, as observed in prior work [AMR25, Wee25], since these hash coefficients can be published in the clear, this enables a recursive Merkle-style construction where the output of the hash itself is then again taken as input to the compressing hash⁵. This enables the hashing to be instantiated with a *fixed* compression factor⁶ independent of the circuit size, meaning unlike in the group-based setting, the size of the rekeying communication can be made independent of $|C|$ (modulo the depth dependence). This recursion introduces one additional constraint - since the output of Hash will need to be taken as input to itself, the hashing must also preserve (up to bounded polynomial factors) the distance of the additive sharing in order for us to apply distributed rounding. In other words, the coefficient $\mathbf{u}$ of Hash must be small. We can accomplish this by once again relying on bit-decomposition, this time using $\text{Bits}(\mathbf{u})$ rather than $\mathbf{u}$ as the coefficients to hash, which allows us to bound the norm of $\|\text{Hash}(\mathbf{x})\|$ relative to $\|\mathbf{x}\|$. Since bit ‘recomposition’ too is a linear operation (multiplying by $[1, 2, \dots, 2^{\log q}]$), this once again preserves the share format described above.

Putting these ideas together, our overall garbling protocol proceeds layer-by-layer over a circuit C as follows:

- On layer j of the circuit, the garbler and evaluator begin with shares $k_{\text{Hash}}^{(j)}, k_{\text{Hash}}^{(j)} + s \cdot \text{Hash}(\{w_i^{(j)}\})$, where $k_{\text{Hash}}^{(j)} \leftarrow \text{RekeySamp}(1^\lambda)$ is independently sampled.
- The garbler and evaluator run Expand to obtain shares of the form $\{k^{(j)}\}, \{k^{(j)} + s' \cdot w_i^{(j)}\}$.
- The garbler and evaluator use HGate to compute the outputs of the j^{th} layer of the circuit $\{k^{(j+1)}\}, \{k^{(j+1)} + s'' \cdot w_i^{(j+1)}\}$.
- The wire shares are then hashed down via Compress to a succinct $(k_{\text{Hash}}'^{(j+1)}, k_{\text{Hash}}'^{(j+1)} + s'' \cdot \text{Hash}(\{w_i^{(j+1)}\}))$.
- The garbler sends the evaluator $(k_{\text{Hash}}^{(j+1)} - k_{\text{Hash}}'^{(j+1)})$, the difference between the garbler’s current share of the hash and a freshly sampled one.

Since both the hash key and rekeying material are sublinear in $|C|$, this yields a succinct and low depth garbling scheme. In the setting of private garbling, this communication is dominated by the 1-bit-per-gate cost of HSS communication.

⁵In group instantiations described above, the coefficients $\mathbf{t}, \mathbf{u}$ must be hidden and hence so are the hash values $\text{Hash}(\mathbf{x}) = \mathbf{u}^T \cdot \mathbf{x}$. However, to apply recursive hashing, the evaluator must know $\text{Hash}(\mathbf{x})$ in the clear.

⁶Prior work generally instantiates this compression factor as a constant such as two. However, for our entire garbling computation to remain in NC^1 , our full constructions will require *constant* recursive depth, the compression factor is set proportional to security parameter λ .

Reusable Garbling. In the setting of reusable garbling [GKP⁺13], the garbler can produce a single garbled circuit which can be reused between multiple sets of garbled inputs. Ultimately, we can view the instantiation of reusable garbling through the homomorphic lattice encodings of [BGG⁺14] as an *approximate* additive sharing of $s^\top \cdot x \otimes G$ where the garbler holds $s^\top, B$ (which can be used to compute $s^\top \cdot B$) and the evaluator holds share $s^\top(B - x \otimes G) + e^\top$ with garbling secret s . The homomorphic evaluation procedures of [BGG⁺14, BTVW17] directly enable the independent derivation from

$$B \mapsto B_C, \quad s^\top(B - x \otimes G) + e^\top \mapsto s^\top(B_C - C(x) \otimes G) + e'^\top$$

for a circuit C . In fact, this derivation exactly corresponds to the garbler and evaluator's respective computation of HGate in the context of privacy-free garbling (and the same computation composed under a layer of FHE in the setting of private garbling). However, just like the techniques for gate evaluation in the non-reusable setting, the depth of both sides of this evaluation procedure grow with the circuit it is evaluating.

We can use the same high-level approach of rekeying and hash compression to break this symmetry in computation depth while preserving succinctness. However, the encoding of [BGG⁺14] puts two additional constraints on our techniques - (1) for the correctness of [BGG⁺14] homomorphic evaluation, the garbler and evaluator's share now *both* depend on a secret s and (2) the reusability of this garbling scheme relies on the garbler sampling a *new* s for each garbled input. Thus, the garbled circuit itself cannot be dependent on s .

While our hashing construction described above suffers from both of these issues (the ciphertexts published for Hash depend on secret s , and the Expand step produces an encoding under a different s' , which would no longer be a [BGG⁺14] encoding), we can actually directly use the key-homomorphic RAM hash of [AMR25] as the natural [BGG⁺14]-compatible analogue of our Hash protocol. This comes at the expense of relying on a stronger assumption (decomposed LWE), but otherwise retains the same structure.

We also need to modify our rekeying protocol to respect these constraints. Recall in the non-reusable setting described above, a garbler can 'rekey' an evaluator's share by simply sending them the difference between his current share k' and his freshly sampled share k . This would mean 'rekey' corresponds to $(s^\top \cdot B_{\text{Hash}} - s'^\top \cdot B'_{\text{Hash}})$, which of course is dependent on s . To avoid this, the garbler instead publishes an additional fresh matrix A in the garbled circuit which it has sampled with a short trapdoor. The rekeying key is now instead computed as a short preimage of the form $A^{-1}(B_{\text{Hash}} - B'_{\text{Hash}})$. When it comes time to garble an input x , the garbler also sends $s^\top A + e^\top$, which, when multiplied by rekeying key, recovers a noisy $(s^\top \cdot B_{\text{Hash}} - s'^\top \cdot B'_{\text{Hash}})$.

With these modifications, however we are able to obtain a succinct NC¹ garbling scheme from decomposed LWE. The same techniques also naturally extend to the setting of attribute-based encryption with succinct and NC¹ key generation.

3 Definitions

Notation We will refer to (layered) circuit $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$ computable by a poly-size and depth d circuit as a series of gates, indexed by a tuple $(i, j) \in \mathbb{N}^2$ corresponding to the depth and number of that particular gate. Each gate $g_{(i,j)}$ is of the form $(\text{op}, \text{lw}, \text{rw})$, where $\text{op} \in \{\text{AND}, \text{OR}, \text{NOT}\}$ and lw, rw are the index tuples corresponding to the left and right input wires respectively. We will use C_i to denote the collection of depth i gates and $C_{[i]}(x)$ to denote the circuit which computes all depth $\leq i$ wires of C on x .

Definition 3.1 (Garbled Circuits). Let λ be a security parameter. A garbling scheme is a tuple of efficient randomized algorithms $\Pi_{\text{garble}} = (\text{Setup}, \text{GarbC}, \text{Garblnp}, \text{Eval})$:

- $\text{Setup}(1^\lambda, 1^d) \rightarrow \text{pp}$: On input security parameter λ and circuit depth d , the Setup algorithm outputs public parameters pp .
- $\text{GarbC}(\text{pp}, C) \rightarrow (\hat{C}, \text{gk})$: On input public parameters pp and circuit C (of depth $\leq d$), the GarbC algorithm outputs garbled circuit $\hat{C}$ and garbling key gk .
- $\text{Garblnp}(\text{gk}, x) \rightarrow \hat{x}$: On input garbling key gk and input x , the Garblnp algorithm outputs a garbled input $\hat{x}$.
- $\text{Eval}(C, \hat{C}, \hat{x}) \rightarrow \{0, 1\}$: On input circuit C , garbled circuit $\hat{C}$ and garbled input $\hat{x}$, the Eval algorithm computes the evaluation of the garbling circuit.

We will consider the following syntactic and semantic properties for a garbling scheme:

- **Correctness:** For all polynomials $p(\cdot)$, $d(\cdot)$, there exists negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, all circuits $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$ of size $|C| \leq p(\lambda)$ and depth $\leq d(\lambda)$, all inputs $x \in \{0, 1\}^\ell$,

$$\Pr \left[\text{Eval}(C, \hat{C}, \hat{x}) = C(x) \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, 1^d) \\ (\text{gk}, \hat{C}) \leftarrow \text{GarbC}(\text{pp}, C) \\ \hat{x} \leftarrow \text{Garblnp}(\text{gk}, x) \end{array} \right] = 1 - \text{negl}(\lambda)$$

- **Decomposability:** We say a garbling scheme Π_{garble} is decomposable if every bit of the input garbling $\hat{x}$ depends on at most one bit of x .
- **Succinctness:** We say a garbling scheme Π_{garble} is *succinct* if there exists a polynomial p such that for every $\lambda, d \in \mathbb{N}$, circuits C with depth d , input $x \in \{0, 1\}^\ell$, and size $|C| > p(\lambda, d, \ell)$, the garbling size satisfy $|\hat{C}| + |\hat{x}| + |\text{dk}| < |C| \cdot \log |C|$.

We say a garbling scheme Π_{garble} is *depth-succinct* if there exists a polynomial p' such that for every $\lambda, d \in \mathbb{N}$, circuits C with depth d , input $x \in \{0, 1\}^\ell$, the garbling size satisfy $|\hat{C}| + |\hat{x}| + |\text{dk}| < p'(\lambda, d, \ell)$.

We note that one could consider even stronger definitions of succinctness (and general garbling) which does not a-priori bound or depend on the depth of the circuit. However, our constructions will only consider this depth-dependent case.

We will consider two different distinct ‘flavors’ of garbled circuits security, each of which will require some additional syntax and algorithms to describe. We will also consider both security definitions in the context of non-reusable and reusable garbled circuits, where changes exclusive to the reusable setting are highlighted in **green**.

- **Input-Hiding Simulation:** In the traditional notion of garbled circuits, we require the garbled circuit to ‘hide’ the input it was evaluated on. To capture this notion, a garbling scheme will also be associated with a tuple of (efficient) simulator algorithms:

- $\text{Sim}_0(1^\lambda, 1^d) \rightarrow (\text{pp}, \text{st}_0)$: On input security parameter λ and depth bound d , the zeroth simulator outputs public parameters pp and state st_0 .
- $\text{Sim}_1(\text{st}_0, C) \rightarrow (\hat{C}, \text{gk}, \text{st}_1)$: On input security prior simulator state st_0 and circuit C , the first simulator outputs garbled circuit $\hat{C}$, the garbling key gk (in the reusable case) and new simulator state st_1 .
- $\text{Sim}_2(\text{st}_1, b) \rightarrow \hat{x}$: On input simulator state st_1 and circuit output bit b , the second simulator outputs a garbled input $\hat{x}$.

For an adversary $\mathcal{A}$, define the following as the (selective) input-hiding simulation security experiment $\text{ExptSS}_{\mathcal{A}}^{(b)}(\lambda)$ as follows:

1. The adversary specifies a depth bound 1^d , a circuit C (of depth $\leq d$), and {one, **arbitrarily many**} inputs x . The challenger responds as follows:
 - If $b = 0$, the challenger computes the real garbling scheme:
 - (a) The challenger samples public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^d)$.
 - (b) The challenger samples garbled circuit $(\hat{C}, \text{gk}) \leftarrow \text{GarbC}(\text{pp}, C)$.
 - (c) The challenger samples garbled input(s) $\hat{x} \leftarrow \text{Garblnp}(\text{gk}, x)$ **for each x submitted by the adversary**.
 - If $b = 1$, the challenger computes the simulated garbling scheme:
 - (a) The challenger samples public parameters $(\text{pp}, \text{st}_0) \leftarrow \text{Sim}_0(1^\lambda, 1^d)$.
 - (b) the challenger samples garbled circuit $(\hat{C}, \text{gk}, \text{st}_1) \leftarrow \text{Sim}_1(\text{st}_0, C)$.
 - (c) The challenger samples garbled input(s) $\hat{x} \leftarrow \text{Sim}_2(\text{st}_1, C(x))$ **for each x submitted by the adversary**.

2. The challenger returns $(pp, \hat{C}, \text{gk}, \{\hat{x}\})$.
3. The adversary outputs a bit b' , which is the output of the experiment.

We note that in the reusable setting, the adversary gets two additional ‘powers’ - it can see the garbling key gk and request as many garbled input labels as desired, while in the non-reusable setting, an adversary makes exactly one such query. We say a scheme Π_{garble} is input-hiding simulation secure if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr[\text{ExptSS}_{\mathcal{A}}^{(0)}(\lambda) = 1] - \Pr[\text{ExptSS}_{\mathcal{A}}^{(1)}(\lambda) = 1] \right| = \text{negl}(\lambda).$$

- **Authenticity:** An orthogonal property of garbled circuits that could be desirable even in the absence of input hiding is the notion of label *authenticity*. This is the main security property of interest in the setting of privacy-free garbling [FNO15] where the garbled circuit instead simply represents a way to authenticate computation. To capture this requirement, we additionally assume our garbling algorithms have the following structure:

- $\text{Garblnp}(\text{gk}, x) \rightarrow \hat{x} = (L_x, \text{dk})$, auk : In the input garbling algorithm, let the garbled input $\hat{x}$ be decomposed into two part - a garbled input label L_x and a decoding key dk . In addition, Garblnp can implicitly compute a authentication key auk .
- $\text{Eval}(C, \hat{C}, \hat{x}) \rightarrow \{0, 1\}$: The evaluation algorithm can then be decomposed into two steps:
 - * $\text{EvalLabel}(C, \hat{C}, L_x) \rightarrow L_{\text{out}}$: First, the EvalLabel algorithm takes the garbled circuit $\hat{C}$ as well as the garbled input labels L_x and computes an output label L_{out} .
 - * $\text{DecodeLabel}(\text{dk}, L_{\text{out}}) \rightarrow \{0, 1\} \cup \perp$: Once the output label is computed, a second DecodeLabel algorithm applies the decoding key dk to the label, which outputs a bit (which is also the output of Eval).
- $\text{Authenticate}(\text{auk}, L_{\text{out}}) \rightarrow \{0, 1\}$: Privacy-free garbling is also equipped with an ‘authentication’ algorithm, which takes in authentication key auk and a label L_{out} and outputs a bit, which is 1 on any label generated by the honest execution of Garblnp , EvalLabel .

Now we can define the (selective) authenticity game of a garbling scheme with respect adversary $\mathcal{A}$ through the following security $\text{ExptAuth}_{\mathcal{A}}(\lambda)$:

1. The adversary specifies a depth bound 1^d , a circuit C (of depth $\leq d$), and one input x^* . The challenger computes the following:
 - (a) The challenger samples public parameters $pp \leftarrow \text{Setup}(1^\lambda, 1^d)$.
 - (b) The challenger samples garbled circuit $(\hat{C}, \text{gk}) \leftarrow \text{GarbC}(pp, C)$.
 - (c) The challenger samples garbled input $\hat{x}^* \leftarrow \text{Garblnp}(\text{gk}, x^*)$, as well as accompanying authentication key auk^* .
2. The challenger returns $(pp, \hat{C}, \text{gk}, \hat{x}^* = L_{x^*}, \text{dk}^*)$.
3. The adversary submits a label L^* . The output of the experiment is 1 iff $1 - C(x^*) = \text{DecodeLabel}(\text{dk}^*, L^*)$ and $\text{Authenticate}(\text{auk}^*, L^*) = 1$.

As in the input-hiding simulation security, the setting of reusable authenticity enables the adversary to see the garbling key gk . As a result, they can generate as many garbled inputs it desires. In either case, we say a scheme Π_{garble} satisfies authenticity if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr[\text{ExptAuth}_{\mathcal{A}}(\lambda) = 1] = \text{negl}(\lambda).$$

3.1 Garbled Circuits with Algebraic Output Labels

Our constructions of private and non-private garbling will all conform to a similar high-level algebraic structure. Below, we describe this structure of our garbled circuits and some properties which are equivalent to the traditional simulation security/authenticity definitions in garbling, which is what our constructions will directly show.

Definition 3.2 (Algebraic Garbled Circuit). We say a garbling scheme Π_{garble} with syntax as [Definition 3.1](#) above is algebraic over a ring $\mathcal{R}$ if it takes the following form:

- **Algebraic Labeling:** The garbling scheme will be associated with an integer ring $\mathcal{R} = \mathbb{Z}$ or $\mathbb{Z}_q$ along with parameter n where $n \in \text{poly}(\lambda)$, along with a distribution $\mathcal{D}^{(\mathcal{R})}$ over $\mathcal{R}^{n+1}$. We let $\mathcal{D}_{n+1}^{(\mathcal{R})}$ denote the marginal distribution of the last entry of $\mathcal{D}^{(\mathcal{R})}$. We require $\mathcal{D}_{n+1}^{(\mathcal{R})} = 0$ with only negligible probability. The garbling algorithms can be broken down as follows (inheriting the additional syntax and structure of the authenticity definition):
 - The input garbling algorithm Garblnp can be broken down into the following steps:
 - * Sample $\mathbf{s}^{(\text{out})} \xleftarrow{\mathcal{R}} \mathcal{D}$.
 - * $\text{Garblnp}'(\text{gk}, x, \mathbf{s}^{(\text{out})})$ returns output ‘tag’ $\mathbf{r}^{(\text{out})}$ and garbled input labels L_x .
 - * Set decoding key dk as $r_{n+1}^{(\text{out})}$, the last entry of $\mathbf{r}^{(\text{out})}$, and authentication key auk as vectors $(\mathbf{r}^{(\text{out})}, \mathbf{s}^{(\text{out})})$.
 - Decomposing Eval into an EvalLabel which computes a label of the form

$$L_{\text{out}} = \mathbf{r}^{(\text{out})} + \mathbf{s}^{(\text{out})} \cdot C(x)$$

where and a decoding step. Recall $\text{dk} = r_n^{(\text{out})}$, so decoding can proceed by simply computing

$$\text{dk} - L_{\text{out}, n+1} = r_{n+1}^{(\text{out})} - r_{n+1}^{(\text{out})} - s_{n+1}^{(\text{out})} \cdot C(x).$$

Since $s_{n+1}^{(\text{out})}$ is nonzero with overwhelming probability, the Eval outputs 0 iff this quantity is 0.

- The Authenticate algorithm, which takes in $\text{auk} = (\mathbf{r}^{(\text{out})}, \mathbf{s}^{(\text{out})})$, returns 1 on label L if $L = \mathbf{r}^{(\text{out})}$ or $L = \mathbf{r}^{(\text{out})} + \mathbf{s}^{(\text{out})}$.
- **One-Wayness:** We define the one-wayness as the algebraic analogue to authenticity. The one-wayness experiment for garbled circuits ExptOW proceeds as follows:
 1. The adversary specifies a depth bound 1^d , a circuit C (of depth $\leq d$), and an input x^* . The challenger computes the following:
 - (a) The challenger samples public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^d)$.
 - (b) The challenger samples garbled circuit $(\hat{C}, \text{gk}) \leftarrow \text{GarbC}(\text{pp}, C)$.
 - (c) The challenger samples garbled input $\hat{x}^* \leftarrow \text{Garblnp}(\text{gk}, x^*)$, as well as accompanying authentication key auk^* using secret randomness $\mathbf{s}^*$.
 2. The challenger returns $(\text{pp}, \hat{C}, \text{gk}, \hat{x}^*)$.
 3. The adversary submits a guess $\mathbf{s}'$. The output of the experiment is 1 iff $\mathbf{s}' = \mathbf{s}^*$.

We say a scheme Π_{garble} satisfies one-wayness if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr[\text{ExptOW}_{\mathcal{A}}(\lambda) = 1] = \text{negl}(\lambda).$$

- **Semantic Security:** We define the (selective) semantic security experiment for garbled circuits $\text{ExptIND}^{(b)}$ as follows. This property has also previously been referred to as obliviousness [[BHR12](#)]:

1. The adversary specifies a depth bound 1^d , a circuit C (of depth $\leq d$), and a pair of inputs x_0, x_1 . The challenger computes the following:

- (a) The challenger samples public parameters $pp \leftarrow \text{Setup}(1^\lambda, 1^d)$.
- (b) The challenger samples garbled circuit $(\hat{C}, \text{gk}) \leftarrow \text{GarbC}(pp, C)$.
- (c) The challenger samples garbled input $\hat{x} \leftarrow \text{Garblnp}(\text{gk}, x_b)$, which it parses as $\hat{x} = (L_x, \text{dk})$.
2. The challenger returns $(pp, \hat{C}, \text{gk}, L_x)$.
3. The adversary submits a bit b' which is the output of the experiment.

We say a scheme Π_{garble} satisfies semantic security if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr[\text{ExptIND}_{\mathcal{A}}^{(1)}(\lambda) = 1] - \Pr[\text{ExptIND}_{\mathcal{A}}^{(0)}(\lambda) = 1] \right| = \text{negl}(\lambda).$$

It is easy to see that any garbling scheme satisfying the structure of algebraic labeling will naturally satisfy correctness. We show below that the normal garbling properties of authenticity and input-hiding simulation security also follow from one-wayness and semantic security respectively in this context. Our reductions below follow the adaptive versions of the definitions, but the selective reductions of one-wayness to authenticity and semantic security to simulation security follow analogously.

Lemma 3.3. *Suppose an algebraic garbling scheme Π_{garble} satisfies one-wayness. Then it is also satisfies authenticity as a privacy-free garbling scheme.*

Proof. Suppose $\mathcal{A}$ is an adversary against the one-wayness of Π_{garble} which succeeds with probability ε . Then consider the following reduction $\mathcal{B}$ against the authenticity of Π_{garble} :

0. Reduction $\mathcal{B}$ begins running $\mathcal{A}$.
1. Adversary $\mathcal{A}$ submits a depth bound 1^d a circuit C , and an input x^* , which $\mathcal{B}$ simply forwards to the challenger.
2. Reduction $\mathcal{B}$ receives $(pp, \hat{C}, \text{gk}, \hat{x}^* = (L_{x^*}, \text{dk}_{x^*}))$, which it simply forwards to $\mathcal{A}$.
3. Adversary $\mathcal{A}$ submits a label $L_{1-C(x^*)}$. Reduction $\mathcal{B}$ computes label $L_{C(x^*)} \leftarrow \text{EvalLabel}(C, \hat{C}, L_{x^*})$ and returns $s' = L_1 - L_0$.

Recall for $\mathcal{A}$ to succeed, adversary $\mathcal{A}$ must submit a label L which both authenticates and decodes to $1 - C(x)$. For the former to be true, $L = \mathbf{r}^{(\text{out})} + b \cdot \mathbf{s}^*$ for $b \in \{0, 1\}$. This can be decomposed into two cases:

- $L = \mathbf{r}^{(\text{out})} + C(x) \cdot \mathbf{s}^*$: In this case, recall that since $\text{dk} = r_n^{(\text{out})}$, this will decode to $C(x)$, and the adversary did not win the authenticity game.
- $L = \mathbf{r}^{(\text{out})} + (1 - C(x)) \cdot \mathbf{s}^*$: In this case, recall that by definition the output L_{out} of EvalLabel is $\mathbf{r}^{(\text{out})} + C(x) \cdot \mathbf{s}^*$. Thus we can compute $\mathbf{s}^*$ simply as their difference, and our reduction $\mathcal{B}$ wins the one-wayness game.

□

Lemma 3.4. *Suppose an algebraic garbling scheme Π_{garble} satisfies semantic security. Then it is also satisfies input-hiding simulation security as a private garbling scheme.*

Proof. We define the private garbling simulators $\text{Sim}_0, \text{Sim}_1, \text{Sim}_2$ as follows:

- $\text{Sim}_0(1^\lambda) \rightarrow (pp, \text{st}_0)$: On input security parameter λ , the zeroth simulator simply runs $pp \leftarrow \text{Setup}(1^\lambda)$ and sets both outputs to pp .
- $\text{Sim}_1(\text{st}_0 = pp, C) \rightarrow (\hat{C}, \text{gk}, \text{st}_1)$: On input security parameter λ and circuit C , the first simulator simply runs $(\hat{C}, \text{gk}) \leftarrow \text{GarbC}(pp, C)$, and outputs $(\hat{C} = \hat{C}, \text{gk} = \text{gk}, \text{st}_1 = \text{gk})$.
- $\text{Sim}_2(\text{st}_1, C(x)) \rightarrow (L_x, \text{dk})$: On input trapdoor $\text{td} = \text{gk}$ and circuit output $C(x)$, the second simulator samples $s_{n+1}^{(\text{out})} \xleftarrow{\mathcal{R}} \mathcal{D}_{n+1}^{(\mathcal{R})}$, runs $\text{Garblnp}(\text{gk}, 0) \rightarrow L_0$, and outputs $L_x = L_0$ and $\text{dk} = L_{0,n+1} - s_{n+1}^{(\text{out})} \cdot C(x)$, where $L_{0,n+1}$ is the last entry of L_0 .

Now, we can argue simulation security through a sequence of hybrids:

- Hyb_0 : This is the original input-hiding simulation security experiment when $b = 0$.
 1. The adversary specifies a depth bound 1^d , a circuit C (of depth $\leq d$), and $\{\text{one, arbitrarily many}\}$ inputs x . The challenger responds as follows:
 - (a) The challenger samples public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^d)$.
 - (b) The challenger samples garbled circuit $(\hat{C}, \text{gk}) \leftarrow \text{GarbC}(\text{pp}, C)$.
 - (c) The challenger samples garbled input(s) $\hat{x} = (L_x, \text{dk}_x) \leftarrow \text{Garblnp}(\text{gk}, x)$ for each x submitted by the adversary.
 2. The challenger returns $(\text{pp}, \hat{C}, \text{gk}, \{\hat{x}\})$.
 3. The adversary outputs a bit b' , which is the output of the experiment.
- Hyb_1 : In this experiment, we sample switch to computing $\text{dk} = L_{x, n+1} - C(x)$.
 - 1.(c) The challenger samples garbled input(s) $\hat{x} = (L_x, \text{dk}_x) \leftarrow \text{Garblnp}(\text{gk}, x)$ and samples $s_{n+1}^{(\text{out})} \xleftarrow{\mathcal{R}} \mathcal{D}_{n+1}^{(\mathcal{R})}$ to set $\text{dk}'_x = L_{x, n+1} - s_{n+1}^{(\text{out})} \cdot C(x)$ for each x submitted by the adversary.
 2. The challenger returns $(\text{pp}, \hat{C}, \text{gk}, \{(L_x, \text{dk}'_x)\})$.
- Hyb_2 : In this experiment, we switch to computing $L_x \leftarrow \text{Garblnp}(\text{gk}, 0)$.
 - 1.(c) The challenger samples garbled input(s) $\hat{x} = (L_x, \text{dk}_x) \leftarrow \text{Garblnp}(\text{gk}, 0)$ and samples $s_{n+1}^{(\text{out})} \xleftarrow{\mathcal{R}} \mathcal{D}_{n+1}^{(\mathcal{R})}$ to set $\text{dk}'_x = L_{x, n+1} - s_{n+1}^{(\text{out})} \cdot C(x)$ for each x submitted by the adversary.

Claim 3.5. For all adversaries $\mathcal{A}$ and all $\lambda \in \mathbb{N}$,

$$\Pr [\text{Hyb}_0(\mathcal{A}, \lambda) = 1] = \Pr [\text{Hyb}_1(\mathcal{A}, \lambda) = 1].$$

Proof. By definition of algebraic labeling, these are sampled identically. $\square$

Claim 3.6. Suppose Π_{garble} is semantically secure. Then for all efficient adversaries $\mathcal{A}$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$|\Pr [\text{Hyb}_1(\mathcal{A}, \lambda) = 1] - \Pr [\text{Hyb}_2(\mathcal{A}, \lambda) = 1]| = \text{negl}(\lambda).$$

Proof. Suppose an adversary $\mathcal{A}$ distinguishes Hyb_1 and Hyb_2 with noticeable probability ϵ . Then the following is a reduction $\mathcal{B}$ against the semantic security of Π_{garble} . For simplicity, we present the case where Π_{garble} is non-reusable, but this extends to the reusable garbling case through a simple hybrid argument.

0. Reduction $\mathcal{B}$ begins running $\mathcal{A}$.

1. Adversary $\mathcal{A}$ specifies a depth bound 1^d , a circuit C (of depth $\leq d$), and an input x^* . Reduction $\mathcal{B}$ forwards $1^d, C$, and inputs $x_0 = x^*, x_1 = x$ to the challenger. $\mathcal{B}$ then samples $s_{n+1}^{(\text{out})} \xleftarrow{\mathcal{R}} \mathcal{D}_{n+1}^{(\mathcal{R})}$ and sets $\text{dk}'_x = L_{x, n+1} - C(x)$, returning $(\text{pp}, \hat{C}, \hat{x} = (L_x, \text{dk}'_x))$ to the adversary.

2. The adversary submits a bit b' which is the output of the experiment.

We can see that when L_x is a garbled input label of x^* , the adversary's view is exactly that in Hyb_1 , and similarly the when L_x is a garbled input label of 0 this is the adversary's view in Hyb_2 . Thus the advantage of $\mathcal{B}$ in the semantic security game is exactly that of $\mathcal{A}$ in distinguishing $\text{Hyb}_1, \text{Hyb}_2$. $\square$

Since Hyb_2 is exactly the adversary's view when receiving simulated garbled inputs per $(\text{Sim}_0, \text{Sim}_1, \text{Sim}_2)$ and depends only on the garbled input evaluation $C(x)$, these claims complete our proof of the lemma. $\square$

4 A General Template for Low-Depth Succinct Garbling

We will take the view of [HK21, HKO22, ILL25b] presenting the (GarbC, GarbInp, Eval) algorithms of a garbling scheme as a two-party protocol between a garbler P_G and evaluator P_E , who jointly hold an encoding of input x and homomorphically compute an encoding of $C(x)$ with one directional communication from the garbler to evaluator (representing the output of GarbC, GarbInp). This protocol roughly can be decomposed into two algorithms Setup, Enc run by P_G , and a subprotocol HGate jointly run by P_G, P_E .

- $\text{Setup}(1^\lambda, 1^d, 1^{|C|}) \rightarrow \text{pp}$: The setup algorithm takes in security parameter λ , along with circuit parameters (in our case depth d and (in some instances) circuit size $|C|$), and generates public data pp , which should be sent to P_E and be accessible for all following algorithms and sub-protocols.
- $\text{Enc}(x) \rightarrow \llbracket x \rrbracket, \text{pd}$: The encoding algorithm takes as input the public data pd and a value x and outputs a two-party encoding $\llbracket x \rrbracket = (\llbracket x \rrbracket_G, \llbracket x \rrbracket_E)$ of x , public data pd . The encoding $\llbracket x \rrbracket_E$ and public data pd should be sent to P_E . Like pp , we will assume the public data pd is implicitly taken as input and available to all following algorithms and sub-protocols.
- $\text{HGate}(\text{op}, \llbracket x \rrbracket, \llbracket y \rrbracket) \rightarrow \llbracket \text{op}(x, y) \rrbracket$: The homomorphic gate evaluation protocol takes as input a gate op and input encodings $\llbracket x \rrbracket, \llbracket y \rrbracket$ (jointly held by P_G, P_E) to compute an encoding of the output encoding $\llbracket \text{op}(x, y) \rrbracket$ (jointly held by P_G, P_E). In some instances (in particular when relying on homomorphic secret sharing) this could require garbler to evaluator communication.

Once an encoding of $y = C(x)$ has been computed, the shares y_G and y_E can roughly be thought of as the decoding key and output label respectively. To make this general garbling framework low-depth while still retaining succinctness, we can retain the prior encoding Enc and homomorphic evaluation HGate sub-protocols while making use of three additional sub-protocols, a rekeying operation Rekey and a homomorphic HRead, HCompress, the latter two of which are defined with respect to a homomorphic Hash function. These protocols have the following properties:

- $\text{Rekey}(1^\lambda, \llbracket x \rrbracket = (\llbracket x \rrbracket_G, \llbracket x \rrbracket_E)) \rightarrow \llbracket x \rrbracket' = (\llbracket x \rrbracket'_G, \llbracket x \rrbracket'_E)$: The rekeying operation will take a joint encoding of a value x and transform it into a new *fresh* encoding of the same value x . Importantly, the garbler's new share of the encoding $\llbracket x \rrbracket'_G$ will be *independent* of the input. To capture this requirement, we will decompose the rekeying protocol into the following steps:
 - $\text{RekeySamp}(1^\lambda) \rightarrow \llbracket x \rrbracket'_G$: The garbler locally samples a fresh share $\llbracket x \rrbracket'_G$.
 - $\text{RekeyGen}(\llbracket x \rrbracket_G, \llbracket x \rrbracket'_G) \rightarrow \text{rk}$: Using the freshly generated share $\llbracket x \rrbracket'_G$, as well as the original input encoding $\llbracket x \rrbracket_G$, the garbler generates a rekeying 'hint' rk , which is sent to the evaluator.
 - $\text{RekeyEval}(\llbracket x \rrbracket_E, \text{rk}) \rightarrow \llbracket x \rrbracket'_E$: The evaluator, using their prior encoding share $\llbracket x \rrbracket_E$ and the rekey rk , computes their new encoding $\llbracket x \rrbracket'_E$.

Such a rekeying immediately enables a low depth garbler by allowing the garbler to generate their shares of each layer of the circuit C independently, then in parallel using HGate to evaluate the wires of each layer, before sending rekeying hints rk to the evaluator to 'bridge' the layers of the circuit.

Unfortunately, in our realisations of such rekeying protocols, the length of rk , and hence the communication between P_G and P_E (which corresponds to the size of the garbled circuit), grows with the length of the value x which needs to be rekeyed. Since we will need to rekey every wire in our circuit, applying this idea naively results in a $\text{poly}(\lambda) \cdot |C|$ overhead, losing any notion of succinctness in the resulting garbled circuit.

- $\text{Hash}(x) \rightarrow h_x$: To compress the communication of our garbling protocol, instead of attempting to rekey the wires of circuit directly, we instead rekey a succinct *hash* of our wire labels, while simultaneously ensuring our hash retains enough structure to execute the layer-by-layer homomorphic computation on the underlying encoding.

- $\text{HRead}(\llbracket h_x \rrbracket, i, P_E : \llbracket x \rrbracket_E) \rightarrow \llbracket x_i \rrbracket$: Using an encoding of the hash h_x , along with an index i , the parties can derive an encoding of the i^{th} bit of x . To enable this sub-protocol on the evaluator's side, they will also need their share of the encoding of the original input $\llbracket x \rrbracket_E$. More concretely, the HRead step will require the evaluator to know (in the clear) the input to a linear hash function underlying Hash, which are simply wire values in the case of privacy-free garbling and one time pad encryptions of wire values in the case of private-garbling.
- $\text{HCompress}(\llbracket x \rrbracket) \rightarrow (\llbracket h_x \rrbracket)$: Given encodings of input values $\llbracket x \rrbracket$, the parties can derive an encoding of h_x , the hash of string x . The computational depth of HCompress will depend *logarithmically* on the length of x .

Both HRead and HCompress will be able to be executed *noninteractively*.

Putting these together, we can realise a general framework for low-depth succinct garbling which will proceed as follows:

Inputs: P_G holds input x , circuit C , while P_E just holds circuit C .

Outputs: P_G outputs a decoding key dk , while P_E outputs a label L .

1. **Input Encoding Phase:** Party P_G samples public parameters $pp \leftarrow \text{Setup}(1^\lambda, 1^d, 1^{|C|})$ and generates encoding and public data $(\llbracket x \rrbracket, pd, st)$, where $\llbracket x \rrbracket = (\llbracket x \rrbracket_G, \llbracket x \rrbracket_E) \leftarrow \text{Enc}(gk, x)$, sending $pd, \llbracket x \rrbracket_E$ to P_E .
2. **Homomorphic Evaluation:** The parties firstly jointly use HGate to evaluate the hash function Hash, obtaining an encoding $\llbracket h_x \rrbracket$. Then, dividing the circuit C into layers $C_1, \dots, C_d$, P_G and P_E iteratively compute the following for $i \in [d]$:
 - We maintain the invariant that for all $i \in [d]$ computed thus far, $h^{(i)}$ is the hash of all of the output wires $w^{(i)}$ of the i^{th} layer of circuit C , with $h_x = h^{(0)}$.
 - For each gate $j \in |C_i|$, the i^{th} layer of circuit C , take gate $g_{(i,j)} = (op_j, lw_j = (ll_j, li_j), rw_j = (rl_j, ri_j)) \in C_i$, where ll, rl indicates the “depth” of the left and right input wires, and li, ri indicate their indices. Then compute $\llbracket w_{lw_j}^{(i)} \rrbracket \leftarrow \text{HRead}(\llbracket h^{(ll_j)} \rrbracket, li_j, P_E : \llbracket w^{(ll_j)} \rrbracket_E)$ and $\llbracket w_{rw_j}^{(i)} \rrbracket \leftarrow \text{HRead}(\llbracket h^{(rl_j)} \rrbracket, ri_j, P_E : \llbracket w^{(rl_j)} \rrbracket_E)$.
 - For all gates $g_{(i,j)} \in C_i$, compute $\llbracket w_j^{(i)} \rrbracket \leftarrow \text{HGate}(op_j, \llbracket w_{lw_j}^{(i)} \rrbracket, \llbracket w_{rw_j}^{(i)} \rrbracket)$
 - Compute encoding $\llbracket h^{(i)} \rrbracket' \leftarrow \text{HCompress}(\{\llbracket w_j^{(i)} \rrbracket\}_{j \in |C_i|})$.
 - Rekey encoding with Rekey($\llbracket h^{(i)} \rrbracket' \rrbracket \rightarrow \llbracket h^{(i)} \rrbracket$.
3. **Output Phase:** Let j_{out} be the index of the output gate. Derive output encoding $\llbracket y \rrbracket = (\llbracket y \rrbracket_G, \llbracket y \rrbracket_E) \leftarrow \text{HRead}(\llbracket h^{(d)} \rrbracket, \llbracket w^{(d)} \rrbracket, j_{\text{out}})$. The garbler and evaluator derive dk, L from $\llbracket y \rrbracket_G$ and $\llbracket y \rrbracket_E$ respectively.

4.1 Garbling Depth

A garbling procedure following the above-outlined template naturally yields the following parallelization of the garbler's computation.

- **Input Encoding Phase:** Compute as above.
- **Homomorphic Evaluation:** Begin by computing $\llbracket h_x \rrbracket_G = \llbracket h^{(0)} \rrbracket_G$ as above. *In parallel*, for $i \in [d]$, perform the following:
 - Sample layer key $\text{RekeySamp}(1^\lambda) \rightarrow \llbracket h^{(i)} \rrbracket_G$.
 - For all gates $g_j = (op_j, lw_j, rw_j) \in C_i$, the garbler computes $\llbracket w_{lw_j}^{(i)} \rrbracket_G \leftarrow \text{HRead}(\llbracket h^{(ll_j)} \rrbracket, li_j)$ and $\llbracket w_{rw_j}^{(i)} \rrbracket \leftarrow \text{HRead}(\llbracket h^{(rl_j)} \rrbracket, ri_j)$ ⁷.

⁷Recall that only the evaluator needs input encodings to execute HRead, while the garbler only needs their share of the hash and the index.

- For all gates $g_j \in C_i$, compute $\llbracket w_j^{(i)} \rrbracket_G \leftarrow \text{HGate}(\text{op}_j, \llbracket w_{lw_j}^{(i)} \rrbracket_G, \llbracket w_{rw_j}^{(i)} \rrbracket_G)$
- Compute encoding $\llbracket h^{(i)} \rrbracket'_G \leftarrow \text{HCompress}(\{\llbracket w_j^{(i)} \rrbracket_G\}_{j \in [C_i]})$.
- In parallel for each $i \in [d]$, compute $\text{RekeyGen}(\llbracket h^{(i)} \rrbracket', \llbracket h^{(i)} \rrbracket) \rightarrow \text{rk}_i$, which is (in the reusable case), output as part of gk .
- **Output Phase:** Compute as above.

We can see from this rearrangement that any garbling scheme which can be decomposed into our template will naturally have garbling depth independent of the circuit depth C , modulo any depth dependence incurred by the setup or encoding procedures, which we will show is independent or logarithmic for all of our constructions. In fact, in some instantiations, our succinct garbling schemes will be computable in depth $O(\log(\lambda, |C|))$, i.e. NC^1 .

5 Non-Reusable Succinct Privacy-Free Garbling

In this section, we present our constructions of non-reusable succinct privacy-free garbling, which can be summarized in the following theorem:

Theorem 5.1 (Non-Reusable Succinct Privacy-Free Garbling under DDH). *Assuming any of the following:*

- DDH in prime-order groups that are subgroups of $\mathbb{Z}_p^*$ for some prime p ;
- DDH in Paillier groups;

There exists a succinct privacy-free garbling scheme (i.e., satisfying only authenticity in Definition 3.1) satisfying the following.

- Garbling sizes of a circuit $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$ with depth D satisfy:

$$|\hat{C}| < (D + |C|^{2/3} \cdot D^{1/3}) \cdot \text{poly}(\lambda), |\hat{x}| < \ell \cdot \text{poly}(\lambda), |\text{dk}| < \text{poly}(\lambda).$$

- $\text{GarbC}(\text{pp}, C)$ can be computed in NC^1 .

Theorem 5.2 (Non-Reusable Succinct Privacy-Free Garbling under RLWE). *Assuming RLWE with respect to $\text{pp} = (\mathcal{R}, p, q, \mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}})$ as specified in Construction 5.28 under $B = \lambda^{\omega(1)}$, there exists a depth-succinct privacy-free garbling scheme satisfying the following.*

- Garbling sizes of a circuit $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$ with depth D satisfy:

$$|\hat{C}| < D \cdot \text{poly}(\lambda), |\hat{x}| < \ell \cdot \text{poly}(\lambda), |\text{dk}| < \text{poly}(\lambda).$$

- $\text{GarbC}(\text{pp}, C)$ can be computed in NC^1 .

In the following, we give preliminaries in Section 5.1, show the group instantiation in prime-order groups in Section 5.2 and show the lattice instantiation in Section 5.3. We omit the construction in Paillier groups, as it is analogous to that in prime-order groups.

5.1 Preliminaries

We consider two types of groups, prime-order groups and Paillier groups (of composite orders) and in this section. We provide a quick review of these groups and the standard DDH assumption in them.

Definition 5.3 (Prime-order Groups). We consider prime-order groups defined by an instance generation algorithm Gen with the following syntax.

- $\text{Gen}(1^\lambda)$ outputs (G, p, g) where G is a group description of prime order $p > 2^\lambda$, and g is a generator of G .

Definition 5.4 (DDH Assumption). We say DDH holds in prime-order groups if the following holds:

$$\left\{ \text{pp}, g, g^a, g^b, g^{ab} \mid \text{pp} = (G, p, g) \leftarrow \text{DDH.Gen}(1^\lambda), \begin{matrix} a, b \leftarrow \mathbb{Z}_p. \end{matrix} \right\}_\lambda$$

$$\approx_c \left\{ \text{pp}, g, g^a, g^b, g^c \mid \text{pp} = (G, p, g) \leftarrow \text{DDH.Gen}(1^\lambda), \begin{matrix} a, b, c \leftarrow \mathbb{Z}_p. \end{matrix} \right\}_\lambda.$$

We will use the distributed discrete log (DDLog) algorithms [BGI16, DKK18, RS21, OSY21] for prime-order groups and Paillier groups respectively.

Lemma 5.5 (Distributed Discrete Log for Prime-Order Groups [BGI16, DKK18]). *For a cyclic group G with order p and a generator g , an error bound $\delta \in (0, 1]$, a message bound B , there exists an algorithm $\text{DDLog}_{G,g}^{\delta,B}$:*

- $\text{DDLog}_{G,g}^{\delta,B}(\varphi : G \rightarrow \{0, 1\}^{\lceil \log(2B/\delta) \rceil}, a \in G)$ takes a function φ mapping group elements to bit strings, and an element a . It outputs a value $\alpha \in \mathbb{Z}_p$.

The algorithm requires $O(\sqrt{B/\delta})$ group operations and evaluations of φ , and has the guarantee that for all $0 < \delta \leq 1$, $B < p$, $a \in G$, and $m \leq B$:

$$\Pr \left[\begin{matrix} \text{DDLog}_{G,g}^{\delta,B}(\varphi, a \cdot g^m) \\ = \text{DDLog}_{G,g}^{\delta,B}(\varphi, a) + m \bmod p \end{matrix} \mid \varphi \leftarrow \$ \right] \geq 1 - \delta,$$

where $\varphi \leftarrow \$$ means sampling at random from all possible mappings.

Remark 5.6. We note that the DDLog algorithm can be implemented in depth $O(\log \lambda)$, assuming $B/\delta < \text{poly}(\lambda)$, φ can be evaluated in depth $O(\log \lambda)$. In our constructions using prime-order groups, we instantiate (pseudo)random mappings φ via a PRF : $\{0, 1\}^\lambda \times [2^\lambda] \times G \rightarrow \{0, 1\}^\lambda$. In more detail, the parties hold a common random key $\leftarrow \{0, 1\}^\lambda$, and maintain a counter $c \in [2^\lambda]$. Every invocation of DDLog is instantiated with $\varphi(\cdot) := \text{PRF}(\text{key}, c, \cdot)$ (with suitably truncated outputs) for distinct counter values c . Indeed, under the DDH assumption in prime-order subgroups of $\mathbb{Z}_{p'}^*$, for primes p' , there exists such PRFs with depth $O(\log \lambda)$ [NR97].

Definition 5.7 (Paillier Groups). Paillier groups are defined by the following instance generation algorithm Gen .

- $\text{Gen}(1^\lambda, 1^\zeta)$ uniformly samples two λ -bit primes p, q such that $p = 2p' + 1$, $q = 2q' + 1$ where p', q' are also primes. It outputs $(N = pq, \zeta)$ as the group description of $G = \mathbb{Z}_{N^{\zeta+1}}^*$.

Lemma 5.8 (Facts about Paillier Groups [Pai99, DJ01]). *Let $G = \mathbb{Z}_{N^{\zeta+1}}^*$ be a Paillier group sampled by $\text{Gen}(1^\lambda, 1^\zeta)$ for a polynomial $\zeta(\lambda)$.*

- G has a subgroup $F = \{(1 + N)^x : x \in \mathbb{N}^\zeta\}$ where discrete log (i.e., finding x) can be efficiently solved.
- G has a subgroup H that's isomorphic to $\mathbb{Z}_N^*$, and $G = F \times H$.
- Consider a random element $g \in G$ such that the Jacobi symbol of $g \bmod N$ is 1. Then $\langle g \rangle$ contains F except with negligible probability. We write $g \leftarrow \text{Samp}(N, \zeta)$ to mean sampling such elements g with Jacobi symbol 1.

The following DDH assumption in Paillier groups is adapted from the formulation by [ADOS22], where the authors formulate a separate “small exponent” assumption stating it's secure to sample the secret exponents in DDH from a smaller, but still sufficiently large, range than the order of g . We directly state the small-exponent variant of DDH in Paillier groups here.

Definition 5.9 (DDH in Paillier Groups). We say DDH holds in Paillier groups if the following holds for every polynomial $\zeta(\lambda)$:

$$\left\{ \text{pp}, g, g^a, g^b, g^{ab} \mid \begin{array}{l} \text{pp} = (N, \zeta) \leftarrow \text{DCR.Gen}(1^\lambda, 1^\zeta), \\ g \leftarrow \text{DCR.Samp}(\text{pp}), a, b \leftarrow [N]. \end{array} \right\}_\lambda \\ \approx_c \left\{ \text{pp}, g, g^a, g^b, g^c \mid \begin{array}{l} \text{pp} = (N, \zeta) \leftarrow \text{DCR.Gen}(1^\lambda, 1^\zeta), \\ g \leftarrow \text{DCR.Samp}(\text{pp}), a, b, c \leftarrow [N^{\zeta+2}]. \end{array} \right\}_\lambda.$$

Lemma 5.10 (Distributed Discrete Log for Paillier Groups [RS21, OSY21]). For a Pailler group $G = \mathbb{Z}_{N^{\zeta+1}}^*$, there exists an algorithm $\text{DDLog}_{N, \zeta}$:

- $\text{DDLog}_{N, \zeta}(a \in G)$ takes an element a and outputs a value $\alpha \in \mathbb{Z}_p$.

The algorithm has the guarantee that for all $a \in G$ and $m \in [N^\zeta]$:

$$\Pr \left[\begin{array}{l} \text{DDLog}_{N, \zeta}(a \cdot (1 + N)^m) \\ = \text{DDLog}_{N, \zeta}(a) + m \bmod N^\zeta \end{array} \right] = 1.$$

Remark 5.11. We note that the DDLog algorithm can be implemented in depth $O(\log \lambda)$, assuming $\log N < \text{poly}(\lambda)$ and $\zeta < O(1)$, which suffices for our construction.

We also recall the ring LWE (RLWE) assumption over polynomial rings of the form $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$, where $n(\lambda)$ is a power-of-2.

Definition 5.12 (RingLWE [LPR10]). We say the RLWE assumption holds with respect to the ring $\mathcal{R}(\lambda)$, a modulus $q(\lambda)$, error and secret distributions $\mathcal{D}_{\text{err}}(\lambda)$, $\mathcal{D}_{\text{sk}}(\lambda)$ if the following holds for every polynomial $m(\lambda)$:

$$\left\{ \begin{array}{l} \mathbf{a}, s \cdot \mathbf{a} + \mathbf{e}_1, \\ (\text{over } \mathcal{R}_q) \end{array} \mid \begin{array}{l} s \leftarrow \mathcal{D}_{\text{sk}}, \mathbf{e}_1 \leftarrow \mathcal{D}_{\text{err}}^m, \\ \mathbf{a} \leftarrow \mathcal{R}_q^m \end{array} \right\}_\lambda \approx_c \left\{ \mathbf{a}, \mathbf{b} \leftarrow \mathcal{R}_q^m \right\}_\lambda$$

5.2 Instantiation Under Groups

Our instantiations under prime-order groups and Pailler groups are analogous. Hence we focus on the prime-order groups in this section.

The starting point of our group instantiation is the DDH-based aHMAC scheme of [ILL25a]. In particular, it contains a pair of evaluation algorithms EvalKey , EvalTag that allows two parties P_G, P_E who jointly hold additive shares of the form $\langle s \cdot x \rangle$, $\langle s \cdot y \rangle$, to *locally* derive additive shares $\langle s' \cdot g(x, y) \rangle$, for any Boolean gate g .⁸ We will directly use these algorithms to implement our HGate protocol, and our main focus is to design compatible protocols HRead and HCompress for these encodings.

In more detail, in our group instantiation, the encoding of a wire value $\llbracket x \rrbracket$ for $x \in \{0, 1\}$ contains a pair of additive shares $\langle s^{\text{id}} \cdot x \rangle$ over $\mathbb{Z}_p$, and $s^{\text{id}} \in [p]^2$ are two secret exponents over a modulus p and with an associated id. Additionally, the value x is revealed to the evaluator.

$$\llbracket x \rrbracket = (\llbracket x \rrbracket_G, \llbracket x \rrbracket_E), \text{ where } \llbracket x \rrbracket_G = (\langle s^{\text{id}} \cdot x \rangle_G, \text{id}), \text{ and } \llbracket x \rrbracket_E = (\langle s^{\text{id}} \cdot x \rangle_E, \text{id}, x).$$

The $\text{id} = (d, \text{type})$ indicates the depth d of the current encoding and its type, i.e., whether it's an InWire (to depth d), or an OutWire. Our encoding of a hash value $\llbracket \mathbf{h} \rrbracket$ similarly contains a pair of additive shares $\langle s^{\text{id}} \otimes \mathbf{h} \rangle$ over $\mathbb{Z}_p$, an id with $\text{type} = \text{hash}$, but without requiring the evaluator to hold $\mathbf{h}$.⁹

Under this encoding, the Rekey steps are as follows, assuming access to public parameters pp that specify the modulus p .

⁸The DDH-based algorithms of [ILL25a] incurs a $1/\text{poly}$ correctness error probability. But we note the techniques from a follow-up [ILL25b] for removing this error can be adapted to our constructions.

⁹ Note that the hash encoding format is still compatible with the described Rekey steps.

- $\text{RekeySamp}(\text{id})$ samples random share $\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x} \rangle'_G \leftarrow [p]^2$, and sets $\llbracket x' \rrbracket_G := (\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x} \rangle'_G, \text{id})$.
- $\text{RekeyGen}(\llbracket x \rrbracket_G, \llbracket x' \rrbracket_G)$ outputs the difference $\text{rk} \leftarrow \langle \mathbf{s}^{\text{id}} \cdot \mathbf{x} \rangle'_G - \langle \mathbf{s}^{\text{id}} \cdot \mathbf{x} \rangle_G$.
- $\text{RekeyEval}(\llbracket x \rrbracket_E, \text{rk})$ computes $\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x} \rangle'_E \leftarrow \langle \mathbf{s}^{\text{id}} \cdot \mathbf{x} \rangle_E + \text{rk}$, and sets $\llbracket x' \rrbracket_E := (\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x} \rangle'_E, \text{id}, \mathbf{x})$.

In the following, we make extensive use of the fact that two parties can locally evaluate any linear functions on additive shares (over $\mathbb{Z}_p$). Examples include addition between shares, multiplication by any scalar $c \in \mathbb{Z}_p$, and bit-composition over shares of bits. When two parties want to derive any share $\langle a \rangle$, it suffices for them to derive $\langle \text{Bits}(a) \rangle$. We will omit explicitly writing out bit-compositions in the following to avoid cluttering.

Instantiation of HRead and HCompress. We present the two sub-protocols HRead, HCompress in Figure 1, and 2. We note that the sub-protocols only run the local algorithms $\text{HRead}_G, \text{HRead}_E, \text{HCompress}_G, \text{HCompress}_E$, (Construction 5.18) and don't involve communication between the parties. The sub-protocols assume the parties P_G, P_E hold public data pd produced in the overall protocol by Enc. (See Equation 5.4.)

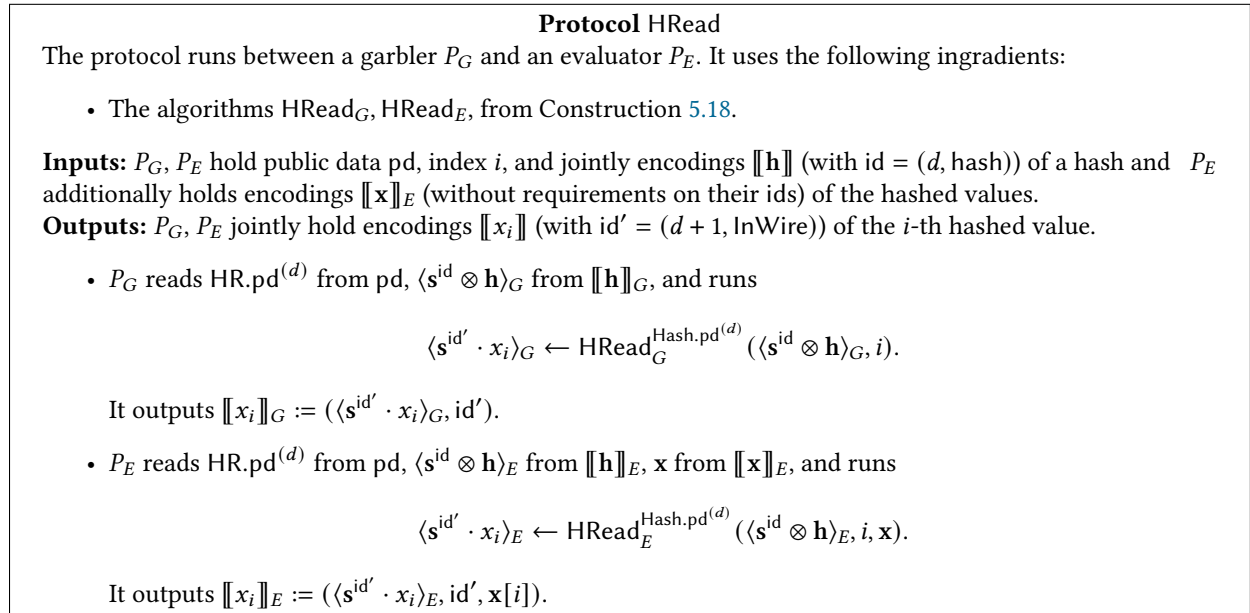


Figure 1: The HRead sub-protocol.

Below, we first present our core construction of a pair of fixed-factor hashing algorithms in Construction 5.15, which uses a SecSwitch algorithm from Construction 5.13 as a building block. Then, we describe wrapper algorithms $\text{HRead}_G, \text{HRead}_E, \text{HCompress}_G, \text{HCompress}_E$ that call fixed-factor hashing in Construction 5.18.

Construction 5.13 (Secret Switching for aHMAC Encodings). The construction is with respect to the following public parameters pp :

- A bound $B < \text{poly}(\lambda)$ on encoded values and $0 < \delta < 1$ on correctness error probability.
- A DDH group instance G with a prime order $p > \lambda^{\omega(1)}$ and a generator $g \in G$.

It uses the DDLog algorithm from Lemma 5.5 instantiated with a suitable PRF : $\{0, 1\}^\lambda \times G \rightarrow \{0, 1\}^\lambda$ to achieve error probability $< \delta$.

Protocol HCompress

The protocol runs between a garbler P_G and an evaluator P_E . It uses the following ingredients:

- The algorithms $\text{HCompress}_G, \text{HCompress}_E$ from Construction 5.18.

Inputs: P_G, P_E hold public data pd and jointly encodings $\llbracket \mathbf{x} \rrbracket$ (with $\text{id} = (d, \text{OutWire})$).

Outputs: P_G, P_E jointly hold the encodings $\llbracket \mathbf{h} \rrbracket$ (with $\text{id}' = (d, \text{hash})$) of the hash.

- P_G reads $\text{HW.pd}^{(d)}$ from pd , $\langle \mathbf{s}^{\text{id}} \otimes \mathbf{x} \rangle$ from $\llbracket \mathbf{x}' \rrbracket_G$ for $i \in S$, and runs

$$\langle \mathbf{s}^{\text{id}'} \otimes \mathbf{h} \rangle_G \leftarrow \text{HCompress}_G^{\text{HW.pd}^{(d)}}(\langle \mathbf{s}^{\text{id}} \otimes \mathbf{x} \rangle_G).$$

It outputs $\llbracket \mathbf{h} \rrbracket_G := (\langle \mathbf{s}^{\text{id}'} \otimes \mathbf{h} \rangle_G, \text{id}')$.

- P_E analogously runs HCompress_E as follows:

$$\langle \mathbf{s}^{\text{id}'} \otimes \mathbf{h} \rangle_E \leftarrow \text{HCompress}_E^{\text{HW.pd}^{(d)}}(\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x} \rangle_E, \mathbf{x}).$$

where $\mathbf{x}$ is read from $\llbracket \mathbf{x} \rrbracket_E$. It outputs $\llbracket \mathbf{h} \rrbracket_E := (\langle \mathbf{s}^{\text{id}'} \otimes \mathbf{h} \rangle_E, \text{id}')$.

Figure 2: The HCompress sub-protocol.

$\text{pd} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s} \in [p]^2, \mathbf{s}' \in \{0, 1\}^\ell)$: Compute the secret-switching public data as

$$\mathbf{r} \leftarrow [p]^\ell, \text{pd} := (\mathbf{c}_0 = g^{\mathbf{r}}, \mathbf{c}_1 = g^{\mathbf{s}[0] \cdot \mathbf{r} + \mathbf{s}'})$$

Additionally, instantiate $\text{DDLog}_{G,g}$ with a (public) random PRF key $\leftarrow \{0, 1\}^\lambda$ which is also included in pd . Since invocations of PRF happens implicitly within invocations of $\text{DDLog}_{G,g}$, we omit it from the following descriptions and similarly from other algorithms in this section.

$\langle \mathbf{s}' \cdot \mathbf{x} \rangle_{G/E} \leftarrow \text{SecSwitch}^{\text{pd}}(\langle \mathbf{s} \cdot \mathbf{x} \rangle_{G/E}, \langle \mathbf{x} \rangle_{G/E})$: Read $(\mathbf{c}_0, \mathbf{c}_1)$ from pd and compute the following

$$\begin{aligned} \mathbf{v}_{G/E} &\leftarrow (\mathbf{c}_1)^{\langle \mathbf{x} \rangle_{G/E}} / (\mathbf{c}_0)^{\langle \mathbf{s}[0] \cdot \mathbf{x} \rangle_{G/E}}, \\ \langle \mathbf{s}' \cdot \mathbf{x} \rangle_{G/E} &\leftarrow \text{DDLog}_{G,g}(\mathbf{v}_{G/E}) \bmod p. \end{aligned}$$

We show the following correctness lemma.

Lemma 5.14 (Correctness of Secret Switching). *For every polynomial length bound $\ell(\lambda)$, there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, encoded value $x \in [B]$, secrets $\mathbf{s} \in [p]^2, \mathbf{s}' \in [p]^\ell$, additive shares $\langle \mathbf{s} \cdot \mathbf{x} \rangle, \langle \mathbf{x} \rangle$, the following holds:*

$$\Pr \left[\begin{array}{l} \langle \mathbf{s}' \cdot \mathbf{x} \rangle_E = \langle \mathbf{s}' \cdot \mathbf{x} \rangle_G + \mathbf{s}' \cdot \mathbf{x} \\ \langle \mathbf{s}' \cdot \mathbf{x} \rangle_G \leftarrow \text{SecSwitch}(\langle \mathbf{s} \cdot \mathbf{x} \rangle_G, \langle \mathbf{x} \rangle_G), \\ \langle \mathbf{s}' \cdot \mathbf{x} \rangle_E \leftarrow \text{SecSwitch}(\langle \mathbf{s} \cdot \mathbf{x} \rangle_E, \langle \mathbf{x} \rangle_E) \end{array} \right] > 1 - \text{negl}(\lambda) - \delta(\lambda).$$

Proof. We first check that the intermediate values $\mathbf{v}_E, \mathbf{v}_G$ during SecSwitch satisfy $\mathbf{v}_E = \mathbf{v}_G \cdot g^{\mathbf{s}' \cdot \mathbf{x}}$.

$$\begin{aligned} \mathbf{v}_E &= \mathbf{c}_1^{\langle \mathbf{x} \rangle_E} / \mathbf{c}_0^{\langle \mathbf{s}[0] \cdot \mathbf{x} \rangle_E} \\ (\text{By Construction}) &= \mathbf{c}_1^{\langle \mathbf{x} \rangle_G + \mathbf{x}} / \mathbf{c}_0^{\langle \mathbf{s}[0] \cdot \mathbf{x} \rangle_G + \mathbf{s}[0] \cdot \mathbf{x}} = \mathbf{v}_G \cdot \mathbf{c}_1^{\mathbf{x}} / \mathbf{c}_0^{\mathbf{s}[0] \cdot \mathbf{x}} \\ (\text{By Setup}) &= \mathbf{v}_G \cdot g^{(\mathbf{s}[0] \cdot \mathbf{r} + \mathbf{s}') \cdot \mathbf{x}} / g^{\mathbf{r} \cdot \mathbf{s}[0] \cdot \mathbf{x}} = \mathbf{v}_G \cdot g^{(\mathbf{s}' \cdot \mathbf{x})}. \end{aligned}$$

Then by the correctness of DDLog (Lemma 5.5), we have $\text{DDLog}(\mathbf{v}_E) = \text{DDLog}(\mathbf{v}_G) + \mathbf{s}' \cdot \mathbf{x} \bmod p$, as desired. $\square$

Construction 5.15 (Fixed Factor Hashing for aHMAC Encodings under DDH). The construction is with respect to the same public parameters $\text{pp} = (B, \delta, G, p, g)$ as specified in Construction 5.13, and additionally

- A hash function $\text{Hash} : \mathbb{Z}^W \rightarrow \mathbb{Z}^{\lceil \log p \rceil}$ with compression factor W , *implicitly* defined by W (the exponents of) group elements g^{b_i} for $i \in [W]$;

$$\text{Hash}(\mathbf{x} \in \mathbb{Z}_B^W) := \sum_i \text{Bits}(b_i) \cdot \mathbf{x}[i] \text{ over } \mathbb{Z} \quad (5.1)$$

It uses as ingredients the secret switching algorithms (Construction 5.13, denoted with prefix SS) and the DDLog algorithm from Lemma 5.5 instantiated with a suitable PRF $: \{0, 1\}^\lambda \times G \rightarrow \{0, 1\}^\lambda$ to achieve error probability $< \delta$.

$\text{pd} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s}, \mathbf{s}' \in [p]^2)$: Read $\{g^{b_i}\}_{i \in [W]}$ from pp , and generate ciphertexts $\{\text{ct}_{i,j,b}\}_{i,j \in [W], b \in \{0,1\}}$ used for extracting hashed values:

$$\begin{aligned} \mathbf{a}_j &\leftarrow [p]^{\lceil \log p \rceil}, \\ \text{ct}_{i,j,b} &\leftarrow (g^{b_i})^{\mathbf{s}[b] \cdot \mathbf{a}_j} / g^{\mathbf{u}_j[i] \cdot \text{Bits}(\mathbf{s}'[b])} \in G \end{aligned}$$

where $\mathbf{u}_j \in \{0, 1\}^W$ denote the j -th unit vector. Next, generate public data $\{\text{pd}_{i,b}\}_{i \in [W], b \in \{0,1\}}$ for computing the hash over shares:¹⁰

$$\text{pd}_{i,b} \leftarrow \text{SS.Setup}(1^\lambda, \mathbf{s}, \text{Bits}(\mathbf{s}'[b]) \otimes \text{Bits}(b_i)).$$

Finally, output

$$\text{pd} := (\text{pp}, \{\text{ct}_{i,j,b}\}, \{\mathbf{a}_j\}, \{\text{pd}_i\}).$$

$\langle \mathbf{s}' \otimes \mathbf{h} \rangle_{G/E} \leftarrow \text{Compress}^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_{G/E}, \langle \mathbf{x} \rangle_{G/E})$: Read $\{\text{pd}_{i,b}\}_{i \in [W], b \in \{0,1\}}$ from pd , and compute

$$\begin{aligned} \langle \mathbf{s}' \otimes \mathbf{h} \rangle_{G/E} &\leftarrow \sum_i \langle \mathbf{s}' \otimes \text{Bits}(b_i) \cdot \mathbf{x}[i] \rangle_{G/E}, \text{ where for } i \in [W], b \in \{0, 1\} : \\ \langle \text{Bits}(\mathbf{s}'[b]) \otimes \text{Bits}(b_i) \cdot \mathbf{x}[i] \rangle_{G/E} &\leftarrow \text{SS.SecSwitch}^{\text{pd}_{i,b}}(\langle \mathbf{s} \cdot \mathbf{x}[i] \rangle_{G/E}, \langle \mathbf{x}[i] \rangle_{G/E}). \end{aligned}$$

$\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_{G/E} \leftarrow \text{Extract}^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{h} \rangle_{G/E}, j, \langle \mathbf{x} \rangle_{G/E})$: First read $\{\text{ct}_{i,j,b}\}_{i,j \in [W], b \in \{0,1\}}$ from pd , and compute

$$\text{diff}_{G/E,j,b} \leftarrow \prod_i (\text{ct}_{i,j,b})^{\langle \mathbf{x}[i] \rangle_{G/E}}.$$

Next read $g^{\mathbf{a}_j}$ and $\alpha, \beta \in [p]^{\lceil \log p \rceil}$ from pd , and compute

$$\begin{aligned} \langle \mathbf{s} \cdot \mathbf{h} \rangle_{G/E} &\leftarrow \sum_{k \in [\lceil \log p \rceil]} 2^k \cdot \langle \mathbf{s} \cdot \mathbf{h}[k] \rangle_{G/E}, \\ \mathbf{v}_{G/E,b} &\leftarrow (g^{\mathbf{a}_j})^{\langle \mathbf{s}[b] \cdot \mathbf{h} \rangle_{G/E}} / \text{diff}_{G/E,j,b}, \\ \langle \text{Bits}(\mathbf{s}'[b]) \cdot \mathbf{x}_j \rangle_{G/E} &\leftarrow \text{DDLog}_{G,g}(\mathbf{v}_{G/E,b}) \bmod p. \end{aligned}$$

We show the following correctness lemma.

Lemma 5.16 (Correctness of Fixed Factor Hashing). *There exists a negligible function negl such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, encoded vectors $\mathbf{x} \in [B]^W$, secrets $\mathbf{s}, \mathbf{s}' \in [p]^2$, additive shares $\langle \mathbf{s} \otimes \mathbf{x} \rangle$, $\langle \mathbf{x} \rangle$, $\langle \mathbf{s} \otimes \text{Hash}(\mathbf{x}) \rangle$, and index $j \in [W]$, the following holds:*

$$\begin{aligned} \Pr \left[\langle \mathbf{s}' \otimes \mathbf{h} \rangle_E = \langle \mathbf{s}' \otimes \mathbf{h} \rangle_G + \mathbf{s}' \otimes \text{Hash}(\mathbf{x}) \mid \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \langle \mathbf{s}' \otimes \mathbf{h} \rangle_G \leftarrow \text{Compress}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_G, \langle \mathbf{x} \rangle_G), \\ \langle \mathbf{s}' \otimes \mathbf{h} \rangle_E \leftarrow \text{Compress}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_E, \langle \mathbf{x} \rangle_E) \end{array} \right] &> 1 - \text{negl}(\lambda) - \delta(\lambda). \\ \Pr \left[\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E = \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G + \mathbf{s}' \cdot \mathbf{x}[j] \mid \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G \leftarrow \text{Extract}_G(\langle \mathbf{s} \otimes \text{Hash}(\mathbf{x}) \rangle_G, j, \langle \mathbf{x} \rangle_G), \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E \leftarrow \text{Extract}_E(\langle \mathbf{s} \otimes \text{Hash}(\mathbf{x}) \rangle_E, j, \langle \mathbf{x} \rangle_E) \end{array} \right] &> 1 - \text{negl}(\lambda) - \delta(\lambda). \end{aligned}$$

¹⁰We abuse notation here, writing $\text{Bits}(b_i) \otimes \text{Bits}(\mathbf{s}[b])$ to mean the flattened vector as the input to SS.Setup .

Proof. The correctness of Compress follows directly from that of SecSwitch and the definition of Hash. To verify the correctness of Extract, it suffices to check the intermediate values $\mathbf{v}_{G,b}$ and $\mathbf{v}_{E,b}$ satisfy $\mathbf{v}_{E,b} = \mathbf{v}_{G,b} \cdot g^{\text{Bits}(\mathbf{s}'[b]) \cdot \mathbf{x}_j}$. (Then, correctness of Extract follows from that of DDLog.)

$$\begin{aligned}
\mathbf{v}_{E,b} &= (g^{a_j})^{\langle \mathbf{s}[b] \cdot \mathbf{h} \rangle_E} / \text{diff}_{E,j,b} \\
(\text{By Construction}) &= (g^{a_j})^{\langle \mathbf{s}[b] \cdot \mathbf{h} \rangle_{G+\mathbf{s}[b] \cdot \mathbf{h}} / \prod_i \text{ct}_{i,j,b}^{\langle \mathbf{x}[i] \rangle_{G+\mathbf{x}[i]}}} \\
&= \mathbf{v}_{G,b} \cdot (g^{a_j})^{\mathbf{s}[b] \cdot \mathbf{h}} / \prod_i \text{ct}_{i,j,b}^{\mathbf{x}[i]} \\
(\text{By Setup}) &= \mathbf{v}_{G,b} \cdot (g^{a_j})^{\mathbf{s}[b] \cdot \mathbf{h}} / \left(\prod_i g^{\mathbf{s}[b] \cdot b_i \cdot \mathbf{a}_j \cdot \mathbf{x}[i]} \right) \cdot \left(\prod_i g^{\mathbf{u}_j[i] \cdot \text{Bits}(\mathbf{s}'[b]) \cdot \mathbf{x}[i]} \right) \\
&= \mathbf{v}_{G,b} \cdot \left(g^{a_j \cdot \mathbf{s}[b]} \right)^{h - \sum_i b_i \cdot x_i} \cdot g^{\text{Bits}(\mathbf{s}'[b]) \cdot \mathbf{x}_j}.
\end{aligned}$$

It suffices to check that $h = \sum_i b_i x_i$:

$$\begin{aligned}
h &:= \sum_{k \in [\lceil \log p \rceil]} 2^k \cdot \mathbf{h}[k] \\
(\text{By Definition}) &= \sum_{k \in [\lceil \log p \rceil]} \sum_i 2^k \cdot \text{Bits}(b_i)[k] \cdot x_i = \sum_i b_i \cdot x_i.
\end{aligned}$$

□

For our security proof later, we note here that the combined public data pd produced by Setup from fixed factor hashing (denoted with prefix FH) and secret switching (denoted with prefix SS) under a random secret $\mathbf{s} \in [p]^2$ are computationally indistinguishable from random. The following lemma follows directly from the DDH assumption (Definition 5.4), and hence we omit its proof.

Lemma 5.17 (Security of Fixed Factor Hashing). *Under the DDH assumption, every efficient adversary $\mathcal{A}$ has negligible advantage in distinguishing the following experiments $\text{Exp}_{\mathcal{A}}^b(1^\lambda)$, for $b \in \{0, 1\}$*

- Sample public parameters pp , and sends them to $\mathcal{A}$, who returns exponents $\mathbf{s}' \in [p]^2$.
- When $b = 0$, sample random exponents $\mathbf{s} \leftarrow [p]^2$ and send pd to $\mathcal{A}$, computed as follows

$$\text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}').$$

- When $b = 1$, sample pd with random group elements from G and send them to $\mathcal{A}$.

Construction 5.18 (Hashing for aHMAC Encodings). The construction is with respect to the following public parameters pp :

- A bound γ on the number of parallel instances of fixed factor hashing, and $0 < \delta < 1$ on the correctness error probability.
- A DDH group instance G with a prime order $p > \lambda^{\omega(1)}$ and a generator $g \in G$.
- A hash function $\text{Hash} : \mathbb{Z}^W \rightarrow \mathbb{Z}^{\lceil \log p \rceil}$ defined as in Equation 5.1.

It uses the fixed factor hashing algorithms (Construction 5.15, denoted with prefix FH) as ingredients, with respect to the above G, p, g, Hash and the following additional parameters B, δ' .

- A bound $B = W$ on encoded values;
- An error bound $\delta' < \delta/\gamma$ (s.t. the error probability over all invocations of the ingredient algorithms is $< \delta$).

For a string $\mathbf{x} \in \{0, 1\}^\ell$ of length $\ell < W \cdot \gamma$, we define its hash $h_{\mathbf{x}}$ to be the γ hashed values of W chunks of $\mathbf{x}$ by Hash (Equation 5.1). For an index $i < W \cdot \gamma$, we define $\text{Chunk}_i \in [\gamma]$ to be the (index of the) chunk corresponding to $\mathbf{x}[i]$ during hashing.

$$\text{Chunk}_i = \lfloor i/W \rfloor. \quad (5.2)$$

$\text{pd} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s}, \mathbf{s}' \in [p]^2)$: Prepare public data for an instance of fixed factor hashing:

$$\text{FH.pd} \leftarrow \text{FH.Setup}(1^\lambda, \mathbf{s}, \mathbf{s}').$$

Output $\text{pd} := \text{FH.pd}$.

$\langle \mathbf{s}' \cdot \mathbf{x}_i \rangle_G \leftarrow \text{HRead}_G^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{h} \rangle_G, i)$: Assumeing $i < W \cdot \gamma$, let $j := \text{Chunk}_i$ as defined in Equation 5.7 and $i' := i \bmod W$. Read FH.pd and run the following:

$$\begin{aligned} \langle \mathbf{s} \otimes \mathbf{h}_j \rangle_G &\leftarrow \langle \mathbf{s} \otimes \mathbf{h}[j \cdot \lceil \log p \rceil : (j+1) \cdot \lceil \log p \rceil] \rangle_G, \\ \langle \mathbf{s}' \cdot \mathbf{x}_i \rangle_G &\leftarrow \text{FH.Extract}^{\text{FH.pd}}(\langle \mathbf{s} \otimes \mathbf{h}_j \rangle_G, i', \langle \mathbf{x}_j \rangle_G) \text{ where } \langle \mathbf{x}_j \rangle_G \leftarrow \mathbf{0}. \end{aligned}$$

$\langle \mathbf{s}' \cdot \mathbf{x}_i \rangle_E \leftarrow \text{HRead}_E^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{h} \rangle_E, i, \mathbf{x})$: Analogous to HRead_G , except running FH.Extract with $\langle \mathbf{x}_j \rangle_E$ defined as:

$$\mathbf{x}_j := \mathbf{x}[j \cdot W : (j+1) \cdot W], \langle \mathbf{x}_j \rangle_E \leftarrow \mathbf{x}_j.$$

$\langle \mathbf{s}' \otimes \mathbf{h} \rangle_G \leftarrow \text{HCompress}_G^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_G)$: Read FH.pd from pd , and run FH.Compress on chunks of $\langle \mathbf{s} \otimes \mathbf{x} \rangle_G$ and $\langle \mathbf{x} \rangle_G \leftarrow \mathbf{0}$. For $j \in [\gamma]$:

$$\begin{aligned} \langle \mathbf{s} \otimes \mathbf{x}_j \rangle_G &\leftarrow \langle \mathbf{s} \otimes \mathbf{x}[j \cdot W : (j+1) \cdot W] \rangle_G, \\ \langle \mathbf{x}_j \rangle_G &\leftarrow \langle \mathbf{x}[j \cdot W : (j+1) \cdot W] \rangle_G, \\ \langle \mathbf{s}' \otimes \mathbf{h}_j \rangle_G &\leftarrow \text{FH.Compress}^{\text{FH.pd}^{(1)}}(\langle \mathbf{s} \otimes \mathbf{x}_j \rangle_G, \langle \mathbf{x}_j \rangle_G), \end{aligned}$$

Finally, concatenate the obtained hashes as $\langle \mathbf{s}' \otimes \mathbf{h} \rangle_G$ and output it.

$\langle \mathbf{s}' \otimes \mathbf{h} \rangle_E \leftarrow \text{HCompress}_E^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_E, \mathbf{x})$: Analogous to HCompress_G , except running FH.Compress with chunks of $\langle \mathbf{x} \rangle_E \leftarrow \mathbf{x}$.

Remark 5.19. As presented, a hash computed by HCompress has a fixed size $\gamma \cdot \text{poly}(\lambda)$. In our application to garbling schemes, we optimize to only run $\lceil |\mathbf{x}|/W \rceil$, instead of γ , instances of the fixed factor hash. The resulting hash has size $\lceil |\mathbf{x}|/W \rceil \cdot \text{poly}(\lambda)$.

We note the following correctness lemma, which follows from those of Compress , Extract .

Lemma 5.20 (Correctness of Hashing). *There exists a negligible function negl such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, encoded vectors $\mathbf{x} \in \{0, 1\}^{W \cdot \gamma}$, index $j \in [W \cdot \gamma]$, secrets $\mathbf{s}, \mathbf{s}' \in [p]^2$, additive shares $\langle \mathbf{s} \otimes \mathbf{x} \rangle$, $\langle \mathbf{s} \otimes \mathbf{h} \rangle$, where $\mathbf{h}$ is hash of $\mathbf{x}$,*

$$\begin{aligned} \Pr \left[\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E = \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G + \mathbf{s}' \cdot \mathbf{x}[j] \mid \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G \leftarrow \text{HRead}_G(\langle \mathbf{s} \otimes \mathbf{h} \rangle_G, j), \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E \leftarrow \text{HRead}_E(\langle \mathbf{s} \otimes \mathbf{h} \rangle_E, j, \mathbf{x}), \end{array} \right] &> 1 - \text{negl}(\lambda) - \delta(\lambda) \\ \Pr \left[\langle \mathbf{s}' \otimes \mathbf{h} \rangle_E = \langle \mathbf{s}' \otimes \mathbf{h} \rangle_G + \mathbf{s}' \otimes \mathbf{h}, \mid \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \langle \mathbf{s}' \otimes \mathbf{h} \rangle_G \leftarrow \text{HCompress}_G(\langle \mathbf{s} \otimes \mathbf{x} \rangle_G), \\ \langle \mathbf{s}' \otimes \mathbf{h} \rangle_E \leftarrow \text{HCompress}_E(\langle \mathbf{s} \otimes \mathbf{x} \rangle_E, \mathbf{x}), \end{array} \right] &> 1 - \text{negl}(\lambda) - \delta(\lambda). \end{aligned}$$

For our security proof later, we note the following lemma, which follows from Lemma 5.17.

Lemma 5.21 (Security of Hashing). *Under the DDH assumption, every efficient adversary $\mathcal{A}$ has negligible advantage in distinguishing the experiments $\text{Exp}_{\mathcal{A}}^b(1^\lambda)$, for $b \in \{0, 1\}$, defined analogously to Lemma 5.17 with respect to the Setup algorithm in Construction 5.18.*

Instantiation of HGate. We set up our encodings for wire values to be directly compatible with the aHMAC evaluation constructions from [ILL25a], which are used for the HGate protocol (Figure 3). We note that HGate only runs local algorithms EvalKey, EvalTag (Construction 5.24) described below, and doesn't involve communication between the parties. Similarly to HRead and HCompress, the sub-protocol assume the parties P_G, P_E hold public data pd produced in the overall protocol by Enc. (See Equation 5.4.)

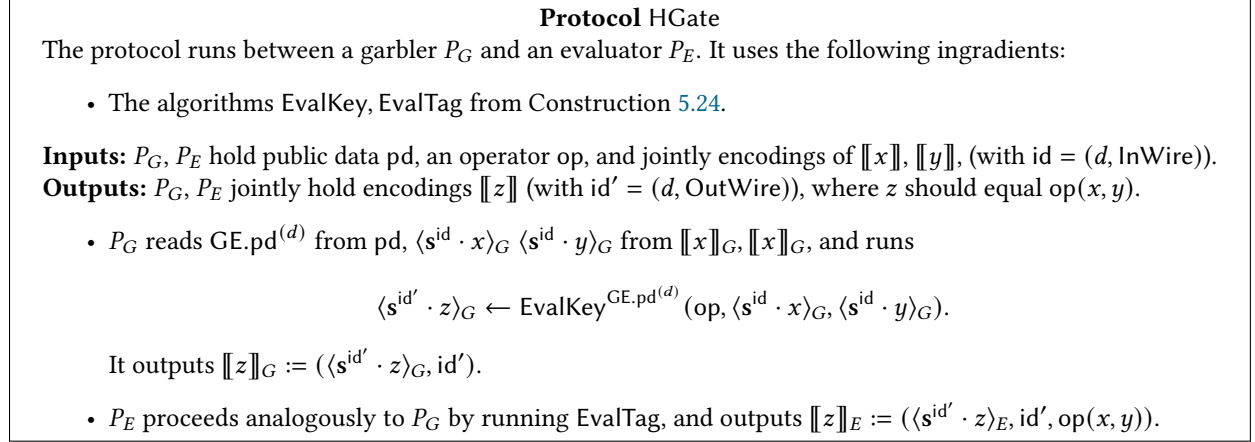


Figure 3: The HGate sub-protocol.

We recall necessary preliminaries from [ILL25a] below (modified slightly for a more modular presentation). When possible, we omit construction details that are not relevant for security proofs and only describe the interfaces.

Construction 5.22 (aHMAC Multiplication under DDH [ILL25a]). The construction is with respect to the same public parameters $pp = (B, \delta, G, p, g)$ as specified in Construction 5.13.

It uses as ingredients the DDLog algorithm from Lemma 5.5 instantiated with a suitable PRF : $\{0, 1\}^\lambda \times G \rightarrow \{0, 1\}^\lambda$ to achieve error probability $< \delta$.

$pd \leftarrow \text{Setup}^{pp}(1^\lambda, s, s', s'' \in [p])$: takes three secrets $s, s', s'' \in [p]$, and computes pd as follows.

$$r \leftarrow [p]^{\lceil \log p \rceil}, \quad pd := (pp, g^r, g^{rs}, g^{rs'}, g^{r s s' + \text{Bits}(s'')}).$$

$\langle s'' \cdot z \rangle_G \leftarrow \text{MultKey}^{pd}(\langle s \cdot x \rangle_G, \langle s' \cdot y \rangle_G)$: takes as inputs two integers representing the 0-shares of $\langle s \cdot x \rangle_G, \langle s' \cdot y \rangle_E \in \mathbb{Z}$ for some secrets $s, s' \in [p]$. It outputs the 0-share of another share $\langle s'' \cdot z \rangle_G$.

$\langle s'' \cdot z \rangle_E \leftarrow \text{MultTag}^{pd}(\langle s \cdot x \rangle_E, \langle s' \cdot y \rangle_E, x, y)$: takes as input the 1-shares of $\langle s \cdot x \rangle_E, \langle s' \cdot y \rangle_E$ for some secret s, s' , and the shared values $x, y \in [B]$. It outputs the 1-share of another share $\langle s'' \cdot z \rangle_E$.

Lemma 5.23 (Correctness of aHMAC Multiplication [ILL25a]). *There exists a negligible function $\text{negl}(\lambda)$ such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, secret elements $s, s', s'' \in [p]$, inputs $x, y \in [B]$ such that $xy < B$, and additive shares $\langle sx \rangle_G, \langle sx \rangle_E, \langle s'y \rangle_G, \langle s'y \rangle_E$, the following holds:*

$$\Pr \left[\begin{array}{l} w_E = w_G + s'' \cdot xy \\ \text{(over } \mathbb{Z}_p) \end{array} \middle| \begin{array}{l} pd \leftarrow \text{Setup}(pp, s, s', s'') \\ w_G = \text{MultKey}(\langle sx \rangle_G, \langle s'y \rangle_G) \\ w_E = \text{MultTag}(\langle sx \rangle_E, \langle s'y \rangle_E, x, y) \end{array} \right] > 1 - \text{negl}(\lambda) - \delta(\lambda).$$

We next recall from [ILL25a] the leveled gate evaluation algorithms EvalKey, EvalTag for aHMAC.

Construction 5.24 (aHMAC (leveled) Gate Evaluation [ILL25a]). The construction is with respect to the same public parameters $pp = (\delta, G, p, g)$ as specified in Construction 5.22.

It uses Construction 5.22 (with prefix Mt) and Construction 5.13 (with prefix SS) as ingredients w.r.t. to G, p, g from above and $B = 2, \delta' = \delta/2$.

$\text{pd} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s}, \mathbf{s}' \in [p]^2)$: prepare two instances of aHMAC multiplication, and an public data for secret switching from $\mathbf{s}$ to $\mathbf{s}'$.

$$\begin{aligned} \text{Mt.pd}_0 &\leftarrow \text{Mt.Setup}(1^\lambda, \mathbf{s}[0], \mathbf{s}[1], \mathbf{s}'[0]), \text{Mt.pd}_1 \leftarrow \text{Mt.Setup}(1^\lambda, \mathbf{s}[0], \mathbf{s}[1], \mathbf{s}'[1]), \\ \text{pd} &:= (\text{Mt.pd}_0, \text{Mt.pd}_1, \text{SS.pd}), \text{SS.pd} \leftarrow \text{SS.Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'). \end{aligned}$$

$\langle \mathbf{s}' \cdot \mathbf{z} \rangle_G \leftarrow \text{EvalKey}^{\text{pd}}(g, \langle \mathbf{s} \cdot \mathbf{x} \rangle_G, \langle \mathbf{s} \cdot \mathbf{y} \rangle_G)$: We show the case for $g = \text{XOR}$. (The case for $g = \text{AND}$ or any binary gate is analogous.) First read SS.pd from pd , and compute $\langle \mathbf{s}' \cdot (\mathbf{x} + \mathbf{y}) \rangle_G$:

$$\begin{aligned} \langle \mathbf{s}' \cdot (\mathbf{x} + \mathbf{y}) \rangle_G &\leftarrow \text{SecSwitch}^{\text{SS.pd}}(\langle \mathbf{s} \cdot (\mathbf{x} + \mathbf{y}) \rangle_G, \langle \mathbf{x} + \mathbf{y} \rangle_G), \\ \text{where } \langle \mathbf{s} \cdot (\mathbf{x} + \mathbf{y}) \rangle_G &\leftarrow \langle \mathbf{s} \cdot \mathbf{x} \rangle_G + \langle \mathbf{s} \cdot \mathbf{y} \rangle_G, \langle \mathbf{x} + \mathbf{y} \rangle_G \leftarrow 0. \end{aligned}$$

Then read $\text{Mt.pd}_0, \text{Mt.pd}_1$ from pd , and compute the final output $\langle \mathbf{s}' \cdot \mathbf{z} \rangle_G$ as follows:

$$\begin{aligned} \langle \mathbf{s}'[b] \cdot \mathbf{x} \mathbf{y} \rangle_G &\leftarrow \text{MultKey}^{\text{Mt.pd}_b}(\langle \mathbf{s}[0] \cdot \mathbf{x} \rangle_G, \langle \mathbf{s}[1] \cdot \mathbf{y} \rangle_G), \text{ for } b \in \{0, 1\}, \\ \langle \mathbf{s}' \cdot \mathbf{z} \rangle_G &\leftarrow 2 \cdot \langle \mathbf{s}' \cdot (\mathbf{x} + \mathbf{y}) \rangle_G - \langle \mathbf{s}' \cdot \mathbf{x} \mathbf{y} \rangle_G. \end{aligned}$$

$\langle \mathbf{s}' \cdot \mathbf{z} \rangle_E \leftarrow \text{EvalTag}^{\text{pd}}(g, \langle \mathbf{s} \cdot \mathbf{x} \rangle_E, \langle \mathbf{s} \cdot \mathbf{y} \rangle_E, \mathbf{x}, \mathbf{y})$: Analogous to EvalKey , except when running SecSwitch , set $\langle \mathbf{x} + \mathbf{y} \rangle_E \leftarrow \mathbf{x} + \mathbf{y}$, and run MultTag instead of MultKey in the second step.

We note the following straightforward correctness and security lemmas and omit their proofs.

Lemma 5.25 (Correctness of aHMAC Gate Evaluation). *There exists a negligible function $\text{negl}(\lambda)$ such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, secret elements $\mathbf{s}, \mathbf{s}' \in [p]^2$, inputs $\mathbf{x}, \mathbf{y} \in \{0, 1\}$, binary gates $g : \{0, 1\}^2 \rightarrow \{0, 1\}$ and additive shares $\langle \mathbf{s} \cdot \mathbf{x} \rangle_G, \langle \mathbf{s} \cdot \mathbf{x} \rangle_E, \langle \mathbf{s} \cdot \mathbf{y} \rangle_G, \langle \mathbf{s} \cdot \mathbf{y} \rangle_E$, the following holds:*

$$\Pr \left[\begin{array}{l} \mathbf{w}_E = \mathbf{w}_G + \mathbf{s}' \cdot g(\mathbf{x}, \mathbf{y}) \\ \text{(over } \mathbb{Z}_p) \end{array} \middle| \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}') \\ \mathbf{w}_G = \text{EvalKey}(g, \langle \mathbf{s} \cdot \mathbf{x} \rangle_G, \langle \mathbf{s} \cdot \mathbf{y} \rangle_G) \\ \mathbf{w}_E = \text{EvalTag}(g, \langle \mathbf{s} \cdot \mathbf{x} \rangle_E, \langle \mathbf{s} \cdot \mathbf{y} \rangle_E, \mathbf{x}, \mathbf{y}) \end{array} \right] > 1 - \text{negl}(\lambda).$$

Lemma 5.26 (Security of aHMAC Gate Evaluation). *Under the DDH assumption, every efficient adversary $\mathcal{A}$ has negligible advantage in distinguishing the experiments $\text{Exp}_{\mathcal{A}}^b(1^\lambda)$, for $b \in \{0, 1\}$, defined analogously to Lemma 5.17 with respect to the Setup algorithm in Construction 5.24.*

Instantiation of Setup and Enc. We now describe the $\text{Setup}(1^\lambda, 1^D, |C|)$ procedure that selects public parameters $\text{pp} := (1^D, G, p, g, \text{Hash}, \delta, \gamma = \lceil |C|^{2/3} \cdot D^{1/3} \rceil)$ for all sub-protocols $\text{HRead}, \text{HCompress}, \text{HGate}$.¹¹

- Sample a DDH instance $(G, p, g) \leftarrow \text{Gen}(1^\lambda)$, with prime order p .
- Set $W = \lceil (|C|/D)^{1/3} \rceil$, and sample $b_i \leftarrow [p]$ for $i \in [W]$, which define a global hash function $\text{Hash} : \mathbb{Z}^W \rightarrow \mathbb{Z}^{\lceil \log p \rceil}$ as specified in Equation 5.1.
- Set $\delta < 1/(p(\lambda) \cdot (3(D+1) + |C|))$ s.t. the probability over all invocations of the sub-protocols is $< 1/p(\lambda)$ for any desired polynomial $p(\lambda)$.
- Additionally, pre-compute values $\{g^{2^i}\}_{i \in [\lceil \log p \rceil]}$ for achieving low-depth exponentiations (by the garbler).

We next describe how to sample gk , and the $\text{Enc}(\text{gk}, \mathbf{x})$ procedure that produces encodings $\llbracket \mathbf{x} \rrbracket$ and public data pd .

- First choose leveled secrets $\mathbf{s}^{\text{id}} \in [p]^2$ as gk for our encodings:

$$\begin{aligned} \mathbf{s}^{(0, \text{hash})}, \mathbf{s}^{(0, \text{OutWire})}, \mathbf{s}^{(D+1, \text{InWire})} &\leftarrow [p]^2, \\ \mathbf{s}^{(d, \text{InWire})}, \mathbf{s}^{(d, \text{OutWire})}, \mathbf{s}^{(d, \text{hash})} &\leftarrow [p]^2 \text{ for } d = 1, \dots, D. \end{aligned} \tag{5.3}$$

¹¹In our garbling scheme definition, the Setup algorithm doesn't take $|C|$ or D as input. This syntactical mismatch can be solved by delaying settings of $\text{Hash}, \delta, \gamma$ which depends on $|C|, D$ to GarbC , when the circuit is known.

- Next sets up public data for the sub-protocols:

$$\begin{aligned}
\text{HW.pd}^{(d)} &\leftarrow \text{Hash.Setup}(1^\lambda, \mathbf{s}^{\text{id}}, \mathbf{s}^{\text{id}'}) , \text{HR.pd}^{(d)} \leftarrow \text{Hash.Setup}(1^\lambda, \mathbf{s}^{\text{id}'}, \mathbf{s}^{\text{id}''}) \\
&\text{where } \text{id} = (d, \text{OutWire}), \text{id}' = (d, \text{hash}), \text{id}'' = (d+1, \text{InWire}) \text{ for } d = 0, \dots, D \\
\text{GE.pd}^{(d)} &\leftarrow \text{GE.Setup}(1^\lambda, \mathbf{s}^{\text{id}}, \mathbf{s}^{\text{id}'}) \\
&\text{where } \text{id} = (d, \text{InWire}), \text{id}' = (d, \text{OutWire}) \text{ for } d = 1, \dots, D \\
\text{pd} &:= (\{\text{HW.pd}^{(d)}\}, \{\text{HR.pd}^{(d)}\}, \{\text{GE.pd}^{(d)}\}).
\end{aligned} \tag{5.4}$$

- Finally use $\mathbf{s}^{\text{id}=(0, \text{OutWire})}$ from the above to compute

$$\begin{aligned}
\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x}[i] \rangle_G &\leftarrow [p]^2, \quad \langle \mathbf{s}^{\text{id}} \cdot \mathbf{x}[i] \rangle_E \leftarrow \langle \mathbf{s}^{\text{id}} \cdot \mathbf{x}[i] \rangle_G + \mathbf{s}^{\text{id}} \cdot \mathbf{x}[i] \text{ over } \mathbb{Z}_p, \\
\llbracket \mathbf{x}[i] \rrbracket_G &:= (\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x}[i] \rangle_G, \text{id}), \quad \llbracket \mathbf{x}[i] \rrbracket_E := (\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x}[i] \rangle_E, \text{id}, \mathbf{x}[i]).
\end{aligned} \tag{5.5}$$

Garbler Depth Complexity. As discussed in Section 4.1, the garbling depth roughly equals the sum of the depth of HRead, HCompress, HGate. We review the depth of each of the sub-protocols.

HRead: The garbler runs HRead_G , which consists of a call to Extract_G .

Extract_G consists mainly of a summation over $\lceil \log p \rceil = O(\lambda)$ integers, each of $O(\lambda)$ bits, an exponentiation in G , a DDLog, and another summation over $O(\lambda)$ integers of $O(\lambda)$ bits. Assuming the group G is a subgroup of $\mathbb{Z}_{p'}^*$, then exponentiation can be computed with depth $O(\log \lambda)$ with pre-computed values $\{g^{2^i}\}_{i \in \lceil \log p \rceil}$. All of the other steps also have depth $O(\log \lambda)$ hence Extract has depth $O(\log \lambda)$.

We conclude the depth of HRead_G is $O(\log \lambda)$.

HCompress: The garbler runs HCompress_G , which mainly consists of a call to Compress .

Compress_G consists of a SecSwitch and two summations of $O(\lambda)$ integers of $O(\lambda)$ bits. SecSwitch further consists of two exponentiations in G , one multiplication in G , and a DDLog. Therefore, similarly to Extract_G , Compress_G has depth $O(\log \lambda)$.

We conclude the depth of HCompress is also $O(\log \lambda)$.

HGate: The garbler runs EvalKey , which mainly consists of a call to SecSwitch and MultKey . The construction of MultKey from [ILL25a] consists of constant number of exponentiations and multiplications in G , similarly to our SecSwitch construction, and hence has depth $O(\log \lambda)$.

Succinctness and Decomposability. The garbling material consists of the public data pd described in Equation 5.4, and a series of re-key materials $\{\text{rk}^{(d)}\}_{d \in [D]}$. We briefly note the following, where we let $W = (|C|/D)^{1/3}$ denote the compression factor of Hash.

- The re-key material of each depth $\text{rk}^{(d)}$ is of size $\lceil |C_d|/W \rceil \cdot \text{poly}(\lambda)$, where $|C_d|$ is the number of depth- d wires. Overall, we have $|\{\text{rk}^{(d)}\}_{d \in [D]}| \leq (|C|/W + D) \cdot \text{poly}(\lambda)$.
- The public data for sub-protocols $\text{HR.pd}^{(d)}$, $\text{HW.pd}^{(d)}$, $\text{GE.pd}^{(d)}$ all have size $< W^2 \cdot \text{poly}(\lambda)$. Overall the public data have total size $\leq W^2 \cdot D \cdot \text{poly}(\lambda)$.

Therefore, with our setting $W = (|C|/D)^{1/3}$ the overall garbling material have size $< (D + |C|^{2/3} \cdot D^{1/3}) \cdot \text{poly}(\lambda)$.

The input encodings described in Equation 5.5 are clearly decomposable, and have size $< \ell \cdot \text{poly}(\lambda)$. The decryption key dk , as specified in Definition 3.1, consists of a single integer of $O(\lambda) < \text{poly}(\lambda)$ bits.

Remark 5.27. We note that in this work we choose to associate the encodings of wire values at each depth d a different secret. This allows us to base security on the standard DDH assumption, but also requires us to setup different instances, with fresh public data, of hashing Hash and gate evaluation GE schemes for each depth. Alternatively,

assuming suitable circular variants of DDH, we can let encodings of all wires values share a common secret, and setup only a single instance of SS, Hash, GE each. In this circular variant, the size of re-key materials remains bounded by $\leq (|C|/W + D) \cdot \text{poly}(\lambda)$, while the size of public data is now bounded by $W^2 \cdot \text{poly}(\lambda)$. Choosing $W = |C|^{1/3}$, the overall garbling material have size $\leq (D + |C|^{2/3}) \cdot \text{poly}(\lambda)$. Similar analysis of garbling sizes also holds for our group instantiation of standard (private) garbling in Section 6.1.

Amortizing Large Public Data. We note that according to the above analysis, an alternative setting of the compression factor $W = |C|$ results in re-key material sizes $|\{\text{rk}^{(d)}\}| \leq D \cdot \text{poly}(\lambda)$, and public data sizes $< |C|^2 \cdot D \cdot \text{poly}(\lambda)$. While such a setting makes little sense in a single instance of garbling, where the size of garbling material is dominated by the size of public data, it can be useful in a scenario where the same garbler is to garble multiple circuits of the same depth. In such a scenario, the public data can be re-used across all circuits garbled by the same garbler, so its cost in garbling size is amortized. The amortized size of garbling material becomes $D \cdot \text{poly}(\lambda)$.

In more detail, we rename the public data to be evk, viewing it as an evaluation key for the designated garbler. We modify the garbling scheme syntax to have an explicit KeyGen algorithm for producing evk as follows.

- $\text{Setup}(1^\lambda, 1^d) \rightarrow \text{pp}$: Unchanged from Definition 3.1.
- $\text{KeyGen}(\text{pp}) \rightarrow (\text{evk}, \text{msk})$: On input public parameters, the KeyGen algorithm outputs a pair of valuation key evk and master secret key msk.
- $\text{GarbC}(\text{msk}, C) \rightarrow (\hat{C}, \text{gk})$: On input a master secret key and a circuit C (of depth $\leq d$), the GarbC algorithm outputs garbled circuit $\hat{C}$ and garbling key gk.
- $\text{Garblnp}(\text{gk}, x) \rightarrow \hat{x}$: Unchanged from Definition 3.1.
- $\text{Eval}(\text{evk}, C, \hat{C}, \hat{x}) \rightarrow \{0, 1\}$: On input an evaluation key evk, circuit C , garbled circuit $\hat{C}$ and garbled input $\hat{x}$, the Eval algorithm computes the evaluation of the garbling circuit.

For correctness, we require for all polynomials $m(\cdot), p(\cdot), d(\cdot)$, there exists negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, all circuits $\{C_i : \{0, 1\}^{\ell_i} \rightarrow \{0, 1\}\}_{i \in [m]}$ of size $|C_i| \leq p(\lambda)$ and depth $\leq d(\lambda)$, all inputs $\{x_i \in \{0, 1\}^{\ell_i}\}_{i \in [m]}$,

$$\Pr \left[\begin{array}{c} \text{Eval}(\text{evk}, C_i, \hat{C}_i, \hat{x}_i) = C_i(x_i) \\ \forall i \in [m] \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, 1^d) \\ (\text{evk}, \text{msk}) \leftarrow \text{KeyGen}(\text{pp}) \\ (\hat{C}_i, \text{gk}_i) \leftarrow \text{GarbC}(\text{msk}, C_i) \\ \hat{x}_i \leftarrow \text{Garblnp}(\text{gk}_i, x_i) \end{array} \right] = 1 - \text{negl}(\lambda).$$

For authenticity, we modify the security game $\text{ExptAuth}_{\mathcal{A}}(\lambda)$ from Definition 3.1 as follows:

1. The adversary specifies a depth bound 1^d , an arbitrary number of circuits $\{C_i\}_{i \in [m]}$ (of depth $\leq d$), and corresponding inputs $\{x_i^*\}$. The challenger computes the following:
 - (a) The challenger samples public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda, 1^d)$, and a pair of evaluation keys and master secret keys $(\text{evk}, \text{msk}) \leftarrow \text{KeyGen}(\text{pp})$.
 - (b) The challenger produces garbled circuits $(\hat{C}_i, \text{gk}_i) \leftarrow \text{GarbC}(\text{msk}, C_i)$, garbled inputs $\hat{x}_i^* \leftarrow \text{Garblnp}(\text{gk}_i, x_i^*)$, and accompanying authentication keys auk_i^* for $i \in [m]$.
2. The challenger returns $(\text{pp}, \text{evk}, \{\hat{C}_i, \hat{x}_i^* = (L_{x^*, i}, \text{dk}_i^*)\})$.
3. The adversary submits a label L_i^* and an index $i \in [m]$. The output of the experiment is 1 iff $1 - C_i(x_i^*) = \text{DecodeLabel}(\text{dk}_i^*, L_i^*)$ and $\text{Authenticate}(\text{auk}_i^*, L_i^*) = 1$.

Security. We sketch *selective* One-Wayness (Definition 3.2, with privacy error $\delta(\lambda) = 1/\text{poly}(\lambda)$) for the garbling scheme via a series of hybrid experiments, where Hyb_0 is the experiment $\text{ExptOW}_{\mathcal{A}}(\lambda)$, and Hyb_3 is an experiment where the adversary's advantage clearly negligible.

- Hyb_0 : We briefly recall the experiment $\text{ExptOW}_{\mathcal{A}}(\lambda)$:
 - The challenger returns public parameters pp according to Setup, and receives a circuit C and an input $\mathbf{x}$ from the adversary.
 - The challenger returns $\widehat{C}$, which consists of the public data pd as described in Equation 5.4, consisting of $\{\text{HW.pd}^{(d)}\}$, $\{\text{HR.pd}^{(d)}\}$, $\{\text{GE.pd}^{(d)}\}$, and the rekey materials $\{\text{rk}^{(d)}\}$.
 - The challenger also returns $\hat{\mathbf{x}}$, which consists of the encodings $\llbracket \mathbf{x} \rrbracket_E$ as described in Equation 5.5 and a decoding key dk that equals the last component of garbler's shares of the output wires.
- Hyb_1 : This hybrid derive the rekey materials $\{\text{rk}^{(d)}\}$ and the decoding key dk differently:
 - Using the input encodings $\llbracket \mathbf{x} \rrbracket_E$, derive evaluator's shares on all intermediate wires and hashes, and set $\{\text{rk}^{(d)}\}$ as the difference between evaluator's shares of hashes instead of the garbler's.
 - Derive the decoding key dk as the last component of evaluator's shares of the output wires, subtracting $s^{D+1, \text{InWire}}[1] \cdot C(\mathbf{x})$.

By the correctness of the garbling scheme, this hybrid is equivalent to the previous, except with $\delta(\lambda) = 1/\text{poly}(\lambda)$ failure probability.

$$|\Pr[\text{Hyb}_{1, \mathcal{A}}(\lambda) = 1] - \Pr[\text{Hyb}_{0, \mathcal{A}}(\lambda) = 1]| \leq \delta(\lambda).$$

- Hyb_2 : This hybrid computes the input encodings as random elements in $\mathbb{Z}_q^2$, instead of following Equation 5.5. This hybrid is equivalent to the previous, by the randomness of $\langle \mathbf{s}^{\text{id}} \cdot \mathbf{x}[i] \rangle_G$. (See Equation 5.5.)

$$\Pr[\text{Hyb}_{2, \mathcal{A}}(\lambda) = 1] = \Pr[\text{Hyb}_{1, \mathcal{A}}(\lambda) = 1].$$

We note that the secret $\mathbf{s}^{(0, \text{OutWire})}$ is only used for generating public data $\text{HW.pd}^{(0)}$, and not anywhere else in the experiment.

- Hyb_3 : This hybrid samples the public data $\{\text{HW.pd}^{(d)}\}$, $\{\text{HR.pd}^{(d)}\}$, $\{\text{GE.pd}^{(d)}\}$, as random group elements in G , instead of as a chain of ciphertexts encrypting the next secret s' under some secret s . By Lemma 5.21 and 5.26, both of which assumes the DDH assumption in prime-order groups, we have

$$|\Pr[\text{Hyb}_{3, \mathcal{A}}(\lambda) = 1] - \Pr[\text{Hyb}_{2, \mathcal{A}}(\lambda) = 1]| < \text{negl}(\lambda).$$

We note that the first component $\mathbf{s}^{(D+1, \text{InWire})}[0]$ of the secret for the output label is no longer used anywhere in the experiment. So the probability of the adversary guessing this random secret is negligible, i.e. $\Pr[\text{Hyb}_{3, \mathcal{A}}(\lambda) = 1] < \text{negl}(\lambda)$.

By a standard hybrid argument, we have

$$\Pr[\text{ExptOW}_{\mathcal{A}}(\lambda) = 1] < \Pr[\text{Hyb}_{3, \mathcal{A}}(\lambda) = 1] + \text{negl}(\lambda) + \delta(\lambda) = \text{negl}(\lambda) + \delta(\lambda).$$

Removing the $1/\text{poly}(\lambda)$ Correctness and Privacy Error. We note two ways to remove the correctness error, which also causes the privacy error, from our group instantiation. First, the simpler solution is to use Pailler instead of prime-order groups, which has an associated DDLog algorithm with negligible error. The Paillier instantiation relies on the corresponding Paillier-based aHMAC evaluation algorithms from [ILL25a] for HGate. And the constructions for HCompress, HRead sub-protocols are analogous to those in prime-order groups. Security of the obtained garbling scheme relies on the DDH assumption in Paillier groups (Definition 5.9).

Second, we can adapt the solutions from [ILL25b, BGI17] for prime-order groups to achieve negligible correctness error at the cost of heavier computation. In more detail, we rely on the fact that both parties running the DDLog

algorithm can locally detect whether a failure has occurred (with false positives). Therefore, the garbler can abort and restart as soon as a failure during DDLog is detected. When setting this failure probability to be $1/\lambda$, the garbler is expected to restart $O(\lambda)$ times. Note that all other steps besides DDLog has negligible failure probability, the final garbled circuit produced by the garbler achieves negligible correctness error. Security still relies on the standard DDH assumption.

We note a subtlety in privacy: the restarting pattern by the garbler depends on the (leveled) secrets $\mathbf{s}^{\text{id}}$ in our constructions, and hence leaks some information about them. Following the solutions from [ILL25b, BGI17], we compensate by deriving $\mathbf{s}$ from linearly combining $O(\lambda \cdot \log p)$ secret bits, and ensuring that at most λ bits are affected by the leakage. We refer to [ILL25b, BGI17] for details of dealing with such leakages.

5.3 Instantiation Under Lattices

Similarly to the previous section, our lattice instantiation uses the aHMAC evaluation algorithms EvalKey, EvalTag based on RLWE from [ILL25c] to implement the HGate protocol, and our main focus is to design compatible protocols HRead and HCompress.

In more detail, in our lattice instantiation, the encoding of a wire value $\llbracket x \rrbracket$ for $x \in \{0, 1\}$ contains a pair of additive shares $\langle \mathbf{s}^{\text{id}} \cdot x \rangle$ over $\mathcal{R}_p$, where $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ is a polynomial ring, p is a modulus, and $\mathbf{s}^{\text{id}} \in \mathcal{R}^2$ are two secret elements with an associated id. Again, the value x is revealed to the evaluator.

$$\llbracket x \rrbracket = (\llbracket x \rrbracket_G, \llbracket x \rrbracket_E), \text{ where } \llbracket x \rrbracket_G = (\langle \mathbf{s}^{\text{id}} \cdot x \rangle_G, \text{id}), \text{ and } \llbracket x \rrbracket_E = (\langle \mathbf{s}^{\text{id}} \cdot x \rangle_E, \text{id}, x).$$

The $\text{id} = (d, \text{type})$ is defined in the same way as the previous section.

Under this encoding, the Rekey steps are as follows, assuming access to public parameters pp that specify the ring $\mathcal{R}$ and modulus p .

- $\text{RekeySamp}(\text{id})$ samples random share $\langle \mathbf{s}^{\text{id}} \cdot x \rangle'_G \leftarrow \mathcal{R}_p^2$, and sets $\llbracket x' \rrbracket_G := (\langle \mathbf{s}^{\text{id}} \cdot x \rangle'_G, \text{id})$.
- $\text{RekeyGen}(\llbracket x \rrbracket_G, \llbracket x' \rrbracket_G)$ outputs the difference $\text{rk} \leftarrow \langle \mathbf{s}^{\text{id}} \cdot x \rangle'_G - \langle \mathbf{s}^{\text{id}} \cdot x \rangle_G$.
- $\text{RekeyEval}(\llbracket x \rrbracket_E, \text{rk})$ computes $\langle \mathbf{s}^{\text{id}} \cdot x \rangle'_E \leftarrow \langle \mathbf{s}^{\text{id}} \cdot x \rangle_E + \text{rk}$, and sets $\llbracket x' \rrbracket_E := (\langle \mathbf{s}^{\text{id}} \cdot x \rangle'_E, \text{id}, x)$.

Instantiation of HRead and HCompress. The sub-protocols HRead, HCompress in our lattice instantiation are the same as those in the previous section (Figure 1, and 2), except they use the local algorithms HRead_G , HRead_E , HCompress_G , HCompress_E from Construction 5.33. We present these new local algorithms below.

Similarly to the previous section, we first present the SecSwitch algorithm (Construction 5.28) as a building block, and next our core construction of a pair of fixed-factor hashing algorithms (Construction 5.30). Then, we describe wrapper algorithms HRead_G , HRead_E , HCompress_G , HCompress_E that call the fixed-factor hashing in Construction 5.33.

We note a technical difference in constructing the wrapper algorithms. While in the group instantiation (Construction 5.18), the wrapper algorithms make parallel calls to fixed-factor hashing, in the lattice instantiation the wrapper algorithms instead make recursive calls. At a high-level, the difference stems from the fact that the hash function in the group instantiation, defined by W coefficients (Equation 5.1), is private to the evaluator, while the hash function in the lattice instantiation (Equation 5.6) is public. The benefit of using recursive, instead of parallel, calls to fixed factor hashing is that we can now achieve a much larger compression factor, and hence much smaller hashes. Consequently, our garbling scheme has much smaller communication cost from rekeying the hashes. (See Theorem 5.2.)

Construction 5.28 (Secret Switching for aHMAC Encodings). The construction is with respect to the following public parameters pp :

- A bound $B < 2^{\text{poly}(\lambda)}$ on encoded values;
- A polynomial ring $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ where $n < \text{poly}(\lambda)$ is a power-of-two;
- Two moduli $p > B \cdot \lambda^{\omega(1)}$, and $q = p \cdot \Delta$, where $\Delta > B \cdot p \cdot \lambda^{\omega(1)}$.

- Two distributions $\mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}}$ over $\mathcal{R}$ with coefficients bounded by $\text{poly}(\lambda)$.

$\text{pd} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s}, \mathbf{s}') \in \mathcal{D}_{\text{sk}}^2$: Compute the secret-switching public data as

$$\begin{aligned} a &\leftarrow \mathcal{R}_q, \mathbf{e} \leftarrow \mathcal{D}_{\text{err}}^2, b \leftarrow \mathbf{s} \cdot a + \mathbf{e} + \mathbf{s}' \cdot \Delta \text{ over } \mathcal{R}_q \\ \text{pd} &:= (a, b, \alpha, \beta), \text{ where } \alpha \leftarrow \mathcal{R}_q^2, \beta \leftarrow \mathcal{R}_p^2. \end{aligned}$$

$\langle \mathbf{s}' \cdot x \rangle_{G/E} \leftarrow \text{SecSwitch}^{\text{pd}}(\langle \mathbf{s} \cdot x \rangle_{G/E}, \langle x \rangle_{G/E})$: Read (a, b) from pd and compute the following

$$\begin{aligned} \langle \mathbf{s} \cdot x \rangle'_{G/E} &\leftarrow \langle \mathbf{s} \cdot x \rangle_{G/E} + \beta, \langle x \rangle'_{G/E} \leftarrow \langle x \rangle_{G/E} + \beta, && \text{over } \mathcal{R}_p, \\ \langle \mathbf{s}' \cdot x \rangle'_{G/E} &\leftarrow a \cdot \langle \mathbf{s} \cdot x \rangle'_{G/E} - b \cdot \langle x \rangle'_{G/E} + \alpha && \text{over } \mathcal{R}_q, \\ \langle \mathbf{s}' \cdot x \rangle_{G/E} &\leftarrow \lfloor \langle \mathbf{s}' \cdot x \rangle'_{G/E} / \Delta \rfloor && \text{over } \mathcal{R}_p. \end{aligned}$$

We show the following correctness lemma.

Lemma 5.29 (Correctness of Secret Switching). *There exists a negligible function negl such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, encoded value $x \in \mathcal{R}_B$, secrets $\mathbf{s}, \mathbf{s}' \in \mathcal{D}_{\text{sk}}$, additive shares $\langle \mathbf{s} \cdot x \rangle$, the following holds:*

$$\Pr \left[\begin{array}{l} \langle \mathbf{s}' \cdot x \rangle_E = \langle \mathbf{s}' \cdot x \rangle_G + \mathbf{s}' \cdot x \\ \langle \mathbf{s}' \cdot x \rangle_G \leftarrow \text{SecSwitch}(\langle \mathbf{s} \cdot x \rangle_G), \\ \langle \mathbf{s}' \cdot x \rangle_E \leftarrow \text{SecSwitch}(\langle \mathbf{s} \cdot x \rangle_E) \end{array} \middle| \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \end{array} \right] > 1 - \text{negl}(\lambda)$$

Proof. We first claim that the intermediate values $\langle \mathbf{s} \cdot x \rangle'_{G/E}, \langle x \rangle'_{G/E}$ during SecSwitch form additive shares of $\mathbf{s} \cdot x$ and x over $\mathcal{R}$. This holds with except probability $O(n \cdot \|\mathbf{s} \cdot x\|_\infty / p)$ (over the randomness of β) by the lifting lemma (Lemma 2) of [BKS19]. By our parameter setting, we have

$$n \cdot \|\mathbf{s} \cdot x\|_\infty \leq \text{poly}(\lambda) \cdot B \ll p.$$

So the failure probability is negligible. We next check that the $\langle \mathbf{s}' \cdot x \rangle'_{G/E}$ values form additive shares of $\mathbf{s}' \cdot x \cdot \Delta + \text{err}$ over $\mathcal{R}_q$.

$$\begin{aligned} \langle \mathbf{s}' \cdot x \rangle'_E &= a \cdot \langle \mathbf{s} \cdot x \rangle'_E - b \cdot \langle x \rangle'_E + \alpha \\ (\text{By Construction and Above}) &= \langle \mathbf{s}' \cdot x \rangle'_G + a \cdot \mathbf{s} \cdot x - b \cdot x \\ (\text{By Setup}) &= \langle \mathbf{s}' \cdot x \rangle'_G + a \cdot \mathbf{s} \cdot x - (\mathbf{s} \cdot a + \mathbf{e} + \mathbf{s}' \cdot \Delta) \cdot x \\ &= \langle \mathbf{s}' \cdot x \rangle'_G + \mathbf{s}' \cdot x \cdot \Delta + \text{err}, \end{aligned}$$

where $\|\text{err}\|_\infty < \text{poly}(\lambda) \cdot B$. Finally, by the rounding lemma (Lemma 1) of [BKS19], the derived values $\langle \mathbf{s}' \cdot x \rangle_{G/E}$ form additive shares of $\mathbf{s}' \cdot x$ as desired, except with probability $O(n \cdot \|\text{err}\| / \Delta)$ (over the randomness of α). By our parameter setting, we have

$$n \cdot \|\text{err}\| \leq \text{poly}(\lambda) \cdot B \ll \Delta.$$

So correctness of SecSwitch holds except with negligible probability. $\square$

Construction 5.30 (Fixed Factor Hashing for aHMAC Encodings). The construction is with respect to the same public parameters $(B, \mathcal{R}, p, q, \mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}})$ as specified in Construction 5.28, and additionally:

- A distribution $\mathcal{D}_{\text{pub}}$ (over $\mathcal{R}$) of elements with uniform binary coefficients.
- A hash function $\text{Hash} : \mathcal{R}^{W \cdot \lceil \log q \rceil} \rightarrow \mathcal{R}^{\lceil \log q \rceil}$ with compression factor $W < \text{poly}(\lambda)$, defined by $W' = W \cdot \lceil \log q \rceil$ ring elements $\mathbf{b}_i \in \mathcal{D}_{\text{pub}}^{\lceil \log q \rceil}$ for $i \in [W']$;

$$\text{Hash}(\mathbf{x} \in \mathcal{R}_B^{W'}) := \sum_i \mathbf{b}_i \cdot \mathbf{x}[i] \text{ over } \mathcal{R}. \quad (5.6)$$

$\text{pd} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s}, \mathbf{s}' \in \mathcal{D}_{\text{sk}}^2)$: Read $\{\mathbf{b}_i\}_{i \in [W']}$ from pp , and generate ciphertexts $\{\text{ct}_{i,j,b}\}_{i,j \in [W'], b \in \{0,1\}}$ used for extracting hashed values:

$$\begin{aligned} \mathbf{a}_j &\leftarrow \mathcal{R}_q^{\lceil \log q \rceil}, \mathbf{e}_j \leftarrow \mathcal{D}_{\text{err}}^{\lceil \log q \rceil}, e_{i,j} \leftarrow \mathcal{D}_{\text{err}} \\ \text{ct}_{i,j,b} &\leftarrow (\mathbf{s}[b] \cdot \mathbf{a}_j + \mathbf{e}_j)^\top \cdot \mathbf{b}_i + e_{i,j} - \mathbf{s}'[b] \cdot \mathbf{u}_j[i] \cdot \Delta \text{ over } \mathcal{R}_q, \end{aligned}$$

where $\mathbf{u}_j \in \{0, 1\}^{W'}$ denote the j -th unit vector. Finally, sample random offsets α, β , and output

$$\text{pd} := (\text{pp}, \{\text{ct}_{i,j,b}\}, \{\mathbf{a}_j\}, \alpha, \beta), \text{ where } \alpha \leftarrow \mathcal{R}_q^2, \beta \leftarrow \mathcal{R}_p^{2 \cdot \lceil \log q \rceil}.$$

$\langle \mathbf{s} \otimes \mathbf{h} \rangle_{G/E} \leftarrow \text{Compress}^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_{G/E})$: Read $\{\mathbf{b}_i\}_{i \in [W']}$ from pd , and compute

$$\langle \mathbf{s} \otimes \mathbf{h} \rangle_{G/E} \leftarrow \sum_i \langle \mathbf{s} \cdot \mathbf{x}[i] \rangle_{G/E} \otimes \mathbf{b}_i \text{ over } \mathcal{R}_p.$$

$\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G \leftarrow \text{Extract}_G^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{h} \rangle_G, j \in [W])$: Read $\mathbf{a}_j \in \mathcal{R}_q^{\lceil \log q \rceil}$ and $\alpha \in \mathcal{R}_q^2, \beta \in \mathcal{R}_p^{2 \cdot \lceil \log q \rceil}$ from pd , and compute

$$\begin{aligned} \langle \mathbf{s} \otimes \mathbf{h} \rangle'_G &\leftarrow \langle \mathbf{s} \otimes \mathbf{h} \rangle_G + \beta && \text{over } \mathcal{R}_p, \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_G &\leftarrow \left(\sum_{k \in [\lceil \log q \rceil]} \mathbf{a}_j[k] \cdot \langle \mathbf{s} \otimes \mathbf{h}[k] \rangle'_G \right) + \alpha && \text{over } \mathcal{R}_q, \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G &\leftarrow \lfloor \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_G / \Delta \rfloor && \text{over } \mathcal{R}_p. \end{aligned}$$

$\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E \leftarrow \text{Extract}_E^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{h} \rangle_E, j, \mathbf{x} \in \mathcal{R}_B^{W'})$: First read $\{\text{ct}_{i,j,b}\}_{i,j \in [W'], b \in \{0,1\}}$ from pd , and compute

$$\text{diff}_j[b] \leftarrow \sum_i \text{ct}_{i,j,b} \cdot \mathbf{x}[i] \text{ over } \mathcal{R}_q.$$

Next read $\alpha \in \mathcal{R}_q^2, \beta \in \mathcal{R}_p^{2 \cdot \lceil \log q \rceil}$ from pd , and compute

$$\begin{aligned} \langle \mathbf{s} \otimes \mathbf{h} \rangle'_E &\leftarrow \langle \mathbf{s} \otimes \mathbf{h} \rangle_E + \beta && \text{over } \mathcal{R}_p, \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_E &\leftarrow \left(\sum_{k \in [\lceil \log q \rceil]} \mathbf{a}_j[k] \cdot \langle \mathbf{s} \otimes \mathbf{h}[k] \rangle'_E \right) - \text{diff}_j + \alpha && \text{over } \mathcal{R}_q, \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E &\leftarrow \lfloor \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_E / \Delta \rfloor && \text{over } \mathcal{R}_p. \end{aligned}$$

We show the following correctness lemma.

Lemma 5.31 (Correctness of Fixed Factor Hashing). *There exists a negligible function negl such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, encoded vectors $\mathbf{x} \in \mathcal{R}_B^{W'}$, secrets $\mathbf{s}, \mathbf{s}' \in \mathcal{D}_{\text{sk}}^2$, additive shares $\langle \mathbf{s} \otimes \mathbf{x} \rangle, \langle \mathbf{s} \cdot \text{Hash}(\mathbf{x}) \rangle$, and index $j \in [W']$, the following holds:*

$$\begin{aligned} \Pr \left[\langle \mathbf{s} \otimes \mathbf{h} \rangle_E = \langle \mathbf{s} \otimes \mathbf{h} \rangle_G + \mathbf{s} \otimes \text{Hash}(\mathbf{x}) \mid \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \langle \mathbf{s} \otimes \mathbf{h} \rangle_G \leftarrow \text{Compress}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_G), \\ \langle \mathbf{s} \otimes \mathbf{h} \rangle_E \leftarrow \text{Compress}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_E) \end{array} \right] &> 1 - \text{negl}(\lambda) \\ \Pr \left[\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E = \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G + \mathbf{s}' \cdot \mathbf{x}[j] \mid \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G \leftarrow \text{Extract}_G(\langle \mathbf{s} \otimes \text{Hash}(\mathbf{x}) \rangle_G, j), \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E \leftarrow \text{Extract}_E(\langle \mathbf{s} \otimes \text{Hash}(\mathbf{x}) \rangle_E, j, \mathbf{x}) \end{array} \right] &> 1 - \text{negl}(\lambda) \end{aligned}$$

Proof. The correctness of Compress follows directly from the definition of Hash. We verify correctness of Extract. First, by the lifting lemma from [BKS19] and analogous to the proof of Lemma 5.29, the intermediate values $\langle \mathbf{s} \otimes \mathbf{h} \rangle'_{G/E}$ form additive shares of $\mathbf{s} \otimes \mathbf{h}$ over $\mathcal{R}$ (and hence over $\mathcal{R}_q$), except with negligible probability. We show that the values $\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_{G/E}$ form additive shares of $\mathbf{s}' \cdot \mathbf{x}_j \cdot \Delta + \text{err}$ over $\mathcal{R}_q$.

$$\begin{aligned}
\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_E &:= \sum_k \mathbf{a}_j[k] \cdot \langle \mathbf{s} \otimes \mathbf{h}[k] \rangle'_E - \text{diff}_j + \alpha \\
(\text{By above}) &= \sum_k \mathbf{a}_j[k] \cdot (\langle \mathbf{s} \otimes \mathbf{h}[k] \rangle'_G + \mathbf{s} \otimes \mathbf{h}[k]) - \text{diff}_j + \alpha \\
&= \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_G + \mathbf{s} \cdot \mathbf{a}_j^\top \cdot \mathbf{h} - \text{diff}_j \\
(\text{By Setup}) &= \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_G + \mathbf{s} \cdot \mathbf{a}_j^\top \cdot \mathbf{h} - \sum_i \mathbf{s} \cdot (\mathbf{a}_j^\top \cdot \mathbf{b}_i) \cdot \mathbf{x}[i] \\
&\quad + \sum_i (\mathbf{e}_j^\top \cdot \mathbf{b}_i - e_{i,j}) \cdot \mathbf{x}[i] + \sum_i \mathbf{s}' \cdot \mathbf{u}_j[i] \cdot \Delta \cdot \mathbf{x}[i] \\
(\text{By Def. of Hash}) &= \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_G + \mathbf{s}' \cdot \mathbf{x}[j] \cdot \Delta + \text{err},
\end{aligned}$$

where $\|\text{err}\|_\infty < \text{poly}(\lambda) \cdot B$. By the rounding lemma from [BKS19] and analogous to the proof of Lemma 5.29 we conclude that the values $\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle'_{G/E}$ form additive share of $\mathbf{s}' \cdot \mathbf{x}_j$ as desired, except with negligible probability. $\square$

For our security proof later, and similarly to the group instantiation (Section 5.2) we show the following lemma.

Lemma 5.32 (Security of Fixed Factor Hashing). *Under the RLWE assumption, every efficient adversary $\mathcal{A}$ has negligible advantage in distinguishing the following experiments $\text{Exp}_{\mathcal{A}}^b(1^\lambda)$, for $b \in \{0, 1\}$*

- Sample public parameters pp , and sends them to $\mathcal{A}$, who returns exponents $\mathbf{s}' \in \mathcal{D}_{\text{sk}}^2$ and an arbitrary number of $\{\mathbf{s}^{(i)} \in \mathcal{D}_{\text{sk}}^2\}_i$.
- When $b = 0$, sample random exponents $\mathbf{s} \leftarrow \mathcal{D}_{\text{sk}}^2$ and send $\text{pd}, \{\text{pd}^{(i)}\}$ to $\mathcal{A}$, computed as follows

$$\text{pd} \leftarrow \text{FH.Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \text{pd}^{(i)} \leftarrow \text{SS.Setup}(1^\lambda, \mathbf{s}, \mathbf{s}^{(i)}).$$

- When $b = 1$, sample $\text{pd}, \{\text{pd}^{(i)}\}$ with random $\mathcal{R}_q$ elements and send them to $\mathcal{A}$.

Proof. We sketch three hybrid experiments, with $\text{Hyb}_0 \equiv \text{Exp}_{\mathcal{A}}^0(1^\lambda)$, and $\text{Hyb}_2 \equiv \text{Exp}_{\mathcal{A}}^1(1^\lambda)$.

- Hyb_0 : In the experiment $\text{Exp}_{\mathcal{A}}^0(1^\lambda)$, pd consists of an RLWE ciphertext of the form $a, b := \mathbf{s} \cdot a + \mathbf{e} + f(\mathbf{s}')$, for some function f , and $\text{pd}^{(i)}$ consists of RLWE ciphertexts of the form $a^{(i)}, \mathbf{c}^{(i)} := \mathbf{r}^{(i),\top} \cdot \mathbf{b}^{(i)} + \mathbf{e}^{(i)} + f^{(i)}(\mathbf{s}^{(i)})$ for some functions $f^{(i)}$, and where $\mathbf{r}^{(i)} := \mathbf{s} \otimes \mathbf{a}^{(i)} + \mathbf{E}^{(i)}$.
- Hyb_1 : Sample the b element and $\mathbf{r}^{(i)}$ vectors from the above as random $\mathcal{R}_q$ elements instead. By RLWE, we have $\text{Hyb}_1 \approx_c \text{Hyb}_0$.
- Hyb_2 : Sample the $\mathbf{c}^{(i)}$ vectors as random $\mathcal{R}_q$ elements instead. Note that the terms $\mathbf{r}^{(i),\top} \cdot \mathbf{b}^{(i)} + \mathbf{e}^{(i)}$, where $\mathbf{r}^{(i)} \leftarrow \mathcal{R}_q^{\lceil \log q \rceil}$ and $\mathbf{b}^{(i)} \leftarrow \mathcal{D}_{\text{pub}}^{\lceil \log q \rceil}$, form non-uniform (R)LWE samples as defined in [BLMR13]. Adopting the arguments there for LWE to the RLWE setting, we can conclude that such non-uniform RLWE samples are pseudorandom under the standard RLWE assumption. We have $\text{Hyb}_2 \approx_c \text{Hyb}_1$.

A standard hybrid argument then concludes the proof. $\square$

Construction 5.33 (Hashing for aHMAC Encodings). The construction is with respect to the following public parameters pp :

- A bound γ on the level of recursive compression.

- A polynomial ring $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ where $n < \text{poly}(\lambda)$ is a power-of-two;
- Two distributions $\mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}}$ over $\mathcal{R}$ with coefficients bounded by $\text{poly}(\lambda)$.
- A hash function $\text{Hash} : \mathcal{R}^{W'} \rightarrow \mathcal{R}^{\lceil \log q \rceil}$ defined as in Equation 5.6 for $W' = W \lceil \log q \rceil$, and $W = \lambda$.

It uses the secret switching algorithms (Construction 5.28, denoted with prefix SS) and the fixed factor hashing algorithms (Construction 5.30, denoted with prefix FH) as ingredients, with respect to the above and the following additional parameters.

- The $\mathcal{D}_{\text{pub}}$ distribution as specified in Construction 5.30.
- A bound $B = (n \cdot W')^\gamma$ on encoded values.
- Two moduli $p = B \cdot \lambda^{\omega(1)}$, and $q = p \cdot \Delta$, where $\Delta = B \cdot p \cdot \lambda^{\omega(1)}$.

For a string $\mathbf{x} \in \{0, 1\}^\ell$ of length $\ell < W^\gamma$, we define its hash $h_{\mathbf{x}}$ to be the (γ) level recursively hashed value of $\mathbf{x}$ by Hash (Equation 5.6), viewing each bit x_i as a ring element. For an index $i < W^\gamma$, we define $\text{path}_i \in [W]^\gamma$ to be a sequence of indices corresponding to $\mathbf{x}[i]$ during recursive hashing.

$$\begin{aligned} \text{path}_i[0] &= i \bmod W', \\ \text{path}_i[t] &= \lfloor i / (\lceil \log p \rceil \cdot W^t) \rfloor \bmod W \text{ for } t \in [\gamma], t > 0 \end{aligned} \quad (5.7)$$

$\text{pd} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s}, \mathbf{s}' \in \mathcal{D}_{\text{sk}}^2)$: Sample secrets $\{\mathbf{s}^{(t)}\}_{t \in [\gamma+1]}$, prepare public data for γ instances of fixed factor hashing, and an for secret switching from $\mathbf{s}$ to $\mathbf{s}'$:

$$\begin{aligned} \mathbf{s}^{(0)}, &\leftarrow \mathbf{s}, \mathbf{s}^{(\gamma)} \leftarrow \mathbf{s}', \mathbf{s}^{(t)} \leftarrow \mathcal{D}_{\text{sk}}^2 \text{ for } t = 1, \dots, \gamma - 1 \\ \text{FH.pd}^{(t)} &\leftarrow \text{FH.Setup}(1^\lambda, \mathbf{s}^{(t)}, \mathbf{s}^{(t+1)}) \text{ for } t \in [\gamma], \\ \text{SS.pd} &\leftarrow \text{SS.Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'). \end{aligned}$$

Output $\text{pd} := (\text{SS.pd}, \{\text{FH.pd}^{(t)}\}_{t \in [\gamma]})$.

$\langle \mathbf{s}' \cdot x_i \rangle_G \leftarrow \text{HRead}_G^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{h} \rangle_G, i)$: Define path_i as in Equation 5.7, and assuming $i < W^\gamma$. Let $\langle \mathbf{s}^{(0)} \otimes \mathbf{h}^{(\gamma)} \rangle_G \leftarrow \langle \mathbf{s} \otimes \mathbf{h} \rangle_G$, and proceed as follows for $j \in [\gamma - 1]$:

- Read $\text{FH.pd}^{(j)}$ from pd , and compute for $k \in [\lceil \log q \rceil]$

$$\langle \mathbf{s}^{(j+1)} \cdot \mathbf{h}^{(\gamma-j-1)}[k] \rangle_G \leftarrow \text{FH.Extract}_G^{\text{FH.pd}^{(j)}}(\langle \mathbf{s}^{(j)} \otimes \mathbf{h}^{(\gamma-j)} \rangle_G, \text{path}_i[\gamma - j - 1] \cdot \lceil \log q \rceil + k). \quad (5.8)$$

Finally, compute $\langle \mathbf{s}' \cdot x_i \rangle_G := \langle \mathbf{s}^{(\gamma)} \cdot x_i \rangle_G \leftarrow \text{FH.Extract}_G^{\text{FH.pd}^{(\gamma-1)}}(\langle \mathbf{s}^{(\gamma-1)} \otimes \mathbf{h}^{(1)} \rangle_G, \text{path}_i[0])$.

$\langle \mathbf{s}' \cdot x_i \rangle_E \leftarrow \text{HRead}_E^{\text{pd}}(\langle \mathbf{s} \cdot h \rangle_E, i, \mathbf{x})$: Analogous to HRead_G , except replacing Equation 5.8 with the following:

$$\begin{aligned} \langle \mathbf{s}^{(j+1)} \cdot \mathbf{h}^{(\gamma-j-1)}[k] \rangle_E &\leftarrow \text{FH.Extract}_E^{\text{FH.pd}^{(j)}}(\langle \mathbf{s}^{(j)} \otimes h^{(\gamma-j)} \rangle_E, \text{path}_i[\gamma - j - 1] \cdot \lceil \log q \rceil + k, \boxed{\overline{\mathbf{h}}^{(\gamma-j)}}), \\ \langle \mathbf{s}^{(\gamma)} \cdot x_i \rangle_E &\leftarrow \text{FH.Extract}_E^{\text{FH.pd}^{(\gamma-1)}}(\langle \mathbf{s}^{(\gamma-1)} \cdot h^{(1)} \rangle_E, \text{path}_i[0], \boxed{\overline{\mathbf{h}}^{(1)}}), \end{aligned}$$

where $\overline{\mathbf{h}}^{(t-j)} \in \mathcal{R}^{W'}$ is the pre-image of the intermediate hash $h^{(t-j)}$, which can be derived from the input $\mathbf{x}$.

$\langle \mathbf{s}' \otimes \mathbf{h} \rangle_G \leftarrow \text{HCompress}_G^{\text{pd}}(\langle \mathbf{s} \otimes \mathbf{x} \rangle_G)$: Read SS.pd from pd . For every index $i \in |\mathbf{x}|$ run SecSwitch to switch the encoding secret from $\mathbf{s}$ to $\mathbf{s}'$:

$$\langle \mathbf{s}' \cdot \mathbf{x}[i] \rangle_G \leftarrow \text{SS.SecSwitch}^{\text{SS.pd}}(\langle \mathbf{s} \cdot \mathbf{x}[i] \rangle_G, \langle \mathbf{x}[i] \rangle_G), \text{ where } \langle \mathbf{x}[i] \rangle_G \leftarrow 0,$$

Next, recursively compress the $\langle \mathbf{s}' \otimes \text{diff} \rangle_G$ by running FH.Compress . Proceed as follows for $j = [\gamma]$:

- Read $\text{FH.pd}^{(j)}$ from pd , and compute the next level hash $\langle \mathbf{s}' \otimes \mathbf{h}^{(j+1)} \rangle_G$ as

$$\langle \mathbf{s}' \otimes \mathbf{h}^{(j+1)} \rangle_G[i] \leftarrow \text{FH.Compress}_G^{\text{FH.pd}^{(j)}}(\langle \mathbf{s}' \otimes \mathbf{h}_i^{(j)} \rangle_G),$$

where $\mathbf{h}_i^{(j)} := \text{flatten}(\mathbf{h}^{(j)})[i \cdot W' : (i+1) \cdot W']$.

Finally, output the obtained $\langle \mathbf{s}' \otimes \mathbf{h}^{(\gamma)} \rangle_G$ as $\langle \mathbf{s}' \otimes \mathbf{h} \rangle_G$.

$\langle \mathbf{s}' \otimes \mathbf{h} \rangle_E \leftarrow \text{HCompress}_E^{\text{pd}}(\langle \mathbf{s} \cdot \mathbf{x} \rangle_E, \mathbf{x})$: Analogous to HCompress , except when running SS.SecSwitch , set $\langle \mathbf{x}[i] \rangle_E \leftarrow \mathbf{x}[i]$, and when running FH.Compress_E , set $\langle \mathbf{h}_i^{(j)} \rangle_E \leftarrow \mathbf{h}_i^{(j)}$ where $\mathbf{h}_i^{(j)}$ is derived from the input $\mathbf{x}$ by recursive hashing.

We note the following correctness and security lemmas. The security lemma follows from applying Lemma 5.32 γ times.

Lemma 5.34 (Correctness of Hashing). *There exists a negligible function negl such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, encoded vectors $\mathbf{x} \in \{0, 1\}^{W^\gamma}$, index $j \in [W^\gamma]$, secrets $\mathbf{s}, \mathbf{s}' \in \mathcal{D}_{\text{sk}}^2$, additive shares $\langle \mathbf{s} \cdot \mathbf{x} \rangle$, $\langle \mathbf{s} \otimes \mathbf{h} \rangle$, where $\mathbf{h}$ is the γ level recursive hash of $\mathbf{x}$,*

$$\Pr \left[\langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E = \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G + \mathbf{s}' \cdot \mathbf{x}[j] \mid \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_G \leftarrow \text{HRead}_G(\langle \mathbf{s} \otimes \mathbf{h} \rangle_G, j), \\ \langle \mathbf{s}' \cdot \mathbf{x}_j \rangle_E \leftarrow \text{HRead}_E(\langle \mathbf{s} \otimes \mathbf{h} \rangle_E, j, \mathbf{x}), \end{array} \right] > 1 - \text{negl}(\lambda)$$

$$\Pr \left[\langle \mathbf{s}' \otimes \mathbf{h} \rangle_E = \langle \mathbf{s}' \otimes \mathbf{h} \rangle_G + \mathbf{s}' \otimes \mathbf{h} \mid \begin{array}{l} \text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{s}'), \\ \langle \mathbf{s}' \otimes \mathbf{h} \rangle_G \leftarrow \text{HCompress}_G(\langle \mathbf{s} \cdot \mathbf{x} \rangle_G), \\ \langle \mathbf{s}' \otimes \mathbf{h} \rangle_E \leftarrow \text{HCompress}_E(\langle \mathbf{s} \cdot \mathbf{x} \rangle_E, \mathbf{x}), \end{array} \right] > 1 - \text{negl}(\lambda).$$

Lemma 5.35 (Security of Hashing). *Under the RLWE assumption, every efficient adversary $\mathcal{A}$ has negligible advantage in distinguishing the following experiments $\text{Exp}_{\mathcal{A}}^b(1^\lambda)$, for $b \in \{0, 1\}$*

- Sample public parameters pp , and sends them to $\mathcal{A}$, who returns secret elements $\mathbf{s}' \in \mathcal{D}_{\text{sk}}^2$.
- When $b = 0$, sample random exponents $\mathbf{s} \leftarrow \mathcal{D}_{\text{sk}}^2$ and send pd to $\mathcal{A}$, computed as follows

$$\text{pd} \leftarrow \text{FH.Setup}(1^\lambda, \mathbf{s}, \mathbf{s}').$$

- When $b = 1$, sample pd with random $\mathcal{R}_q$ elements and send them to $\mathcal{A}$.

Instantiation of HGate. Again, the sub-protocol HGate in our lattice instantiation is the same as the previous section (Figure 3), which runs the local algorithms EvalKey , EvalTag (Construction 5.24) from [ILL25b]. The only difference is the local algorithms EvalKey , EvalTag now use lattice based building blocks, MultKey , MultTag from Construction 5.36 as described below. The local algorithms EvalKey , EvalTag satisfy correctness analogous to Lemma 5.25 and security analogous to Lemma 5.26 (under RLWE instead of DDH).

Construction 5.36 (aHMAC Multiplication under Lattices [ILL25c]). The construction is with respect to the same public parameters $\text{pp} = (B, \mathcal{R}, p, q, \mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}})$ as specified in Construction 5.28.

$\text{pd} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s}, \mathbf{s}', \mathbf{s}'' \in \mathcal{D}_{\text{sk}})$: takes three secrets $\mathbf{s}, \mathbf{s}', \mathbf{s}'' \in \mathcal{D}_{\text{sk}}$, and computes pd as follows.

$$\begin{aligned} a &\leftarrow \mathcal{R}_q, e_1, e_2, e_3 \leftarrow \mathcal{D}_{\text{err}}, \alpha \leftarrow \mathcal{R}_q, \beta \leftarrow \mathcal{R}_p, \\ b &\leftarrow \mathbf{s} \cdot a + e_1, b' \leftarrow \mathbf{s}' \cdot a + e_2, \\ c &\leftarrow \mathbf{s}' \cdot b + e_3 - \mathbf{s}'' \cdot \Delta, \\ \text{pd} &:= (\text{pp}, \alpha, \beta, a, b, b', c). \end{aligned}$$

$\langle s'' \cdot z \rangle_G \leftarrow \text{MultKey}^{\text{pd}}(\langle s \cdot x \rangle_G, \langle s' \cdot y \rangle_G)$: takes as inputs public data pd and two ring elements representing the 0-shares of some authenticated shares $\langle s \cdot x \rangle_G, \langle s' \cdot y \rangle_E \in \mathcal{R}$ w.r.t. some secrets $s, s' \in \mathcal{R}$. It outputs the 0-share of an authenticated share $\langle s'' \cdot z \rangle_G$.

$\langle s'' \cdot z \rangle_E \leftarrow \text{MultTag}^{\text{pd}}(\langle s \cdot x \rangle_E, \langle s' \cdot y \rangle_E, x, y)$: takes as input public data pd , the 1-shares of some authenticated shares $\langle s \cdot x \rangle_E, \langle s' \cdot y \rangle_E$ w.r.t. some secret s, s' , and the shared values $x, y \in \mathcal{R}$. It outputs the 1-share of an authenticated share $\langle s'' \cdot z \rangle_E$.

Lemma 5.37 (Correctness of aHMAC Multiplication [ILL25c]). *There exists a negligible function $\text{negl}(\lambda)$ such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, secret elements $s, s', s'' \in \mathcal{D}_{\text{sk}}$, inputs $x, y \in \mathcal{R}_B$ such that $\|xy\|_\infty < B$, and additive shares (over $\mathcal{R}$) $\langle sx \rangle_G, \langle sx \rangle_E, \langle s'y \rangle_G, \langle s'y \rangle_E$, the following holds:*

$$\Pr \left[\begin{array}{l} w_E = w_G + s'' \cdot xy \\ (\text{over } \mathcal{R}) \end{array} \middle| \begin{array}{l} \text{pd} \leftarrow \text{Setup}(\text{pp}, s, s', s'') \\ w_G = \text{MultKey}(\langle sx \rangle_G, \langle s'y \rangle_G) \\ w_E = \text{MultTag}(\langle sx \rangle_E, \langle s'y \rangle_E, x, y) \end{array} \right] > 1 - \text{negl}(\lambda).$$

Instantiation of Setup and Enc. We now describe the $\text{Setup}(1^\lambda, 1^D, |C|)$ procedure that selects public parameters $\text{pp} := (1^D, \mathcal{R}, p, q, \mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}}, \text{Hash}, \gamma = \log_\lambda |C|)$ for all sub-protocols HRead, HCompress, HGate.

- Let $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$ be a polynomial ring, where $n(\lambda) < \text{poly}(\lambda)$ is a power-of-two, and $\mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}}$ be truncated discrete gaussian distribution over $\mathcal{R}$ with coefficients bounded by $\text{poly}(\lambda)$.
- Select moduli p, q satisfying requirements from Construction 5.28, under $B = \lambda^{\omega(1)} > (n \cdot \lambda \cdot \lceil \log q \rceil)^\gamma$.
- Let $\mathcal{D}_{\text{pub}}$ be as specified in Construction 5.30. Sample $b_i \leftarrow \mathcal{D}_{\text{pub}}$ for $i \in \lambda \cdot \log q$, which define a global hash function $\text{Hash} : \mathcal{R}^{\lambda \cdot \lceil \log q \rceil} \rightarrow \mathcal{R}^{\lceil \log q \rceil}$ with compression factor λ as specified in Equation 5.6.

The sampling of gk , and the $\text{Enc}(\text{gk}, \mathbf{x})$ procedure that produces encodings $\llbracket \mathbf{x} \rrbracket$, public data pd is analogous to the previous section (Equation 5.3, 5.4, and 5.5), except the secrets are sampled from $\mathcal{D}_{\text{sk}}^2$ instead of $[p]^2$.

Garbler Depth Complexity. Similarly to the group instantiation, we review the depth of each of the sub-protocols HRead, HCompress, HGate.

HRead: Mainly consists of γ sequential calls to Extract.

Extract consists mainly of a ring multiplication over $\mathcal{R}_q$ and a coefficient rounding. Both of these steps have depth $O(\log \lambda)$.

We conclude the depth of HRead is $O(\gamma \cdot \log \lambda)$, where $\gamma = \log_\lambda |C|$.

HCompress: Mainly consists of a call to SecSwitch, and γ sequential calls to Compress.

SecSwitch similarly consists of two ring multiplications and a coefficient rounding, and has depth $O(\log \lambda)$.

Compress consists of a ring multiplication and a summation over λ ring elements. Both of these steps have depth $O(\log \lambda)$.

We conclude the depth of HCompress is also $O(\gamma \cdot \log \lambda)$.

HGate: Mainly consists of a call to SecSwitch and MultKey. The construction of MultKey from [ILL25b] consists of constant number of ring multiplications and a coefficient rounding, similarly to our SecSwitch construction, and hence has depth $O(\log \lambda)$.

Succinctness and Decomposability. Similarly to the group instantiation, the garbling material consists of the public data pd and a series of re-key materials $\{\text{rk}^{(d)}\}_{d \in [D]}$. Thanks to recursive hashing, the public data and re-key material for each depth $d \in [D]$ have a fixed size $\text{poly}(\lambda)$, independent of $|C|$. Overall, they have size $< D \cdot \text{poly}(\lambda)$.

The analysis for decomposability and the size of input encodings and dk are analogous to the group instantiation.

Security. The selective one-wayness can be shown analogously to that of the group instantiation, based on the standard RLWE assumption (Definition 5.12).

6 Non-Reuseable Partially Succinct Garbling

In this section, we will show the *private* analogue of the constructions in Section 5.

Theorem 6.1 (Non-Reuseable Partially Succinct Garbling under DDH). *Assuming any of the following:*

- *DDH in prime-order groups that are subgroups of $\mathbb{Z}_p^*$ for some prime p ;*
- *DDH in Paillier groups;*

And additionally a PRF in NC1, there exists a succinct garbling scheme satisfying the following.

- *Garbling sizes of a circuit $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$ with depth D satisfy:*

$$|\hat{C}| < (D + |C|^{2/3} \cdot D^{1/3}) \cdot \text{poly}(\lambda) + |C|, |\hat{x}| < \ell \cdot \text{poly}(\lambda), |\text{dk}| < \text{poly}(\lambda).$$

- *GarbC($1^\lambda, C$) can be computed in depth $\text{poly}(\lambda, \log |C|)$.*

Further assuming a local PRG with constant depth, then GarbC($1^\lambda, C$) can be computed in NC1. The instantiation under DDH in prime-order groups suffers a $1/\text{poly}(\lambda)$ correctness and privacy error, which can be removed via privacy amplification at the cost of increasing the depth of GarbC($1^\lambda, C$) to $\text{poly}(\lambda, \log C)$ again.

Theorem 6.2 (Non-Reuseable Partially Succinct Garbling under RLWE). *Assuming RLWE with respect to $\text{pp} = (\mathcal{R}, p, q, \mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}})$ as specified in Construction 5.28 under $B = 2$, and additionally a PRF in NC1, there exists a succinct garbling scheme satisfying the following.*

- *Garbling sizes of a circuit $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$ with depth D satisfy:*

$$|\hat{C}| < D \cdot \text{poly}(\lambda) + |C|, |\hat{x}| < \ell \cdot \text{poly}(\lambda), |\text{dk}| < \text{poly}(\lambda).$$

- *GarbC($1^\lambda, C$) can be computed in depth $\text{poly}(\lambda, \log |C|)$.*

Further assuming a local PRG with constant depth, then GarbC($1^\lambda, C$) can be computed in NC1.

6.1 Instantiation Under Groups

The starting point of our group instantiation is the (leveled) partially succinct garbling scheme of [ILL25b]. In particular, it contains a sub-protocol that allows two parties P_G, P_E who jointly hold additive shares of the form $\langle s \cdot \bar{x} \rangle, \langle s \cdot \bar{y} \rangle$, to derive additive shares $\langle s' \cdot \overline{g(x, y)} \rangle$, for any Boolean gate g , where the notation $\bar{x} := x \oplus r_x$ denotes a masked value x by a pseudo-random mask r_x . We will directly use this sub-protocol as our HGate protocol. Together with the HRead, HCompress sub-protocols from Section 5, we obtain a partially succinct garbling scheme (with privacy).

In more detail, the encoding of a wire value $\llbracket x \rrbracket$ for $x \in \{0, 1\}$ is the same as Section 5.2, except the wire value x is masked as $\bar{x}$. The Rekey steps are exactly the same as Section 5.2.

$$\llbracket x \rrbracket = (\llbracket x \rrbracket_G, \llbracket x \rrbracket_E), \text{ where } \llbracket x \rrbracket_G = (\langle s^{\text{id}} \cdot \bar{x} \rangle_G, \text{id}), \text{ and } \llbracket x \rrbracket_E = (\langle s^{\text{id}} \cdot \bar{x} \rangle_E, \text{id}, \bar{x}).$$

In the following, we first recall necessary preliminaries for the HGate sub-protocol adapted from [ILL25a], and then explain the Setup and Enc algorithms for our garbling scheme. (The HRead, HCompress sub-protocols remain the same as Section 5.2.)

Protocol HGate

The protocol runs between a garbler P_G and an evaluator P_E . It uses the following ingredients:

- The algorithms EvalKey, EvalTag from Construction 5.24.
- The algorithms ExtEval from Construction 6.3.
- A PRF : $\{0, 1\}^\lambda \times [2^\lambda] \rightarrow \{0, 1\}$ in NC1.

Inputs: P_G, P_E hold public data pd , an operator op , and jointly encodings of $\llbracket x \rrbracket, \llbracket y \rrbracket$, (with $id = (d, \text{InWire})$). We assume op also contains the input and output wire ids w_1, w_2, w_3 .

Outputs: P_G, P_E jointly hold encodings $\llbracket z \rrbracket$ (with $id' = (d, \text{OutWire})$), where z should equal $op(x, y)$.

- P_G reads $\text{GE.pd}^{(d)}$ from pd , $\langle s^{id} \cdot \bar{x} \rangle_G, \langle s^{id} \cdot \bar{y} \rangle_G$ from $\llbracket x \rrbracket_G, \llbracket y \rrbracket_G$, and runs

$$\langle s^{id''} \cdot u[i] \rangle_G \leftarrow \text{EvalKey}^{\text{GE.pd}^{(d)}}(eq_i, \langle s^{id} \cdot \bar{x} \rangle_G, \langle s^{id} \cdot \bar{y} \rangle_G) \text{ for } i \in [4],$$

where $id'' = (d, \text{TmpWire})$, indicating an intermediate secret $s^{id''} \in \{0, 1\}^{\lceil 3 \cdot \log p \rceil}$, and where eq_i is a binary gate checking if $(\bar{x}, \bar{y})$ represents i . Then, it reads $\text{HSS.pd}_{\text{key}}^{(d)}$ from pd , and runs

$$\langle z \rangle_G \leftarrow \text{ExtEval}^{\text{HSS.pd}_{\text{key}}^{(d)}}(f_{op}, \{\langle s^{id''} \cdot u[i] \rangle_G, \langle u[i] \rangle_G\}_{i \in [4]}),$$

where $\langle u[i] \rangle_G \leftarrow 0$, and f_{op} is defined as follows,

$$f_{op}(\text{key} \parallel \text{Bits}(s^{id'})) = \text{Bits}(1, s^{id'}) \otimes \text{Tb},$$

and $\text{Tb} \in \{0, 1\}^{1 \times 4}$ is a truth table of op , adjusted for masks over the inputs: ^a

$$\begin{aligned} \text{Tb}[(\bar{x}, \bar{y})] &= op(\bar{x} \oplus r_x, \bar{y} \oplus r_y) \oplus r_z, \text{ where} \\ r_x &\leftarrow \text{PRF}(\text{key}, w_1), r_y \leftarrow \text{PRF}(\text{key}, w_2), r_z \leftarrow \text{PRF}(\text{key}, w_3). \end{aligned} \quad (6.1)$$

It sets $\langle \bar{z} \rangle_G \leftarrow \langle z[0] \rangle_G$, and $\langle \text{Bits}(s^{id'}) \cdot \bar{z} \rangle_G \leftarrow \langle z[1:] \rangle_G$.

- P_E analogously runs EvalTag and ExtEval with $\langle u[i] \rangle_E \leftarrow u[i]$ to derive $\langle \bar{z} \rangle_E$ and $\langle s^{id'} \cdot \bar{z} \rangle_E$.
- P_G sends its share $\langle \bar{z} \rangle_G \bmod 2$ to P_E , who recovers $\bar{z}$. They respectively output $\llbracket z \rrbracket_G := (\langle s^{id'} \cdot \bar{z} \rangle_G, id')$, and $\llbracket z \rrbracket_E := (\langle s^{id'} \cdot \bar{z} \rangle_E, id', \bar{z})$.

^aWhen the output of op is an output of the circuit, we avoid the wire mask by setting $r_z = 0$.

Figure 4: The HGate sub-protocol with privacy.

Instantiation of HGate. We recall necessary preliminaries on the (leveled) construction of HGate (Figure 4) based on DDH from [ILL25b]. The sub-protocol consists first of local computations by both parties, running the EvalKey, EvalTag algorithms from Construction 5.24 and then the ExtEval algorithm which we describe below in Construction 6.3. In the end, the protocol incurs 1-bit communication from the garbler P_G to the evaluator P_E . In our description (Figure 4), the sub-protocol assume the parties P_G, P_E hold public data pd produced in the overall protocol by Enc.

Construction 6.3 (Extended HSS Evaluation under DDH [BGI16]). The construction is with respect to the same public parameters (B, δ, G, p, g) as Construction 5.13 under $B = 2$, and additionally

- $B_f < \text{poly}(\lambda)$ is a polynomial circuit size bound.

It uses as ingredients the DDLog algorithm from Lemma 5.5 instantiated with a suitable PRF : $\{0, 1\}^\lambda \times G \rightarrow \{0, 1\}^\lambda$

to achieve error probability $< \delta / \text{poly}(B_f)$ (s.t. the error probability over all invocations of DDLog is $< \delta$).

$\text{pd}_{\mathbf{m}} \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, \mathbf{s} \in \{0, 1\}^{\lceil 3 \log p \rceil}, \mathbf{m} \in \{0, 1\}^{\ell_m})$: First sample random coefficients $\mathbf{c} \leftarrow [p]^{\lceil 3 \log p \rceil}$, and define $\mathbf{s} = \mathbf{c}^\top \mathbf{s} \bmod p$. Then compute public data $\text{pd}_{\mathbf{m}}$ w.r.t. $\mathbf{m}$ as follows:

$$\begin{aligned} \mathbf{r} &\leftarrow \mathbb{Z}_p^{\ell_m}, \mathbf{R} \leftarrow \mathbb{Z}_p^{\ell_m \times \lceil 3 \log p \rceil}, \\ \text{pd}_{\mathbf{m}} &:= (\text{pp}, g^{\mathbf{c} \otimes \mathbf{r}}, g^{\mathbf{c} \otimes \mathbf{R}}, g^{\mathbf{s} \cdot \mathbf{r} + \mathbf{m}}, g^{\mathbf{s} \cdot \mathbf{R} + \mathbf{m} \otimes \mathbf{s}}). \end{aligned}$$

$\langle \mathbf{z} \rangle_{G/E} \leftarrow \text{ExtEval}^{\text{pd}_{\mathbf{m}}}(f : \{0, 1\}^{\ell_m} \rightarrow \{0, 1\}^{\ell_z \times \ell_x}, \{\langle \mathbf{s} \cdot \mathbf{x}_i \rangle_{G/E}, \langle \mathbf{x}_i \rangle_{G/E}\}_{i \in [\ell_x]})$: Takes an NC1 Boolean circuit f and ℓ_x shares $\langle \mathbf{s} \cdot \mathbf{x}_i \rangle_{G/E}, \langle \mathbf{x}_i \rangle_{G/E}$, w.r.t. the secret $\mathbf{s} \in \{0, 1\}^{\lceil 3 \log p \rceil}$ as input. Outputs an additive share $\langle \mathbf{z} \rangle_{G/E}$ where $\mathbf{z} \in \mathbb{Z}^{\ell_z}$ should equal

$$\mathbf{z} = f(\mathbf{m}) \cdot \mathbf{x}, \text{ where } \mathbf{x} := (\dots x_i \dots), \text{ and the matrix product is over } \mathbb{Z}.$$

Lemma 6.4 (Correctness of Extended HSS Evaluation [BGI16]). *There exists a negligible function $\text{negl}(\lambda)$ such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, secret string $\mathbf{s} \in \{0, 1\}^{\lceil 3 \log p \rceil}$, NC1 Boolean circuit $f : \{0, 1\}^{\ell_m} \rightarrow \{0, 1\}^{\ell_x}$ with size $|f| \leq B_f$, strings $\mathbf{m} \in \{0, 1\}^{\ell_m}$, $\mathbf{x} \in \{0, 1\}^{\ell_x}$, and additive shares $\langle \mathbf{s} \otimes \mathbf{x} \rangle_G, \langle \mathbf{s} \otimes \mathbf{x} \rangle_E$, following holds:*

$$\Pr \left[\begin{array}{c} \mathbf{w}_E = \mathbf{w}_G + f(\mathbf{m}) \cdot \mathbf{x} \\ (\text{over } \mathbb{Z}_p) \end{array} \middle| \begin{array}{l} \text{pd}_{\mathbf{m}} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{m}) \\ \mathbf{w}_G := \text{ExtEval}(f, \{\langle \mathbf{s} \otimes \mathbf{x}[i] \rangle_G, \langle \mathbf{x}[i] \rangle_G\}), \\ \mathbf{w}_E := \text{ExtEval}(f, \{\langle \mathbf{s} \otimes \mathbf{x}[i] \rangle_E, \langle \mathbf{x}[i] \rangle_E\}) \end{array} \right] > 1 - \text{negl}(\lambda) - \delta(\lambda).$$

Remark 6.5. We note a mismatch between our encoding format, which requires a secret vector $\mathbf{s} \in [p]^2$, and the extended HSS evaluation interface, which requires a secret string $\mathbf{s} \in \{0, 1\}^{\lceil 3 \log p \rceil}$. We resolve this issue by introducing an intermediate encoding format, indicated by `TmpWire`, using a string as the secret as required by the HSS evaluation interface. Such intermediate encodings will only exist as outputs of the `EvalKey`, `EvalTag` algorithms (Construction 5.24), and as inputs to the `ExtEval` algorithms.¹² (See our HGate sub-protocol, Figure 4, and our instantiation of the overall Setup algorithm, Equation 6.2).

For our security proof later, we note here that the public data pd produced by the Setup of extended HSS evaluations under a random secret $\mathbf{s} \in \{0, 1\}^{\lceil 3 \log p \rceil}$ are computationally indistinguishable from random. The following lemma follows directly from the security of the BHHO [BHHO08] crypto-system. We refer to [BHHO08] for details.

Lemma 6.6 (Security of HSS Evaluation). *Under the DDH assumption, every efficient adversary $\mathcal{A}$ has negligible advantage in distinguishing the following experiments $\text{Exp}_{\mathcal{A}}^b(1^\lambda)$, for $b \in \{0, 1\}$*

- Sample public parameters pp , and sends them to $\mathcal{A}$, who returns bits $\mathbf{m} \in \{0, 1\}^{\ell_m}$.
- When $b = 0$, sample random exponents $\mathbf{s} \leftarrow \{0, 1\}^{\lceil 3 \log p \rceil}$ and send pd to $\mathcal{A}$, computed as follows

$$\text{pd} \leftarrow \text{Setup}(1^\lambda, \mathbf{s}, \mathbf{m}),$$

- When $b = 1$, sample pd , with random group elements from G and send them to $\mathcal{A}$.

Instantiation of Setup and Enc. The $\text{Setup}(1^\lambda, 1^D, |C|)$ procedure that selects public parameters $\text{pp} := (1^D, G, p, g, \text{Hash}, \delta, \gamma = \lceil |C|^{2/3} \cdot D^{1/3} \rceil, B_f)$ for all sub-protocols `HRead`, `HCompress`, `HGate` is the same as described in Section 5.2, except additionally set B_f as follows:

- Set B_f to be the circuit size computing masked truth tables as described in Equation 6.1. We have $B_f \leq O(|\text{PRF}|)$

We next describe how to sample gk and the $\text{Enc}(\text{gk}, \mathbf{x})$ procedure that produces encodings $\llbracket \mathbf{x} \rrbracket$ and public data pd .

¹²`EvalKey`, `EvalTag` can be modified in the natural way to take $\mathbf{s}' \in \{0, 1\}^{\lceil 3 \log p \rceil}$ during their setup, and output such intermediate encodings.

- First choose leveled secrets $\mathbf{s}^{\text{id}} \in [p]^2$ as gk for our encodings as in Equation 5.3, and additionally the following

$$\mathbf{s}^{(d, \text{TmpWire})} \leftarrow \{0, 1\}^{\lceil 3 \log p \rceil} \text{ for } d = 1, \dots, D.$$

- Next set up public data for the sub-protocols as in Equation 5.4, except sampling $\text{GE.pd}^{(d)}$ differently and $\text{HSS.pd}^{(d)}$ additionally as follows:

$$\text{GE.pd}^{(d)} \leftarrow \text{GE.Setup}(1^\lambda, \mathbf{s}^{\text{id}}, \mathbf{s}^{\text{id}''}), \text{HSS.pd}_{\text{key}}^{(d)} \leftarrow \text{HSS.Setup}(1^\lambda, \mathbf{s}^{\text{id}'}, (\text{key} \parallel \text{Bits}(\mathbf{s}^{\text{id}'}))) \quad (6.2)$$

where $\text{id} = (d, \text{InWire})$, $\text{id}'' = (d, \text{TmpType})$, $\text{id}' = (d, \text{OutWire})$, $\text{key} \leftarrow \{0, 1\}^\lambda$, for $d = 1, \dots, D$.

- Finally use $\mathbf{s}^{\text{id}=(0, \text{OutWire})}$ from the above to compute

$$\begin{aligned} \bar{\mathbf{x}}[i] &\leftarrow \mathbf{x}[i] \oplus \text{PRF}(\text{key}, i), \langle \mathbf{s}^{\text{id}} \cdot \bar{\mathbf{x}}[i] \rangle_G \leftarrow [p]^2, \\ \langle \mathbf{s}^{\text{id}} \cdot \mathbf{x}[i] \rangle_E &\leftarrow \langle \mathbf{s}^{\text{id}} \cdot \bar{\mathbf{x}}[i] \rangle_G + \mathbf{s}^{\text{id}} \cdot \bar{\mathbf{x}}[i] \text{ over } \mathbb{Z}_p, \\ \llbracket \mathbf{x}[i] \rrbracket_G &:= (\langle \mathbf{s}^{\text{id}} \cdot \bar{\mathbf{x}}[i] \rangle_G, \text{id}), \quad \llbracket \mathbf{x}[i] \rrbracket_E := (\langle \mathbf{s}^{\text{id}} \cdot \bar{\mathbf{x}}[i] \rangle_E, \text{id}, \bar{\mathbf{x}}[i]). \end{aligned} \quad (6.3)$$

Garbler Depth Complexity. Compared to the privacy-free construction in Section 5.2 only HGate changes.

HGate: The garbler first runs EvalKey, which has depth $O(\log \lambda)$. It then runs ExtEval to evaluate the function f_{op} as described in Equation 6.1, which has depth $\text{poly}(\lambda)$ in general. Hence HGate has depth $\text{poly}(\lambda)$.

We note that when replacing PRF with a local PRG with constant depth, then the depth of ExtEval and hence of HGate becomes $O(\log \lambda)$. The size of key for the local PRG needs to become larger as a trade-off.

Succinctness and Decomposability. The garbling material consists of the public data and re-key material of size $< (D + |C|^{2/3} \cdot D^{1/3}) \cdot \text{poly}(\lambda)$, similarly to the privacy-free construction in Section 5.2, and additionally of 1-bit per gate from the communication in HGate sub-protocol. Therefore, the overall garbling material have size $< |C| + (D + |C|^{2/3} \cdot D^{1/3}) \cdot \text{poly}(\lambda)$.

The input encoding and decryption key have the same sizes as the privacy-free construction in Section 5.2.

Removing the $1/\text{poly}(\lambda)$ Correctness and Privacy Error. Similarly to the privacy-free construction in Section 5.2, we obtain a garbling scheme with $1/\text{poly}$ correctness and privacy error. The two solutions described there (1) using Paillier groups instead of prime-order groups and (2) applying privacy amplification techniques from [ILL25c, BGI17] can also be applied here. We note that even when assuming a local PRG with constant depth, the privacy amplification from solution (2) will make the depth of HGate become $\text{poly}(\lambda)$.

Security. We sketch *selective* semantic security (Definition 3.2, with privacy error $2 \cdot \delta(\lambda) = 1/\text{poly}(\lambda)$) for the garbling scheme via a series of hybrid experiments, where Hyb_0 is the experiment $\text{ExptIND}_{\mathcal{A}}^{(0)}(\lambda)$, and Hyb_3 is the experiment $\text{ExptIND}_{\mathcal{A}}^{(1)}(\lambda)$.

- Hyb_0 : We briefly recall the experiment $\text{ExptIND}_{\mathcal{A}}^{(0)}(\lambda)$:
 - The challenger returns public parameters pp according to Setup, and receives a circuit C and input $\mathbf{x}_0$ from the adversary.
 - The challenger returns $\widehat{C}$, which consists of the public data pd as described in Equation 5.4 and Equation 6.2, rekey materials $\{\text{rk}^{(d)}\}$, and a bit $b^{(i)}$ for every intermediate wire i .
 - The challenger also returns input encodings $\llbracket \mathbf{x} \rrbracket_E$ as described in Equation 6.3.
- Hyb_1 : This hybrid computes the rekey materials $\{\text{rk}^{(d)}\}$ and the bits $b^{(i)}$ differently.
 - Using the input encodings $\llbracket \mathbf{x} \rrbracket_E$, derive evaluator's shares on all intermediate wires and hashes, and set $\{\text{rk}^{(d)}\}$ as the difference between evaluators shares of hashes, instead of the garbler's.

- Derive the bits $b^{(i)}$ as evaluator's share of $\langle \bar{z} \rangle_E$ on the wire i , subtracting the value $\bar{z}$, which can be derived from the correct wire value z on wire i and the pseudo-random mask r_z on it.

By the correctness of the garbling scheme, this hybrid is equivalent to the previous, except with $\delta(\lambda) = 1/\text{poly}(\lambda)$ failure probability.

$$|\Pr[\text{Hyb}_{1,\mathcal{A}}(\lambda) = 1] - \Pr[\text{Hyb}_{0,\mathcal{A}}(\lambda) = 1]| \leq \delta(\lambda).$$

- Hyb_2 : This hybrid computes the input encodings as random elements in $\mathbb{Z}_q^2$, instead of following Equation 6.3. This hybrid is equivalent to the previous, by the randomness of $\langle \mathbf{s}^{\text{id}} \cdot \bar{\mathbf{x}}[i] \rangle_G$. (See Equation 6.3).

$$\Pr[\text{Hyb}_{2,\mathcal{A}}(\lambda) = 1] = \Pr[\text{Hyb}_{1,\mathcal{A}}(\lambda) = 1].$$

We note that the secret $\mathbf{s}^{(0,\text{OutWire})}$ is only used for generating public data $\text{HW.pd}^{(0)}$, and not anywhere else in the experiment.

- This hybrid samples the public data $\{\text{HW.pd}^{(d)}\}$, $\{\text{HR.pd}^{(d)}\}$, $\{\text{GE.pd}^{(d)}\}$, $\{\text{HSS.pd}_{\text{key}}^{(d)}\}$ as random group elements in G , instead of a chain of ciphertexts encrypting the next secret s' under some secret s . By Lemma 5.21, 5.26, and 6.6, all of which assume the DDH assumption in prime-order groups, we have

$$|\Pr[\text{Hyb}_{3,\mathcal{A}}(\lambda) = 1] - \Pr[\text{Hyb}_{2,\mathcal{A}}(\lambda) = 1]| < \text{negl}(\lambda).$$

We note that the PRF key no longer used anywhere in the experiment, except for generating the wire masks for every intermediate wire.

- Hyb_4 : This hybrid samples the masked wire values as random bits, in particular, the $\bar{z}$ values used for generating bits $b^{(i)}$ as described in Hyb_1 are now random bits, independent of the input $\mathbf{x}_0$. By the security of PRF, we have

$$|\Pr[\text{Hyb}_{4,\mathcal{A}}(\lambda) = 1] - \Pr[\text{Hyb}_{3,\mathcal{A}}(\lambda) = 1]| < \text{negl}(\lambda).$$

- Hyb_5 : This is the experiment $\text{ExptIND}_{\mathcal{A}}^{(1)}(\lambda)$. By a symmetric arguments as above, we have

$$|\Pr[\text{Hyb}_{5,\mathcal{A}}(\lambda) = 1] - \Pr[\text{Hyb}_{4,\mathcal{A}}(\lambda) = 1]| < \text{negl}(\lambda) + \delta(\lambda).$$

By a standard hybrid argument, we have

$$|\Pr[\text{ExptIND}_{\mathcal{A}}^{(1)}(\lambda)] - \Pr[\text{ExptIND}_{\mathcal{A}}^{(0)}(\lambda)]| < \text{negl}(\lambda) + 2\delta(\lambda).$$

6.2 Instantiation Under Lattices

Similarly to the group instantiation, which combines the group constructions of HRead, HCompress from Section 5.2 and the group construction of HGate from [ILL25b], we can instead combine the lattice constructions of HRead, HCompress from Section 5.3 and the lattice construction of HGate from [ILL25b] to obtain a low-depth partially succinct garbling with privacy.

In more detail, the encoding $\llbracket x \rrbracket$ of wire values x now consists of additive shares over $\mathcal{R}_p$ and masked wire values in the clear:

$$\llbracket x \rrbracket = (\llbracket x \rrbracket_G, \llbracket x \rrbracket_E), \text{ where } \llbracket x \rrbracket_G = (\langle \mathbf{s}^{\text{id}} \cdot \bar{\mathbf{x}} \rangle_G, \text{id}), \text{ and } \llbracket x \rrbracket_E = (\langle \mathbf{s}^{\text{id}} \cdot \bar{\mathbf{x}} \rangle_E, \text{id}, \bar{\mathbf{x}}).$$

The Rekey steps, HRead, HCompress protocols are the same as in Section 5.3.

Instantiation of HGate. The HGate sub-protocol stays the same as (Figure 4), except using the lattice constructions of the ingredients, EvalKey, EvalTag from Section 5.3 and ExtEval which we describe below in Construction 6.7.

Construction 6.7 (Extended HSS Evaluation under lattice [BKS19]). The construction is with respect to the same public parameters $(\mathcal{R}, p, q, \mathcal{D}_{\text{err}}, \mathcal{D}_{\text{sk}})$ as Construction 5.28 under $B = 2$. It uses as ingredients: $\text{PRF} : \{0, 1\}^\lambda \times [2^\lambda] \rightarrow \mathcal{R}_p$. $\text{PRF}' : \{0, 1\}^\lambda \times [2^\lambda] \rightarrow \mathcal{R}_q$.

$\text{pd}_m \leftarrow \text{Setup}^{\text{pp}}(1^\lambda, s \in \mathcal{D}_{\text{sk}}, \mathbf{m} \in \{0, 1\}^{\ell_m})$: Compute public data pd_m w.r.t. $\mathbf{m}$ as follows:

$$\begin{aligned} \mathbf{a} &\leftarrow \mathcal{R}_q^{\ell_m}, r_1, r_2 \leftarrow \mathcal{D}_{\text{sk}}, \mathbf{e}, \mathbf{e}_1, \mathbf{e}_2, \mathbf{e}'_1, \mathbf{e}'_2 \leftarrow \mathcal{D}_{\text{err}}^{\ell_m}, \text{seed} \leftarrow \{0, 1\}^\lambda, \\ \mathbf{b} &:= s \cdot \mathbf{a} + \mathbf{e}, \quad \mathbf{c}_1 := r_1 \cdot \mathbf{a} + \mathbf{e}_1 + \mathbf{m} \cdot \Delta, \quad \mathbf{c}'_1 := r_1 \cdot \mathbf{b} + \mathbf{e}'_1, \\ \mathbf{c}_2 &:= r_2 \cdot \mathbf{a} + \mathbf{e}_2, \quad \mathbf{c}'_2 := r_2 \cdot \mathbf{b} + \mathbf{e}'_2 + \mathbf{m} \cdot \Delta. \\ \text{pd}_m &:= (\text{pp}, \text{seed}, \mathbf{c}_1, \mathbf{c}'_1, \mathbf{c}_2, \mathbf{c}'_2). \end{aligned}$$

$\langle \mathbf{z} \rangle_{G/E} \leftarrow \text{ExtEval}^{\text{pd}_m}(f : \{0, 1\}^{\ell_m} \rightarrow \{0, 1\}^{\ell_z \times \ell_x}, \{\langle s \cdot \mathbf{x}_i \rangle_{G/E}, \langle \mathbf{x}_i \rangle_{G/E}\}_{i \in [\ell_x]})$: Takes an NC1 Boolean circuit f and ℓ_x shares $\langle s \cdot \mathbf{x}_i \rangle_{G/E}, \langle \mathbf{x}_i \rangle_{G/E}$, w.r.t. the global secret $s \in \mathcal{D}_{\text{sk}}$ as input. Outputs an additive share $\langle \mathbf{z} \rangle_{G/E}$ where $\mathbf{z}$ should equal

$$\mathbf{z} = f(\mathbf{m}) \cdot \mathbf{x} \text{ over } \mathbb{Z}.$$

Lemma 6.8 (Correctness of Extended HSS Evaluation [BKS19]). *There exists a negligible function $\text{negl}(\lambda)$ such that for every $\lambda \in \mathbb{N}$, public parameters pp as specified, secret elements $s \in \mathcal{D}_{\text{sk}}$, NC1 Boolean circuit $f : \{0, 1\}^{\ell_m} \rightarrow \{0, 1\}^{\ell_z \times \ell_x}$ strings $\mathbf{m} \in \{0, 1\}^{\ell_m}$, $\mathbf{x} \in \{0, 1\}^{\ell_x}$, and additive shares $\langle s \cdot \mathbf{x} \rangle_G, \langle s \cdot \mathbf{x} \rangle_E, \langle \mathbf{x} \rangle_G, \langle \mathbf{x} \rangle_E$, following holds:*

$$\Pr \left[\begin{array}{l} \mathbf{w}_E = \mathbf{w}_G + f(\mathbf{m})^\top \mathbf{x} \\ (\text{over } \mathcal{R}_p) \end{array} \middle| \begin{array}{l} \text{pd}_m \leftarrow \text{Setup}(1^\lambda, s, \mathbf{m}) \\ \mathbf{w}_G = \text{ExtEval}(f, \{\langle s \cdot \mathbf{x}[i] \rangle_G, \langle \mathbf{x}[i] \rangle_G\}), \\ \mathbf{w}_E = \text{ExtEval}(f, \{\langle s \cdot \mathbf{x}[i] \rangle_E, \langle \mathbf{x}[i] \rangle_E\}) \end{array} \right] > 1 - \text{negl}(\lambda).$$

Remark 6.9. The mismatch between our encoding format, which requires a secret vector $\mathbf{s} \in \mathcal{D}_{\text{sk}}^2$, and the extended HSS evaluation interface, which requires a secret $s \in \mathcal{D}_{\text{sk}}$ can be resolved analogously to the group instantiation (Section 6.1) by introducing an intermediate encoding format, indicated by TmpWire , using a single element as the secret as required by the HSS evaluation interface.

Similarly to the group instantiation (Section 6.1), we note the following straightforward lemma and omit its proof.

Lemma 6.10 (Security of HSS Evaluation). *Under the RLWE assumption, every efficient adversary $\mathcal{A}$ has negligible advantage in distinguishing the experiments $\text{Exp}_{\mathcal{A}}^b(1^\lambda)$, for $b \in \{0, 1\}$, defined analogously to Lemma 6.6.*

Instantiation of Setup and Enc. The $\text{Setup}(1^\lambda, 1^D, |C|)$ and Enc steps are the same as Section 5.3, with analogous modifications to group construction in Section 6.1.

Garbler Depth Complexity. Similarly to the group construction in Section 6.1, the depth of HGate is $\text{poly}(\lambda)$ in general. When replacing PRF with a local PRG with constant depth, the depth of HGate becomes $O(\log \lambda)$.

Succinctness and Decomposability. Similarly to the group construction in Section 6.1, the overall garbling material have size $< |C| + D \cdot \text{poly}(\lambda)$.

The input encoding and decryption key have the same sizes as the privacy-free construction in Section 5.3.

Security. The selective semantics security can be shown analogously to that of the group instantiation, based on the standard RLWE assumption (Definition 5.12).

7 Reusable Succinct Privacy-Free Garbling

In this section, we prove the following statement:

Theorem 7.1 (Reusable Succinct Privacy-Free Garbling). *Assuming decomposed LWE [Assumption 7.5](#) (parameterized as [Construction 7.12](#)), there exists a reusable succinct privacy-free garbling scheme Π_{garble} satisfying the following:*

- For a circuit $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$ of depth d , the public data pd and garbling material $|\hat{C}|, |\hat{x}|, |\text{gk}|$ are of size $\text{poly}(\lambda, d)$ (independent of circuit size $|C|$). The garbled input $\hat{x}$ additionally on ℓ and is decomposable.
- The garbling algorithms (Setup, GarbC, GarbInp) are computable in NC^1 .
- Π_{garble} satisfies selective authenticity.

7.1 Preliminaries

Notation. For a matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times m}$, we use $\|\mathbf{M}\|$ to denote the maximum absolute value of its entries. We use $\mathbf{A} \otimes \mathbf{B}$ to denote the Kronecker product between $\mathbf{A}$ and $\mathbf{B}$. We will use $\mathbf{I}$ to denote the identity matrix. We reserve the symbol $\mathbf{G}$ to denote the $n \times m$ gadget matrix $\mathbf{I}_n \otimes [1, 2, 4, \dots, 2^{\lceil \log q \rceil} - 1, 0, \dots, 0]$. We let $\mathbf{G}^{-1}(\cdot) : \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q^m$ denote entry-wise bit decomposition (padded with zeroes when necessary) where $\mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{A}) = \mathbf{A}$. We use $\mathcal{D}_\sigma$ to refer to the uniform distribution over range $[-\sigma, \sigma] \subset \mathbb{Z}^{13}$. We use $\mathcal{D} \approx_s \mathcal{D}'$ to denote distributions with negligible (in security parameter λ) statistical distance. We use $a(\lambda) \gg b(\lambda)$ to denote a superpolynomial gap between $a(\lambda)$ and $b(\lambda)$ - i.e. $a(\lambda) \geq b(\lambda) \cdot \lambda^{\omega(1)}$. We use $\lfloor \cdot \rfloor_p : \mathbb{Z}_q \rightarrow \mathbb{Z}_p$ to denote the rounding operation, where we interpret elements of $\mathbb{Z}_q$ as elements of $\mathbb{Z}$ and multiply by (q/p) before rounding to the nearest integer. We will use $\lfloor \cdot \rfloor_p$.

Lemma 7.2 (Smudging Lemma (Adapted From [\[AJL⁺12\]](#))). *Let λ be a security parameter and $B = B(\lambda), B' = B'(\lambda)$ be functions where $B \ll B'$. Then the distributions*

$$\mathcal{D}_{B+B'} \approx_s \mathcal{D}_{B'}$$

are statistically close.

Lemma 7.3 (Spooky Rounding (Adapted From [\[DHRW16\]](#))). *For moduli $p, q \in \mathbb{N}$ along with target value $v \in \mathbb{Z}_q$ and error bound, where $p \cdot \sigma \ll q$, consider an additive sharing $v_1, v_2 \xleftarrow{\mathbb{R}} \mathbb{Z}_q$ subject to $v_1 + v_2 = v \pmod q$ then $\lfloor v_1 \rfloor_p + \lfloor v_2 \rfloor_p = \lfloor v \rfloor_p$.*

Definition 7.4 (Leftover Hash Lemma (Adapted from [\[ABB10\]](#))). *Let λ be a security parameter and n, m, q be lattice parameters where $m \geq 2n \log q$, $n \geq \lambda$ and prime $q > 2$. Then for all fixed vectors $\mathbf{e} \in \mathbb{Z}_q^m$ and $k \in \text{poly}(\lambda)$, the following distributions are statistically close:*

$$(\mathbf{A}, \mathbf{AR}, \mathbf{e}^\top \mathbf{R}) \approx_s (\mathbf{A}, \mathbf{U}, \mathbf{e}^\top \mathbf{R}),$$

where $\mathbf{A} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times m}, \mathbf{R} \xleftarrow{\mathbb{R}} \{0, 1\}^{m \times k}, \mathbf{U} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k}$.

Assumption 7.5 (Decomposed LWE [\[AMR25\]](#)). *Let λ be a security parameter and $n = n(\lambda), m = m(\lambda), q = q(\lambda), \sigma = \sigma(\lambda)$, and $\chi = \chi(\lambda)$ be lattice parameters. We say that the decomposed LWE assumption $\text{dLWE}_{n,m,q,t,\chi,\sigma}$ assumption holds if for all PPT adversaries $\mathcal{A}$:*

$$\left| \Pr \left[\mathcal{A} \left(\{ \mathbf{A}_i, \mathbf{B}_j, \mathbf{s}^\top \mathbf{A}_i \mathbf{B}_j + \mathbf{s}^\top \mathbf{G} \cdot \delta_{i,j} + \mathbf{e}_{i,j}^\top \}_{i,j \in [t]} \right) = 1 \right] - \Pr \left[\mathcal{A} \left(\{ \mathbf{A}_i, \mathbf{B}_j, \mathbf{u}_{i,j} \}_{i,j \in [t]} \right) = 1 \right] \right| = \text{negl}(\lambda)$$

where $\mathbf{A}_i \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k}, \mathbf{B}_j \xleftarrow{\mathbb{R}} \mathcal{D}_{\chi}^{k \times m}, \mathbf{s} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n, \mathbf{e}_{i,j} \leftarrow \mathcal{D}_{\sigma}^m, \mathbf{u}_{i,j} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^m$ for $i, j \in [t]$.

Definition 7.6 (Lattice Gadget Trapdoors [\[Ajt96, GPV08, MP12\]](#)). *Let λ be a security parameter and n, m, q be lattice parameters with $m \geq 3n \log q$ and $n \geq \lambda$. There exists efficient randomized algorithms (TrapGen, SamplePre) such that:*

¹³In general, we will use uniform noise in our constructions, which can clearly be sampled in NC^1

- $\text{TrapGen}(1^n, 1^m, q) \rightarrow (\mathbf{A}, \text{td}_{\mathbf{A}})$: On input matrix dimensions n, m and modulus q , TrapGen outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor $\text{td}_{\mathbf{A}} \in \mathbb{Z}_q^{m \times m}$.
- $\text{SamplePre}(\mathbf{A}, \text{td}, \mathbf{T}, \sigma) \rightarrow \mathbf{P}$: On input matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, trapdoor matrix $\text{td} \in \mathbb{Z}_q^{m \times m}$, target matrix $\mathbf{T} \in \mathbb{Z}_q^{n \times \ell}$, and width parameter $\sigma > 0$, the SamplePre outputs a preimage $\mathbf{P} \in \mathbb{Z}_q^{m \times n'}$.

A gadget trapdoor satisfies the following properties:

- **Trapgen Uniformity**: In the output $(\mathbf{A}, \text{td}_{\mathbf{A}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\mathbf{A}$ is statistically close to uniform $\mathbf{A} \stackrel{\mathcal{R}}{\leftarrow} \mathbb{Z}_q^{n \times m}$.
- **Correctness**: For all $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, all gadget trapdoors $\text{td} \in \mathbb{Z}_q^{m \times m}$, and all $\mathbf{T} \in \mathbb{Z}_q^{n \times \ell}$, and all $\mathbf{P} \leftarrow \text{SamplePre}(\mathbf{A}, \text{td}, \mathbf{T}, \sigma)$, $\mathbf{A} \cdot \mathbf{P} = \mathbf{T}$. Furthermore, in $(\mathbf{A}, \text{td}_{\mathbf{A}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\text{td}_{\mathbf{A}}$ is a gadget trapdoor $\mathbf{A} \cdot \text{td} = \mathbf{G}$ with $\|\text{td}\| = 1$.
- **Preimage Distribution**: There exists $B = B(\lambda)$, such that for all $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, all gadget trapdoors $\text{td} \in \mathbb{Z}_q^{m \times m}$ such that $\mathbf{A} \cdot \text{td} = \mathbf{G}$, and all $\sigma \geq m \cdot \|\text{td}\| \cdot \log n$, the distributions of

$$(\text{SamplePre}(\mathbf{A}, \text{td}, \mathbf{T}, \sigma), \mathbf{T}) \approx_s (\mathbf{P}, \mathbf{A} \cdot \mathbf{P})$$

where $\mathbf{T} \stackrel{\mathcal{R}}{\leftarrow} \mathbb{Z}_q^{n \times \ell}$ and $\mathbf{P} \stackrel{\mathcal{R}}{\leftarrow} \mathcal{D}_{B, \sigma}^{m \times \ell}$ are statistically close.

Remark 7.7. We note the above notion of gadget trapdoors is slightly weaker than the usual definition, where the preimage distribution is explicitly required to be a specific coset of the discrete gaussian distribution with width σ . Looking ahead to [Section 7.3.1](#), this will be in the interest of realising a simple NC^1 instantiation of preimage sampling.

Definition 7.8 (Lattice-based Homomorphic Evaluation [[GSW13](#), [BGG⁺14](#)]). Let λ be a security parameter and n, m, q, σ be lattice parameters for $m \geq 2n \log q$. Let $C_{\ell, d}$ be a family of depth d boolean circuits $C : \{0, 1\}^{\ell} \rightarrow \{0, 1\}$. There exists efficient (deterministic) algorithms:

- $\text{EvalF}(\mathbf{B}, C) \rightarrow \mathbf{B}_C$: On input matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times \ell \cdot m}$ and circuit $C \in C_{\ell, d}$, the input-independent evaluation EvalF computes matrix $\mathbf{B}_C$.
- $\text{EvalFX}(\mathbf{B}, C, x) \rightarrow \mathbf{H}_{\mathbf{B}, C, x}$: On input matrix $\mathbf{B}$, circuit $C \in C_{\ell, d}$, and input $x \in \{0, 1\}^{\ell}$, the input-dependent evaluation EvalFX computes low norm matrix $\mathbf{H}_{\mathbf{B}, C, x}$.

Such that the following properties hold:

- **Low Norm**: The matrix $\mathbf{H}_{\mathbf{B}, C, x}$ has low norm $\|\mathbf{H}_{\mathbf{B}, C, x}\| \leq m^{O(d)}$.
- **Homomorphism**: For any matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times \ell \cdot m}$, circuit $C \in C_{\ell, d}$ and input $x \in \{0, 1\}^{\ell}$, the equation

$$(\mathbf{B} - x^{\top} \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbf{B}, C, x} = \mathbf{B}_C - C(x) \cdot \mathbf{G}$$

holds.

Definition 7.9 (Key-Homomorphic RAM Encodings [[AMR25](#)]). Let λ be a security parameter and n, m, q, σ be lattice parameters for $m \geq 2n \log q$. Let $\ell \in \text{poly}(\lambda)$. There exists efficient (and deterministic apart from RSetup) algorithms:

- $\text{RSetup}(1^{\lambda}) \rightarrow (\text{pp}, \mathbf{Z})$: On input security parameter λ , the Setup algorithm outputs public parameter pp , and matrix $\mathbf{Z} \in \mathbb{Z}_q^{n \times \ell}$.
- $\text{RHash}(\text{pp}, x) \rightarrow \mathbf{d}$: On input public parameters pp and input $x \in \{0, 1\}^{\ell}$, the hash algorithm deterministically computes digest vector $\mathbf{d} \in \mathbb{Z}_q^k$.
- $\text{RRead}(\text{pp}, \mathbf{B}, i) \rightarrow \mathbf{B}_{(i)}^{(r)}$: On input public parameters pp , matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times k \cdot m}$, and index $i \in [\ell]$, the input-independent read function computes matrix $\mathbf{B}_{(i)}^{(r)}$.

- $\text{RXRead}(\text{pp}, \mathbf{B}, i, x) \rightarrow \mathbf{H}_{\mathbf{B}, \langle i \rangle, x}^{(r)}$: On input public parameters pp , matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times k \cdot m}$, index $i \in [\ell]$, and bitstring $x \in \{0, 1\}^\ell$, the input-dependent read function computes matrix $\mathbf{H}_{\mathbf{B}, \langle i \rangle, x}^{(r)}$.
- $\text{RWrite}(\text{pp}, \mathbf{B}, S, \mathbf{B}_S) \rightarrow \mathbf{B}_S^{(w)}$: On input public parameters pp , and writing indices $S \subseteq [\ell]$, matrix $\mathbf{B}_S \in \mathbb{Z}_q^{n \times m \cdot |S|}$, the write function computes a matrix $\mathbf{B}_S^{(w)}$.
- $\text{RXWrite}(\text{pp}, \mathbf{B}, S, \mathbf{B}_S, x^{(w)}, x) \rightarrow \mathbf{H}_{\mathbf{B}, S, x}^{(w)}$: On input public parameters pp , and writing indices $S \subseteq [\ell]$, written bits $x^{(w)} \in \{0, 1\}^{|S|}$ and matrix $\mathbf{B}_S \in \mathbb{Z}_q^{n \times m \cdot |S|}$, along with input $x \in \{0, 1\}^\ell$ the write function computes a matrix $\mathbf{H}_{\mathbf{B}, S, x}^{(w)}$.

Such that the following properties hold:

- **Succinctness:** The lengths of public parameters pp , matrix $\mathbf{Z}$ and digest $\mathbf{d}$ are independent of the input length $|x| = \ell$.
- **Low Norm:** The matrices $\mathbf{H}_{\mathbf{B}, \langle i \rangle, x}^{(r)}, \mathbf{H}_{\mathbf{B}, S, x}^{(w)}$ has low norm $\|\mathbf{H}_{\mathbf{B}, \langle i \rangle, x}^{(r)}, \mathbf{H}_{\mathbf{B}, S, x}^{(w)}\| \leq \text{poly}(\lambda)$.
- **Read Homomorphism:** For any matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times k \cdot m}$, indices $i \in [\ell]$ and input $x \in \{0, 1\}^\ell$, the equation

$$[\mathbf{B} - \text{RHash}(x) \otimes \mathbf{G} \mid \mathbf{Z}] \cdot \mathbf{H}_{\mathbf{B}, \langle i \rangle, x} = \mathbf{B}_{\langle i \rangle} - x_i \cdot \mathbf{G}$$

holds.

- **Write Homomorphism:** For any set $S = \{i_1, \dots, i_{|S|}\} \subseteq [\ell]$, matrices $\mathbf{B} \in \mathbb{Z}_q^{n \times k \cdot m}, \mathbf{B}_S \in \mathbb{Z}_q^{n \times m \cdot |S|}$, strings $x^{(w)} \in \{0, 1\}^{|S|}, x \in \{0, 1\}^\ell$,

$$[\mathbf{B} - \text{RHash}(x) \otimes \mathbf{G} \mid \mathbf{B}_S - x^{(w)} \otimes \mathbf{G} \mid \mathbf{Z}] \cdot \mathbf{H}_{\mathbf{B}, S, x}^{(w)} = \mathbf{B}_S^{(w)} - \text{RHash}(x') \otimes \mathbf{G}$$

where $x'_i = x_j^{(w)}$ for any $i_j \in S$ and $x'_i = x_i$ otherwise.

- **LHL Compatible:** For all fixed vectors $\mathbf{e} \in \mathbb{Z}_q^t$ and $k \in \text{poly}(\lambda)$, the distributions

$$(\text{pp}, \mathbf{Z}, \mathbf{Z} \cdot \mathbf{R}, \mathbf{e}^\top \mathbf{R}) \approx_s (\text{pp}, \mathbf{Z}, \mathbf{U}, \mathbf{e}^\top \mathbf{R})$$

are statistically close, where $(\text{pp}, \mathbf{Z}) \leftarrow \text{RSetup}(1^\lambda), \mathbf{R} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m}$, and $\mathbf{U} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times t}$.

- **LWE indistinguishability:** Given

$$\{\text{pp}, \mathbf{Z}, \mathbf{s}^\top \mathbf{Z} + \mathbf{e}^\top\} \approx_c \{\text{pp}, \mathbf{Z}, \mathbf{t}^\top\}$$

where $(\text{pp}, \mathbf{Z}) \leftarrow \text{RSetup}(1^\lambda), \mathbf{s} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n, \mathbf{e}^\top \xleftarrow{\mathbb{R}} \mathcal{D}_\sigma^m$ and $\mathbf{t} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^m$.

Remark 7.10. We note the above can also be instantiated with the succinct LWE assumption of [Wee24] (essentially corresponding to the matrix commitment abstraction of [Wee25]). This translates translates to a concretely smaller instantiation (where $t \approx m$ rather than $t \approx 4m^5$ using decomposed LWE), but requires a private-coin RSetup in contrast to the public-coin RSetup algorithm from decomposed LWE.

Remark 7.11. We briefly discuss the properties here that were not explicitly outlined in prior work. The LHL Compatibility follows from a straightforward inspection of the structure of the $\mathbf{Z}$ matrix. In the succinct LWE instantiation, this is a uniformly random matrix, and so corresponds exactly to the classic leftover hash lemma. In the decomposed LWE instantiation, this matrix takes the form of $\mathbf{Z} = \mathbf{A} \cdot (\mathbf{I}_{m^2} \otimes \mathbf{B}) + \text{vec}(\mathbf{I}_{m^2}) \otimes \mathbf{G}$ where $\mathbf{A}$ is a random $n \times m$ matrix. This is sufficient for to apply LHL. The LWE indistinguishability property essentially corresponds exactly to the decomposed (resp. succinct) LWE assumptions.

7.2 Construction

We are now ready to present our construction of reusable privacy-free garbling.

Construction 7.12 (Succinct Reusable Privacy-Free Garbling). Let λ be the security parameter and d be a bound on circuit depth. Our scheme relies on the following lattice ingredients:

- A lattice gadget trapdoor sampler (TrapGen, SamplePre) per [Definition 7.6](#).
- Lattice-based homomorphic evaluation (EvalF, EvalFX) per [Definition 7.8](#).
- Key-homomorphic RAM encoding (RSetup, RHash, RRead, RXRead, RWrite, RXWrite) per [Definition 7.9](#).

We parameterize our scheme as follows:¹⁴

$$n \geq \lambda \quad , \quad m = 3n \log q \quad , \quad B = \lambda^{\log(\lambda)} \quad , \quad \sigma_{\text{large}} = \sigma \cdot B^{d+3}$$

$$\Delta = B^{d+4} \cdot \sigma \quad , \quad p = \sigma \cdot B \quad , \quad q = \Delta \cdot p,$$

where the dLWE $_{n,m,q,2m^2\lambda,\chi,\sigma}$ is hard with respect to security parameter λ , which in turn fix parameters t, k of the RAM encoding. We can now present our scheme as follows:

- Setup($1^\lambda, 1^d$) $\rightarrow$ pd: On input security parameter λ and bound on circuit depth d , the setup algorithm samples the following:

- Sample $(\text{pp}, Z) \leftarrow \text{RSetup}(1^\lambda)$.
- Sample random matrix $\mathbf{B} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}$.

- Enc(x): For an input value $x \in \{0, 1\}^\ell$, the garbler samples the following:

- Sample input matrix $\mathbf{B}_{\text{inp}} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times m \cdot \ell}$.
- Sample secret $\mathbf{s} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n$ and trapdoored matrix $(\mathbf{A}, \text{td}_\mathbf{A}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$.
- To compute error vectors, sample and set

$$\mathbf{e}_0 \xleftarrow{\mathbb{R}} \mathcal{D}_\sigma^t, \mathbf{R}_\mathbf{A} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m}, \mathbf{R}_\mathbf{B} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m \cdot k}, \mathbf{R}_{\text{inp}} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m \cdot \ell}$$

$$\mathbf{e}_1^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_\mathbf{A}, \mathbf{e}_2^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_\mathbf{B}, \mathbf{e}_3^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_{\text{inp}}.$$

- Set garbled input as:

$$\mathbf{C}_Z = \mathbf{s}^\top \cdot \mathbf{Z} + \mathbf{e}_0^\top, \quad \mathbf{C}_\mathbf{A} = \mathbf{s}^\top \cdot \mathbf{A} + \mathbf{e}_1^\top$$

$$\mathbf{C}_\mathbf{B} = \mathbf{s}^\top \cdot (\mathbf{B} - \text{RHash}(0) \otimes \mathbf{G}) + \mathbf{e}_2^\top \text{¹⁵}, \quad \mathbf{C}_x = \mathbf{s}^\top \cdot (\mathbf{B}_{\text{inp}} - x^\top \otimes \mathbf{G}) + \mathbf{e}_3^\top.$$

- We treat matrix $\mathbf{B}_{\text{inp}}$ as $\llbracket x \rrbracket_G$ and $(x, \mathbf{B}_{\text{inp}}, \mathbf{C}_x)$ as the $\llbracket x \rrbracket_E$, which is sent to the evaluator. The other components $(\mathbf{C}_Z, \mathbf{C}_\mathbf{A}, \mathbf{C}_\mathbf{B})$ are will be used as auxiliary information as part of the garbled input.

- HGate(op, $\llbracket x \rrbracket, \llbracket y \rrbracket$): The gate evaluation correspond exactly to lattice-based homomorphic evaluation of [Definition 7.8](#).

- HGate $_G(\text{op}, \mathbf{B}_x, \mathbf{B}_y) \rightarrow \mathbf{B}_{\text{op}(x,y)}$: The garbler's gate-evaluation algorithm corresponds to the input-independent homomorphic evaluation $\llbracket \text{op}(x, y) \rrbracket_G = \mathbf{B}_{\text{op}(x,y)} \leftarrow \text{EvalF}([\mathbf{B}_x \mid \mathbf{B}_y], \text{op})$.
- HGate $_E(\text{op}, (x, \mathbf{B}_x, \mathbf{C}_x), (y, \mathbf{B}_y, \mathbf{C}_y)) \rightarrow \mathbf{C}_{\text{op}(x,y)}$: The evaluators gate-evaluation algorithm corresponds to the input-dependent homomorphic evaluation
 - * Compute input-independent evaluation $\mathbf{B}_{\text{op}(x,y)} \leftarrow \text{EvalF}([\mathbf{B}_x \mid \mathbf{B}_y], \text{op})$.

¹⁴We choose these parameters for simplicity and are not optimized for poly/polylog factors in λ .

¹⁵In the [\[AMR25, Wee25\]](#) instantiations of RAM encodings, $\text{RHash}(0) = 0$, so this is simply $\mathbf{s}^\top \cdot \mathbf{B} + \mathbf{e}_2^\top$

- * Compute input-dependent homomorphic evaluation $H_{\text{op}(x,y)} = \text{EvalFX}([B_x \mid B_y], \text{op}, (x \parallel y))$.
- * Set encoding as $\llbracket \text{op}(x, y) \rrbracket_E = (\text{op}(x, y), B_{\text{op}(x,y)}, [C_x \mid C_y] \cdot H_{\text{op}(x,y)})$.
- $\text{Hash}(\hat{C}, x) \rightarrow h_x$: This corresponds exactly to RHash function of key-homomorphic RAM encodings of [Definition 7.9](#). In the reusable setting, the garbler will not need his share of $\llbracket x \rrbracket_G$ to compute HRead_G , HCompress_G , and so we omit these as inputs below.
 - $\text{HRead}_G(B_{h_x}, i)$: The garbler's read algorithm corresponds to input-independent read $\llbracket x_i \rrbracket_G = B_{x_i} \leftarrow \text{RRead}(\text{pp}, B_{h_x}, i)$.
 - $\text{HRead}_E((h_x, B_{h_x}, C_{h_x}), i, (x, B_x, C_x))$: The evaluator's read algorithm corresponds to input-dependent read
 - * Compute input-independent read $B_{x_i} \leftarrow \text{RRead}(\text{pp}, B_{h_x}, i)$.
 - * Compute input-dependent $H_{x_i} = \text{RXRead}(\text{pp}, B_{h_x}, i, x)$.
 - * Set encoding $\llbracket x_i \rrbracket_E = (x_i, B_{x_i}, [C_{h_x} \mid C_Z] \cdot H_{x_i})$.
 - $\text{HCompress}_G(B_x)$: The garbler's compress algorithm corresponds to input-independent write $\llbracket h_x \rrbracket_G = B_{h_x} \leftarrow \text{RWrite}(\text{pp}, B, \llbracket x \rrbracket, B_x)$.
 - $\text{HCompress}_E(x, B_x, C_x)$: The evaluator's read algorithm corresponds to input-dependent write
 - * Compute input-independent write

$$B_{h_x} \leftarrow \text{RWrite}(\text{pp}, B, \llbracket x \rrbracket, B_x).$$
 - * Compute input-dependent $H_{h_x} = \text{RXWrite}(\text{pp}, B, \llbracket x \rrbracket, B_x, 0, x)$.
 - * Set encoding $\llbracket h_x \rrbracket_E = (h_x, B_{h_x}, [C_B \mid C_x \mid C_Z] \cdot H_{h_x})$.

- $\text{Rekey}(\llbracket x \rrbracket) \rightarrow \llbracket x \rrbracket'$:
 - $\text{RekeySamp}(\text{gk}) \rightarrow \llbracket x \rrbracket'_G$: Sample fresh random matrix $B' \xleftarrow{R} \mathbb{Z}_q^{n \times k \cdot m}$, which will be set as $\llbracket x \rrbracket'_G$.
 - $\text{RekeyGen}(\llbracket x \rrbracket_G, \llbracket x \rrbracket'_G) \rightarrow \text{rk}$: Parsing $\llbracket x \rrbracket_G = B_{h_x}$ and $\llbracket x \rrbracket'_G = B'$, sample

$$\text{rk} \leftarrow \text{SamplePre}([A \mid Z], \begin{bmatrix} \text{td}_A \\ 0 \end{bmatrix}, B' - B_{h_x}, \sigma),$$

which is sent to the evaluator.

- $\text{RekeyEval}(\llbracket x \rrbracket_E, \text{rk}) \rightarrow \llbracket x \rrbracket'_E$: Evaluator parses their share $\llbracket x \rrbracket_E = (h_x, B_{h_x}, C_{h_x})$ computes their new share as $(h_x, B', C_{h_x} + [C_A \mid C_Z] \cdot \text{rk})$ ¹⁶.
- **Output Phase:** The garbler samples $\mathbf{u} \xleftarrow{R} \mathbb{Z}_q^n$ and $\mathbf{e}_{\text{large}} \leftarrow \mathcal{D}_{\sigma_{\text{large}}}$ and rounding shifts $\delta \xleftarrow{R} \mathbb{Z}_q$.
 - The garbler parses their share of the output wire $\llbracket y_{\text{out}} \rrbracket_G = B_{\text{out}}$. They output $\mathbf{u}, \delta$, decoding key as $\text{dk} = \lfloor (\mathbf{s}^\top \cdot B_{\text{out}} + \mathbf{e}_{\text{large}}) \cdot \mathbf{G}^{-1}(\mathbf{u}) + \delta \rfloor_p$ and authentication key $\text{auk} = \lfloor \mathbf{s}^\top [B_{\text{out}} \mid B_{\text{out}} \cdot \mathbf{G}^{-1}(\mathbf{u})] + \delta \cdot \mathbf{1}_{m+1}^\top \rfloor_p, \lfloor \mathbf{s}^\top [G \mid \mathbf{u}] \rfloor_p$.
 - The evaluator parses their share of the output wire $\llbracket y_{\text{out}} \rrbracket_E = (y_{\text{out}}, B_{\text{out}}, C_{\text{out}})$. Compute output label as $\lfloor [C_{\text{out}} \mid C_{\text{out}} \cdot \mathbf{G}^{-1}(\mathbf{u})] + \delta \cdot \mathbf{1}_{m+1} \rfloor_p$

Remark 7.13 (Succinct Homomorphic Encodings). For notational convenience, when arguing correctness and security of our scheme, we will use $(\text{SEvalF}, \text{SEvalFX})$ to denote succinct homomorphic evaluation performed directly on the RAM encoding(s) of [Definition 7.9](#). This corresponds directly to composing read, homomorphic evaluation of [Definition 7.8](#), and write in both the input-dependent and input-independent settings, and the HRead , HGate , HCompress operations when used in the above garbling construction [Construction 7.12](#).

¹⁶Note that rk allows the evaluator to efficiently compute B' with A, Z, B_{h_x} , which are all known.

- $\text{SEvalF}(\text{pp}, \mathbf{B}, f) \rightarrow \mathbf{B}_f$: On input matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times k \cdot m \cdot t}$ where $\mathbf{B} = [\mathbf{B}_1 \mid \dots \mid \mathbf{B}_t]$ and function $f : \{0, 1\}^{\ell_1} \times \dots \times \{0, 1\}^{\ell_t} \rightarrow \{0, 1\}^{\ell'}$ in $\mathcal{F}_d$, the input-independent evaluation EvalF computes matrix $\mathbf{B}_f \in \mathbb{Z}_q^{n \times k \cdot m}$.
- $\text{SEvalFX}(\text{pp}, \mathbf{B}, f, x) \rightarrow \mathbf{H}_{\mathbf{B}, f, x}$: On input matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times k \cdot m \cdot t}$ where $\mathbf{B} = [\mathbf{B}_1 \mid \dots \mid \mathbf{B}_t]$, function $f \in \{0, 1\}^{\ell_1} \times \dots \times \{0, 1\}^{\ell_t} \rightarrow \{0, 1\}^{\ell'}$ in $\mathcal{F}_d$, and inputs $x = (x_1, \dots, x_t)$ for $x_i \in \{0, 1\}^{\ell_i}$ for $i \in [t]$, the input-dependent evaluation EvalFX computes low norm matrix $\mathbf{H}_{\mathbf{B}, f, x}$.

We note that that this composition inherits the low norm properties of $\mathbf{H}_f, \mathbf{K}_f$ by simply multiplying together the $\mathbf{H}$ matrices as well as respects the homomorphism:

$$[\mathbf{B}_1 - \text{RHash}(x_1) \otimes \mathbf{G} \mid \dots \mid \mathbf{B}_t - \text{RHash}(x_t) \mid \mathbf{Z}] \cdot \mathbf{H}_{\mathbf{B}, f, x} = \mathbf{B}_f - \text{RHash}(f(x_1, \dots, x_t)) \otimes \mathbf{G}.$$

7.3 Garbler Depth Complexity

We follow [Section 4.1](#) in instantiating a low-depth version of the garbler's computation. We note it is easy to see that bit decomposition, as well as elementary (modular) arithmetic [[CDL01](#)] and matrix operations (i.e. addition, multiplication, tensor product, bit decomposition) over $\mathbb{Z}_q$ for q of poly-bounded length can all be implemented in NC1. This is sufficient to implement the majority of our garbling scheme, including [[BGG⁺14](#)] lattice homomorphisms. We discuss the remaining ingredients below.

7.3.1 Log-Depth Preimage Sampling

We show a simple gadget trapdoor which is easy to see is in NC¹ and suffices for our applications. We can take constant B to be any function $\lambda^{\omega(1)}$.

- $\text{TrapGen}(1^n, 1^m, q) \rightarrow (\mathbf{A}, \text{td}_{\mathbf{A}})$: On input matrix dimensions n, m with $m \geq 3n \log q$ and modulus q ,
 - Sample matrix $\mathbf{B} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times 2n \log q}$
 - Sample matrix $\mathbf{R} \xleftarrow{\mathbb{R}} \{0, 1\}^{(2n \log q) \times (m - 2n \log q)}$
 - Output $\mathbf{A} = [\mathbf{B} \mid \mathbf{B} \cdot \mathbf{R} - \mathbf{G}]$ and $\text{td}_{\mathbf{A}} = \begin{bmatrix} \mathbf{R} \\ -\mathbf{I} \end{bmatrix}$.
- $\text{SamplePre}(\mathbf{A}, \text{td}, \mathbf{T}, \sigma)$: On input matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, trapdoor $\text{td} \in \mathbb{Z}_q^{m \times n \log q}$, target matrix $\mathbf{T} \in \mathbb{Z}_q^{n \times k}$ and parameter σ ,
 - Sample $\mathbf{R} \xleftarrow{\mathbb{R}} [\sigma \cdot B]^{m \times k}$ and set $\mathbf{V} = \mathbf{A} \cdot \mathbf{R}$.
 - Output $\text{td} \cdot \mathbf{G}^{-1}(\mathbf{T} - \mathbf{V}) + \mathbf{R}$.

To see this construction meets the definition of gadget trapdoor [Definition 7.6](#), we can first observe that correctness follows from the relation

$$\begin{aligned} \mathbf{A} \cdot \mathbf{P} &= \mathbf{A} \cdot (\text{td} \cdot \mathbf{G}^{-1}(\mathbf{T} - \mathbf{V}) + \mathbf{R}) \\ &= (\mathbf{A} \cdot \text{td}) \cdot \mathbf{G}^{-1}(\mathbf{T} - \mathbf{V}) + \mathbf{A} \cdot \mathbf{R} \\ &= \mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{T} - \mathbf{V}) + \mathbf{V} \\ &= \mathbf{T} - \mathbf{V} + \mathbf{V} \\ &= \mathbf{T} \end{aligned}$$

Uniformity follows from the leftover hash lemma, while the preimage norm is apparent by construction. Finally, to observe preimage indistinguishability, we will show the distribution of $(\mathbf{A}, \mathbf{P}, \mathbf{T})$ generated through preimage sampling is statistically close to sampling $\mathbf{P}$ as a uniform matrix in $[\sigma \cdot B]^{m \times k}$ and setting $\mathbf{T} = \mathbf{A} \cdot \mathbf{P}$. Since the latter distribution is independent of any trapdoor to $\mathbf{A}$, preimage indistinguishability follows.

1. This is the real preimage sampling experiment, where $\mathbf{T} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k}$ and $\mathbf{P} \leftarrow \text{SamplePre}(\mathbf{A}, \text{td}, \mathbf{T}, \sigma)$.

2. In this experiment, sample $U \xleftarrow{R} \mathbb{Z}_q^{n \times k}$ and computing $P = td \cdot G^{-1}(U) + R$ and $T = A \cdot P$.
3. In this experiment, sample $P \xleftarrow{R} [\sigma \cdot B]^{m \times k}$.

We can see experiments (1) and (2) above are equivalent, by implicitly setting $U = (T - V)$. and steps (2) and (3) are statistically close by a simple noise-smudging argument as $\sigma \cdot B \gg \sigma$.

7.3.2 Log-Depth Key-Homomorphic RAM Encoding

Recall at a high level [AMR25] RAM encodings RHash is encoded as recursively as linear ‘Merkle hash’ of the input, where each level can be implemented through log-depth elementary matrix operations. Thus to ensure the garbler’s overall computation is log-depth, we are restricted to performing an only *constant* number of recursions. This can be accomplished by setting the ‘arity’ of Hash to λ (parameter β in [AMR25]). This incurs a λ factor in the size of the hash public parameters compared to the usual choice of choosing this arity to be a constant > 1 . However, now for any circuit family of polynomially bounded size, because there exists constant $c_{\max}$ such that $|C| \leq \lambda^{c_{\max}}$, we can hash all the wires of C in a string of length $\lambda^{c_{\max}}$, which can be done in depth $c_{\max} \cdot \log(\lambda, d)$.

7.4 Succinctness and Decomposability

Recall the garbling material consists of a set of public parameters pp of RSetup and a series of matrices $\{rk^{(i)}\}_{i \in [d]}$, C_A, C_Z, C_x, dk over $\mathbb{Z}_q$ dimensions dependent on n, m, t, k . From inspection of our parameter instantiation we can see $q \in 2^{\text{poly}(\lambda, d)}$, and thus $n, m \in \text{poly}(\lambda, d)$. Recall that t, k are also of size $\text{poly}(\lambda, d)$ by the succinctness property of the key-homomorphic hash.

Note that the garbled input labels consists of $(C_A, C_Z, C_B, C_x, B_{\text{inp}})$, where $C_A, C_Z, C_B, B_{\text{inp}}$ are computable independent of x , and C_x is computed as

$$\begin{aligned} C_x &= s^T \cdot (B_{\text{inp}} - x^T \otimes G) + e_3^T \\ &= s^T \cdot ([B_{\langle 1 \rangle}^{(r)} \mid \cdots \mid B_{\langle \ell \rangle}^{(r)}] - x^T \otimes G) + e_3^T \\ &= \left[s^T \cdot (B_{\langle 1 \rangle}^{(r)} - x_1 \cdot G) + e_{3,1}^T \mid \cdots \mid s^T \cdot (B_{\langle \ell \rangle}^{(r)} - x_\ell \cdot G) + e_{3,\ell}^T \right] \end{aligned}$$

where each $e_{2,i}$ are simply length m the blocks of the error vector e_3 . It is easy to see each ‘block’ of C_x depends on exactly one bit x_i of x , giving us decomposability.

7.5 Correctness

To show correctness of our scheme, it suffices to show it conforms to our notion of algebraic labeling - i.e. taking the garbler’s authentication key as $(r^{(\text{out})}, s^{(\text{out})})$, the decoding key is the last entry of $r^{(\text{out})}$ and the evaluator’s label takes the form of $(r^{(\text{out})} + C(x) \cdot s^{(\text{out})})$.

Inspecting our scheme, recall from our authentication key that

$$r^{(\text{out})} = [s^T [B_{\text{out}} \mid B_{\text{out}} \cdot G^{-1}(u)] + \delta \cdot \mathbf{1}_{m+1}^T]_p, \quad s^{(\text{out})} = [s^T [G \mid u]]_p$$

With this in mind, let $(h^{(i)}, B^{(i)}, C^{(i)})$ be the evaluator’s share of the hash of all depth i wires (Where $B^{(0)}$ consists of the hash of the input encoding). Similarly, let $x^{(i)}$ be the value of all depth i wires (with $x = x^{(0)}$). We will inductively bound the distance of

$$\|s^T \cdot (B^{(i)} - h^{(i)} \otimes G) - C^{(i)}\| \leq B^{i+1} \cdot \sigma$$

Let $e^{(i)}$ refer to this difference. To begin with the base case, recall $(h^{(0)}, B^{(0)}, C^{(0)})$ is computed from the garbled input (x, B_{inp}, C_x) as:

$$h^{(0)} = \text{RHash}(x), \quad B^{(0)} \leftarrow \text{RWrite}(pp, B, [\ell], B_{\text{inp}}), \quad C^{(0)} = [C_B \mid C_x \mid C_Z] \cdot H^{(0)}$$

where $\mathbf{H}^{(0)} \leftarrow \text{RXWrite}(\text{pp}, \mathbf{B}, [\ell], \mathbf{B}_{\text{inp}}, x, 0)$. From the write homomorphism of $\mathbf{H}$, this equation for $\mathbf{C}^{(0)}$ can be rewritten as

$$\begin{aligned} [\mathbf{C}_B \mid \mathbf{C}_x \mid \mathbf{C}_Z] \cdot \mathbf{H}^{(0)} &= \mathbf{s}^\top [\mathbf{B} \mid \mathbf{B}_{\text{inp}} \mid \mathbf{Z}] \cdot \mathbf{H}^{(0)} + [\mathbf{e}_2^\top \mid \mathbf{e}_3^\top \mid \mathbf{e}_0^\top] \cdot \mathbf{H}^{(0)} \\ &= \mathbf{s}^\top (\mathbf{B}^{(0)} - h^{(0)} \otimes \mathbf{G}) + [\mathbf{e}_2^\top \mid \mathbf{e}_3^\top \mid \mathbf{e}_0^\top] \cdot \mathbf{H}^{(0)} \end{aligned}$$

Recall from our construction we can bound the entries of $\mathbf{e}_0, \mathbf{e}_2, \mathbf{e}_3$ with $\sigma \cdot t \in \sigma \cdot \text{poly}(\lambda)$ and we can bound the entries of $\mathbf{H}^{(0)}$ with $\text{poly}(\lambda)$ by the low norm property of [Definition 7.9](#), so the combined error vector has entries bound by $\sigma \cdot \text{poly}(\lambda) \ll \sigma \cdot B$.

Next, recall $\mathbf{C}^{(i)}$ is derived from $\mathbf{C}^{(0)}, \dots, \mathbf{C}^{(i-1)}$, the evaluator first set

$$\mathbf{C}_{C_i}^{(i-1)} = [\mathbf{C}^{(0)} \mid \dots \mid \mathbf{C}^{(i-1)} \mid \mathbf{C}_Z] \cdot \mathbf{H}_{C_i} \quad (7.1)$$

then computes

$$\mathbf{C}^{(i)} = \mathbf{C}_{C_i}^{(i-1)} + [\mathbf{C}_A \mid \mathbf{C}_Z] \cdot \text{rk}^{(i)} \quad (7.2)$$

where $\mathbf{H}_{C_i} \leftarrow \text{SEvalFX}(\text{pp}, [\mathbf{B}^{(0)} \mid \dots \mid \mathbf{B}^{(i-1)}], C_i, C_{[i-1]}(x))$ and $\text{rk}^{(i)}$ is sampled as a preimage of $\mathbf{B}^{(i)} - \mathbf{B}_{C_i}^{(i-1)}$ relative to $[\mathbf{A} \mid \mathbf{Z}]$ with width σ . Substituting $\mathbf{C}_A = \mathbf{s}^\top \mathbf{A} + \mathbf{e}_0^\top$, $\mathbf{C}_Z = \mathbf{s}^\top \mathbf{Z} + \mathbf{e}_1^\top$ and $\mathbf{C}^{(i')} = \mathbf{s}^\top \mathbf{B}^{(i)} + \mathbf{e}_{(i')}^\top$ for $i' \in [0, i-1]$, we can rewrite [Eq. \(7.1\)](#)

$$\begin{aligned} \mathbf{C}_{C_i}^{(i-1)} &= [\mathbf{C}^{(0)} \mid \dots \mid \mathbf{C}^{(i-1)} \mid \mathbf{C}_Z] \cdot \mathbf{H}_{C_i} = \mathbf{s}^\top \cdot [\mathbf{B}^{(0)} \mid \dots \mid \mathbf{B}^{(i-1)} \mid \mathbf{Z}] \cdot \mathbf{H}_{C_i} + [\mathbf{e}_{(0)}^\top \mid \dots \mid \mathbf{e}_{(i-1)}^\top \mid \mathbf{e}_1^\top] \cdot \mathbf{H}_{C_i} \\ &= \mathbf{s}^\top \cdot \mathbf{B}_{C_i}^{(i-1)} + \mathbf{e}_{C_i}^{(i-1)\top} \end{aligned}$$

relying on lattice homomorphism from [Remark 7.13](#). Since $\|\mathbf{H}_{C_i}\| \leq m^{O(d)}$ where $m \in \text{poly}(\lambda)$ and C_i is depth 1, we can bound the magnitude of the new error vector $\|\mathbf{e}_{C_i}^{(i-1)\top}\| \leq B^i \cdot \sigma \cdot \text{poly}(\lambda)$. Now we can substitute [Eq. \(7.2\)](#) as

$$\begin{aligned} \mathbf{C}_{C_i}^{(i-1)} + [\mathbf{C}_A \mid \mathbf{C}_Z] \cdot \text{rk}^{(i)} &= \mathbf{s}^\top \mathbf{B}_{C_i}^{(i-1)} + \mathbf{e}_{C_i}^{(i-1)\top} + \mathbf{s}^\top [\mathbf{A} \mid \mathbf{Z}] \cdot \text{rk}^{(i)} + [\mathbf{e}_0^\top \mid \mathbf{e}_1^\top] \cdot \text{rk}^{(i)} \\ &= \mathbf{s}^\top (\mathbf{B}_{C_i}^{(i-1)} + (\mathbf{B}^{(i)} - \mathbf{B}_{C_i}^{(i-1)})) + \mathbf{e}_{C_i}^{(i-1)\top} + [\mathbf{e}_0^\top \mid \mathbf{e}_1^\top] \cdot \text{rk}^{(i)} \\ &= \mathbf{s}^\top \cdot \mathbf{B}^{(i)} + \mathbf{e}^{(i)} \end{aligned}$$

Recall that from construction $\|\text{rk}^{(i)}\| \leq B \cdot \sigma$ and $\|\mathbf{e}_0, \mathbf{e}_1\| \leq \sigma$ so we can bound the new error vector $\|\mathbf{e}^{(i)}\| \leq B \cdot \sigma \cdot \text{poly}(\lambda) + B^i \cdot \sigma \cdot \text{poly}(\lambda)$. Since B is superpolynomial, we can loosely upper bound this quantity by $B^{i+1} \cdot \sigma$.

This allows us to conclude $\mathbf{C}^{(d)} = \mathbf{s}^\top \cdot \mathbf{B}^{(d)} + \mathbf{e}^{(d)}$ for $\|\mathbf{e}^{(d)}\| \leq B^{d+1} \cdot \sigma$. Now all that remains to compute $\mathbf{C}_{\text{out}}$ is a $\text{HRead}(\cdot)$ on the value of the output wire. Using an analogous argument, we can bound $\|\mathbf{C}_{\text{out}} - \mathbf{s}^\top (\mathbf{B}_{\text{out}} - \mathbf{C}(x) \cdot \mathbf{G})\| \leq B^{d+2} \cdot \sigma$. Now to argue correctness, we can simply observe that we chose our rounding modulus p such that $(q/p) = \Delta \gg B^{d+2} \cdot \sigma$, which means that $\lfloor \mathbf{C}_{\text{out}} + \delta \cdot \mathbf{1}_m \rfloor_p = \lfloor \mathbf{s}^\top \cdot \mathbf{B} + \delta \cdot \mathbf{1}_m \rfloor_p - \lfloor \mathbf{s}^\top \cdot \mathbf{C}(x) \cdot \mathbf{G} \rfloor_p$ with $1 - \text{negl}(\lambda)$ probability by [Lemma 7.3](#) (with the rounding shift). Similarly, we can reason that since $\Delta \gg \|\mathbf{e}_{\text{large}}\|$ our decoding key $(\mathbf{s}^\top \cdot \mathbf{B}_{\text{out}} + \mathbf{e}_{\text{large}}^\top) \cdot \mathbf{G}^{-1}(\mathbf{u})$ will round to $\lfloor \mathbf{s}^\top \cdot \mathbf{B}_{\text{out}} \cdot \mathbf{G}^{-1}(\mathbf{u}) \rfloor_p$ with overwhelming probability.

7.6 Security

Below, we will prove the one-wayness of [Construction 7.12](#), which by [Lemma 3.3](#) suffices to show authenticity.

Theorem 7.14 (One-Wayness). *Assuming decomposed LWE, garbling scheme [Construction 7.12](#) satisfies selective reusable one-wayness as an algebraic garbling scheme.*

Proof. Consider the following sequence of hybrids:

- Hyb_{real} : This is the real selective one-wayness experiment instantiated with [Construction 7.12](#).

1. The adversary $\mathcal{A}$ specifies circuit C and input x^* to the challenger.

2. The challenger samples the following:

- (a) Setup material $(pp, Z) \leftarrow \text{RSetup}(1^\lambda)$
- (b) Randomizing binary matrices:

$$\mathbf{R}_A \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m}, \mathbf{R}_B \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m \cdot k}, \mathbf{R}_{\text{inp}} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m \cdot \ell}$$

- (c) Trapdoor matrix $(\mathbf{A}, \text{td}_A) \leftarrow \text{TrapGen}(1^n, 1^m, q)$.
- (d) Encoding matrices $\mathbf{B} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}, \mathbf{B}_{\text{inp}} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times m \cdot \ell}$ and decoding vector $\mathbf{u} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n$.
- (e) LWE secret and error vectors :

$$\mathbf{s} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n, \quad \mathbf{e}_0 \xleftarrow{\mathbb{R}} \mathcal{D}_{\sigma}^t, \quad \mathbf{e}_{\text{large}} \xleftarrow{\mathbb{R}} \mathcal{D}_{\sigma_{\text{large}}}^{m \cdot k}$$

$$\mathbf{e}_1^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_A, \quad \mathbf{e}_2^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_B, \quad \mathbf{e}_3^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_{\text{inp}}.$$

- (f) Rekeying material for each $i \in [d]$:

$$\mathbf{B}^{(0)} \leftarrow \text{RWrite}(pp, \mathbf{B}, [\ell], \mathbf{B}_{\text{inp}}), \quad \mathbf{B}^{(i)} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}$$

$$\mathbf{B}_{C_i}^{(i-1)} \leftarrow \text{SEvalF}(pp, [\mathbf{B}^{(0)} \mid \dots \mid \mathbf{B}^{(i-1)}], C_i), \quad \text{rk}^{(i)} \leftarrow \text{SamplePre}\left([\mathbf{A} \mid \mathbf{Z}], \begin{bmatrix} \text{td}_A \\ 0 \end{bmatrix}, \mathbf{B}^{(i)} - \mathbf{B}_{C_i}^{(i-1)}, 1\right)$$

- (g) Rounding shift $\delta \xleftarrow{\mathbb{R}} \mathbb{Z}_q$.

3. The challenger computes the adversary's view as follows:

- (a) Set public data $\text{pd} = (pp, Z, \mathbf{B}, \delta)$.
- (b) Set garbled circuit and public garbling key $\hat{C} = \text{gk} = \{\text{rk}^{(i)}\}_{i \in [d]}$.
- (c) Set garbled input label $L_x = (C_A, C_Z, C_B, C_x, \mathbf{B}_{\text{inp}})$ where

$$\mathbf{C}_Z = \mathbf{s}^\top \cdot \mathbf{Z} + \mathbf{e}_0^\top, \quad \mathbf{C}_A = \mathbf{s}^\top \cdot \mathbf{A} + \mathbf{e}_1^\top$$

$$\mathbf{C}_B = \mathbf{s}^\top \cdot (\mathbf{B} - \text{RHash}(0) \otimes \mathbf{G}) + \mathbf{e}_2^\top, \quad \mathbf{C}_x = \mathbf{s}^\top \cdot (\mathbf{B}_{\text{inp}} - \mathbf{x}^\top \otimes \mathbf{G}) + \mathbf{e}_3^\top.$$

- (d) Set decoding key $\text{dk} = \left\lfloor \left(\mathbf{s}^\top \cdot \mathbf{B}_{\text{out}} + \mathbf{e}_{\text{large}}^\top \right) \cdot \mathbf{G}^{-1}(\mathbf{u}) + \delta \right\rfloor_p$ where

$$\mathbf{B}_{\text{out}} \leftarrow \text{RRead}(pp, \mathbf{B}^{(d)}, |C|^{17})$$

4. The adversary outputs $\mathbf{s}'$ and wins if $\mathbf{s}' = \lfloor \mathbf{s}^\top [\mathbf{G} \mid \mathbf{u}] \rfloor_p$.

- Hyb_0 : In this experiment, the adversary wins if it outputs any $\mathbf{s}'$ which satisfies the following:

4. The adversary outputs $\mathbf{s}' \in \mathbb{Z}_p^{m+1}$. Recover candidate of $\mathbf{s}$ as $\mathbf{s}_q \in \mathbb{Z}_q^n$ where we set the i^{th} bit of each entry of $\mathbf{s}_q$ as $\lceil \mathbf{s}'^\top \cdot \mathbf{G}^{-1}(\mathbf{I} \cdot p \cdot 2^{i-1}) \rceil_2$. The adversary wins if $\|\mathbf{s}_q^\top \cdot \mathbf{Z} - \mathbf{C}_Z\| \leq \sigma$.

- $\text{Hyb}_{1,i}$: In this experiment, the challenger samples the rekeying keys without using td_A :

- 2.(f) Rekeying material for each $i \in [d]$:

$$\mathbf{B}^{(0)} \leftarrow \text{RWrite}(pp, \mathbf{B}, [\ell], \mathbf{B}_{\text{inp}}), \quad \text{rk}^{(i)} \xleftarrow{\mathbb{R}} \mathcal{D}_B^{(m+t) \times k \cdot m}$$

$$\mathbf{B}_{C_i}^{(i-1)} \leftarrow \text{SEvalF}(pp, [\mathbf{B}^{(0)} \mid \dots \mid \mathbf{B}^{(i-1)}], C_i), \quad \mathbf{B}^{(i)} = [\mathbf{A} \mid \mathbf{Z}] \cdot \text{rk}^{(i)} + \mathbf{B}_{C_i}^{(i-1)}$$

- Hyb_2 : In this experiment, the challenger samples $\mathbf{A}$ as a random matrix.

2.(c) Trapdoor matrix $\mathbf{A} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times m}$.

¹⁷We will use the 'last' wire of C as the output wire

- Hyb₃: In this experiment, the challenger derives matrices $\mathbf{A}, \mathbf{B}, \mathbf{B}_{\text{inp}}$ and vector $\mathbf{u}$ from $\mathbf{Z}$.

2.(c) Matrix $\mathbf{A} = \mathbf{Z} \cdot \mathbf{R}_A$.

2.(d) Encoding matrices $\mathbf{B} = \mathbf{Z} \cdot \mathbf{R}_B + \text{RHash}(0) \otimes \mathbf{G}, \mathbf{B}_{\text{inp}} = \mathbf{Z} \cdot \mathbf{R}_{\text{inp}} + \mathbf{x}^\top \otimes \mathbf{G}$ and decoding vector $\mathbf{u} = \mathbf{Z} \cdot \mathbf{r}_u$ for $\mathbf{r}_u \xleftarrow{\mathbb{R}} \{0, 1\}^m$.

- Hyb₄: In this experiment, the challenger generates $\mathbf{C}_A, \mathbf{C}_B, \mathbf{C}_x$ and dk from $\mathbf{C}_Z$.

3.(c) Set garbled input label $L_x = (\mathbf{C}_A, \mathbf{C}_Z, \mathbf{C}_B, \mathbf{C}_x, \mathbf{B}_{\text{inp}})$ where

$$\mathbf{C}_Z = \mathbf{s}^\top \cdot \mathbf{Z} + \mathbf{e}_0^\top, \quad \mathbf{C}_A = \mathbf{C}_Z \cdot \mathbf{R}_A$$

$$\mathbf{C}_B = \mathbf{C}_Z \cdot \mathbf{R}_B, \quad \mathbf{C}_x = \mathbf{C}_Z \cdot \mathbf{R}_x.$$

3.(d) Set decoding key $\text{dk} = \lfloor ((\mathbf{C}_{\text{out}} + \mathbf{e}_{\text{large}}) \cdot \mathbf{G}^{-1}(\mathbf{u}) - \mathbf{C}_Z \cdot \mathbf{r}_u \cdot \mathbf{C}(x)^\top) + \delta \rfloor_p$.

- Hyb₅: In this experiment, $\mathbf{C}_Z$ is replaced with a uniform random vector.

3.(c) Set garbled input label $L_x = (\mathbf{C}_A, \mathbf{C}_Z, \mathbf{C}_B, \mathbf{C}_x, \mathbf{B}_{\text{inp}})$ where

$$\mathbf{C}_Z \xleftarrow{\mathbb{R}} \mathbb{Z}_q^t, \quad \mathbf{C}_A = \mathbf{C}_Z \cdot \mathbf{R}_A$$

$$\mathbf{C}_B = \mathbf{C}_Z \cdot \mathbf{R}_B, \quad \mathbf{C}_x = \mathbf{C}_Z \cdot \mathbf{R}_x.$$

Lemma 7.15. For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that

$$\Pr[\text{Hyb}_{\text{real}}(\mathcal{A}, \lambda) = 1] \leq \Pr[\text{Hyb}_0(\mathcal{A}, \lambda) = 1] - \text{negl}(\lambda).$$

Proof. Observe when the first m entries of $\mathbf{s}'$ are in fact $\lfloor \mathbf{s}^\top \cdot \mathbf{G} \rfloor_p$, the computation $\mathbf{s}^\top \cdot \mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{I} \cdot p \cdot 2^{i-1}) \bmod p \equiv \mathbf{s}^\top \cdot 2^{i-1} \bmod p$ indeed recovers the i^{th} bit of $\mathbf{s}$ when rounded to $\mathbb{Z}_2$. By construction, $\mathbf{s}^\top \mathbf{B}_Z - \mathbf{C}_Z = \mathbf{e}_0^\top$, which is bounded entries $\|\mathbf{e}_0^\top\| \leq \sigma$. $\square$

Lemma 7.16. For all adversaries $\mathcal{A}$,

$$\Pr[\text{Hyb}_0(\mathcal{A}, \lambda) = 1] = \Pr[\text{Hyb}_{1,0}(\mathcal{A}, \lambda) = 1].$$

Proof. These hybrids are identical, as there is no $\text{rk}^{(0)}$. $\square$

Lemma 7.17. For all adversaries $\mathcal{A}$ and $i \in [d]$, there exists a negligible function $\text{negl}(\lambda)$ such that

$$|\Pr[\text{Hyb}_{1,i-1}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_{1,i}(\mathcal{A}, \lambda) = 1]| = \text{negl}(\lambda).$$

Proof. This follows from preimage distribution property of lattice gadget trapdoor. $\square$

Lemma 7.18. For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that

$$|\Pr[\text{Hyb}_{1,d}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_2(\mathcal{A}, \lambda) = 1]| = \text{negl}(\lambda).$$

Proof. This follows from the trapgen uniformity of lattice gadget trapdoors. $\square$

Lemma 7.19. For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that

$$|\Pr[\text{Hyb}_2(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_3(\mathcal{A}, \lambda) = 1]| = \text{negl}(\lambda).$$

Proof. This follows from the leftover hash lemma (specifically the LHL compatibility of the key-homomorphic RAM encoding). $\square$

Lemma 7.20. *For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$|\Pr[\text{Hyb}_3(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_4(\mathcal{A}, \lambda) = 1]| = \text{negl}(\lambda).$$

Proof. First observe that for C_A , C_B , and C_x , this change is purely syntactic, as we can rewrite

$$\begin{aligned} C_A &= C_Z \cdot R_A = (s^\top \cdot Z + e_0^\top) \cdot R_A = s^\top \cdot (Z \cdot R_A) + e_0^\top \cdot R_A = s^\top \cdot A + e_1^\top \\ C_B &= C_Z \cdot R_B = (s^\top \cdot Z + e_0^\top) \cdot R_B = s^\top \cdot ((Z \cdot R_B + \text{RHash}(0) \otimes G) - \text{RHash}(0) \otimes G) + e_0^\top \cdot R_B \\ &= s^\top \cdot (B - \text{RHash}(0) \otimes G) + e_2^\top \\ C_x &= C_Z \cdot R_x = (s^\top \cdot Z + e_0^\top) \cdot R_x = s^\top \cdot ((Z \cdot R_x + x^\top \otimes G) - x^\top \otimes G) + e_0^\top \cdot R_x \\ &= s^\top \cdot (B_{\text{inp}} - x^\top \otimes G) + e_3^\top \end{aligned}$$

For the decoding key, first we can observe similar to the prior cases that $C_Z \cdot r_u = s^\top \cdot u + e_0^\top \cdot r_u$. In addition, recall from our correctness analysis that $\|C_{\text{out}} - s^\top \cdot (B_{\text{out}} - C(x) \cdot G)\| \leq B^{d+2} \cdot \sigma$. Taking these two together, we can bound

$$\left\| \left((C_{\text{out}} + e_{\text{large}}^\top) \cdot G^{-1}(u) - C_Z \cdot r_u \cdot C(x) \right) - (s^\top \cdot B_{\text{out}}) \right\| \leq B^{d+2} \cdot \sigma + \text{poly}(\lambda).$$

Since we chose $e_{\text{large}} \gg (B^{d+2} \cdot \sigma)$, the noisy distributions are statistically close by a smudging argument [Lemma 7.2](#). $\square$

Lemma 7.21. *Assuming decomposed LWE, for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$|\Pr[\text{Hyb}_4(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_5(\mathcal{A}, \lambda) = 1]| = \text{negl}(\lambda).$$

Proof. Specifically, this will follow from the LWE indistinguishability property of the key-homomorphic RAM encoding, where

$$\{\text{pp}, Z, s^\top \cdot Z + e^\top\} \approx \{\text{pp}, Z, t^\top\}.$$

Observe at by Hyb_4 , all the elements in the view of the adversary dependent on s can be derived from $C_Z = s^\top \cdot Z + e_0^\top$. This can be directly replaced with the (decomposed) LWE challenge vector. $\square$

Lemma 7.22. *For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$\Pr[\text{Hyb}_5(\mathcal{A}, \lambda) = 1] = \text{negl}(\lambda).$$

Proof. Since C_Z is a random vector, with overwhelming probability there does not exist s_q for which $\|C_Z - s_q \cdot Z\| \leq \sigma$. $\square$

Taking together [Lemmas 7.15](#) to [7.22](#) gives us the proof of our theorem. $\square$

8 Reusable Fully Succinct Garbling

In this section, we build upon the prior construction of [Section 7](#) to the setting of *private* garbling. Formally, we show the following theorem:

Theorem 8.1 (Reusable Succinct Private Garbling). *Assuming decomposed LWE [Assumption 7.5](#) parameterized as [Construction 8.4](#), there exists a reusable succinct private garbling scheme Π_{garble} satisfying the following:*

- For a circuit $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$ of depth d , the public data pd and garbling material $|\hat{C}|, |\hat{x}|, |\text{gk}|$ are of size $\text{poly}(\lambda, d)$ (independent of circuit size $|C|$).
- The garbling algorithms (Setup, GarbC, GarbInp) are computable in NC^1 .
- Π_{garble} satisfies selective input-hiding simulation security.

8.1 Preliminaries

We will reuse many of the preliminaries and notation from the prior [Section 7](#). In addition, we introduce the following additional ingredients:

Definition 8.2 (Homomorphic Encryption and Evaluation à la [GSW13]). Let λ be a security parameter and n, m, q, σ be lattice parameters for $m \geq 2n \log q$. Let $C_{\ell, d}$ be a family of depth d boolean circuits $C : \{0, 1\}^\ell \rightarrow \{0, 1\}$. There exists efficient algorithms:

- **KeyGen**(1^λ) $\rightarrow$ pk, sk: On input security parameter λ , the key generation algorithm outputs public and secret keys of the form:

$$\text{pk} = \mathbf{B}_{\text{fhe}} = \begin{bmatrix} \bar{\mathbf{B}}_{\text{fhe}} \\ \mathbf{s}^\top \cdot \bar{\mathbf{B}}_{\text{fhe}} + \mathbf{e}_{\text{fhe}}^\top \end{bmatrix} \in \mathbb{Z}_q^{(n+1) \times m}, \quad \text{sk} = \mathbf{r}^\top = [\mathbf{s}^\top \mid -1] \in \mathbb{Z}_q^{n+1},$$

where $\mathbf{s} \in \mathbb{Z}_q^n$, $\bar{\mathbf{B}}_{\text{fhe}} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{e}_{\text{fhe}} \xleftarrow{\mathcal{R}} \mathcal{D}_\sigma$. We will in general use $\bar{\mathbf{B}}$ and $\underline{\mathbf{B}}$ to denote the first n rows and last row of matrices (either public keys or ciphertexts) $\mathbf{B} \in \mathbb{Z}_q^{(n+1) \times m}$.

- **Encrypt**(pk, x) $\rightarrow$ ct: On input public key $\text{pk} = \mathbf{B}_{\text{fhe}}$ and bit $x \in \{0, 1\}^\ell$, the encryption algorithm outputs a ciphertext of the form:

$$\text{ct}_x = \mathbf{B}_{\text{fhe}} \mathbf{R} - x \otimes \mathbf{G} \in \mathbb{Z}_q^{(n+1) \times \ell \cdot m},$$

where $\mathbf{R} \xleftarrow{\mathcal{R}} \{0, 1\}^{m \times \ell \cdot m}$ is the encryption randomness. Note that ct_x is decomposable, and can be written as $[\mathbf{B}_{\text{fhe}} \cdot \mathbf{R}_1 - x_1 \cdot \mathbf{G} \mid \cdots \mid \mathbf{B}_{\text{fhe}} \cdot \mathbf{R}_\ell - x_\ell \cdot \mathbf{G}]$ where $\mathbf{R} = [\mathbf{R}_1 \mid \cdots \mid \mathbf{R}_\ell]$.

- **FHEEval**(C) $\rightarrow C_{\text{fhe}}$: On input circuit $C \in C_{\ell, d}$, the homomorphic evaluation algorithm deterministically produces circuit $C_{\text{fhe}} : \{0, 1\}^{\ell \cdot (n+1) \cdot m \cdot \log q} \rightarrow \mathbb{Z}_q^{(n+1) \times m}$, which takes in a (bit representation) of ct_x and outputs a matrix $\mathbf{B}_{C(x)} \in \mathbb{Z}_q^{(n+1) \times m}$.

Since we will not be directly relying on the structure of the FHE scheme, we omit the security requirements associated with the above FHE scheme and describe the structural properties:

- **Homomorphism:** Let $\mathbf{B}_{\text{fhe}}$ be an public key matrix and ciphertext $\text{ct} = \text{Bits}(\mathbf{X})$ for $\mathbf{X} = \mathbf{B}_{\text{fhe}} \cdot \mathbf{R} - x^\top \otimes \mathbf{G}$ be an FHE ciphertext. Then there exists matrix $\mathbf{R}_C$ where

$$C_{\text{fhe}}(\mathbf{X}) = \mathbf{B}_{\text{fhe}} \cdot \mathbf{R}_C - C(x) \cdot \mathbf{G}$$

- **Low Norm:** In the above described relation, matrix $\|\mathbf{R}_C\| \leq m^{O(d)} \cdot \|\mathbf{R}\|$.
- **Circuit Depth:** For a circuit C of depth d , circuit C_{fhe} has $d \cdot O(\log m + \log \log q)$.

Definition 8.3 (Bit Packing and Homomorphic Evaluation for Matrix-Valued Circuits [BTWV17]). Let λ be a security parameter and n, m, q, σ be lattice parameters where $m \geq 2n \log q$ and $n \geq \lambda$. For matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times m}$, let $\text{Bits}(\mathbf{M}) \in \{0, 1\}^{n \cdot m \cdot \log q}$ denote the entry-wise bit-decomposition and flattening of $\mathbf{M}$. Then there exists efficient (deterministic) algorithms:

- **Pack**($\mathbf{B}$) $\rightarrow \mathbf{B}_{\text{Pack}}$: On input matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times (m^2 \cdot n \cdot \log q)}$, Pack computes matrix $\mathbf{B}_{\text{Pack}} \in \mathbb{Z}_q^{n \times m}$.
- **PackE**($1^n, 1^m, q$) $\rightarrow \mathbf{H}_{\text{Pack}}$: On input dimensions n, m, q , PackE produces low norm matrix $\mathbf{H}_{\text{Pack}} \in \mathbb{Z}_q^{(m^2 \cdot n \cdot \log q) \times m}$

Such that the following properties hold:

- **Low Norm:** The matrix $\mathbf{H}_{\text{Pack}}$ has low norm $\|\mathbf{H}_{\text{Pack}}\| = 1$.
- **Homomorphism:** For any matrix $\mathbf{B} \in \mathbb{Z}_q^{n \times (n \cdot m^2 \cdot \log q)}$, the relation

$$(\mathbf{B} - \text{Bits}(\mathbf{M})^\top \otimes \mathbf{G}) \cdot \mathbf{H}_{\text{Pack}} = \mathbf{B}_{\text{Pack}} - \mathbf{M}$$

holds.

By composing this with [Definition 7.8](#) encodings, this allows one to extend homomorphic evaluation to *matrix-valued* circuits, where the output of C takes the form of a matrix $\in \mathbb{Z}_q^{n \times m}$.

8.2 Construction

Our construction of private garbling follows from ‘dual-use’ technique of [BTWV17], used to lift attribute-based encryption to predicate encryption [GVW15, BTWV17] and attribute-based LFE to LFE [QWW18]. At a high level, this technique shares a decryption secret $\mathbf{s}$ between a GSW-style FHE scheme and the underlying base primitive (in our case the privacy-free garbling scheme). This enables input/attribute hiding by simply encrypting the input under the FHE scheme, and instead issuing garbled labels of the FHE ciphertexts, along with running the FHE evaluation of the circuit to be garbled. Then, using the structural similarity of the base primitive with the FHE scheme allows one to ‘simultaneously’ decode the attribute-based primitive while decrypting the FHE scheme.

Construction 8.4 (Succinct Reusable Garbling). Let λ be the security parameter and d be a bound on circuit depth. Our scheme relies on the following lattice ingredients:

- GSW Fully homomorphic encryption Π_{fhe} Definition 8.2.
- Lattice homomorphic evaluation Definition 7.8.
- Bit-packing Definition 8.3.
- The privacy-free reusable garbling protocol $\Pi_{\text{PF-Garble}}$ of Section 7, which we will use as-is except for the explicit specification of secret vector $\mathbf{s}$ and the decoding phase.

We set our parameters as follows:

$$n \geq \lambda \quad , \quad m = 3n \log q \quad , \quad B = \lambda^{\log(\lambda)} \quad , \quad \sigma_{\text{large}} = \sigma \cdot B^{d_{\text{fhe}}+3}$$

$$\Delta = B^{d_{\text{fhe}}+4} \cdot \sigma \quad , \quad p = \sigma \cdot B \quad , \quad q = \Delta \cdot p,$$

where the $d_{\text{fhe}} \in d \cdot O(\log m + \log \log q)$ is the depth of the FHE evaluation circuit C'_{fhe} specified below and $\text{dLWE}_{n,m,q,2m^2\lambda,\chi,\sigma}$ is hard with respect to security parameter λ , which in turn fix parameters t, k of the RAM encoding. Our private garbling scheme proceeds as follows:

- **Input Encoding Phase:** The garbler samples public data $\text{pd} \leftarrow \Pi_{\text{PF-Garble}}.\text{Setup}(1^\lambda, 1^{d_{\text{fhe}}})$ of the privacy free garbling scheme, which will also act as public data of the private garbling scheme.

To compute the input encoding, the garbler also samples FHE key-pair

$$\text{pk} = \mathbf{B}_{\text{fhe}} = \left[\mathbf{s}^\top \cdot \bar{\mathbf{B}}_{\text{fhe}} + \mathbf{e}_{\text{fhe}}^\top \right] \in \mathbb{Z}_q^{(n+1) \times m} \quad , \quad \text{sk} = \mathbf{r}^\top = [\mathbf{s}^\top \mid -1] \in \mathbb{Z}_q^{n+1},$$

where $\mathbf{B}_{\text{fhe}} \xleftarrow{\mathcal{R}} \mathbb{Z}_q^{n \times m}$, $\mathbf{s}^\top \xleftarrow{\mathcal{R}} \mathbb{Z}_q^n$ and $\mathbf{e}_{\text{fhe}}^\top \xleftarrow{\mathcal{R}} \mathcal{D}_\sigma^m$. Using this key, the garbler computes ciphertext $\text{ct}_x \leftarrow \text{Encrypt}(\text{pk}, x)$, and derives privacy-free input encoding $\llbracket \text{ct}_x \rrbracket \leftarrow \text{Enc}(\text{gk}, \text{Bits}(\text{ct}_x))$ using the same secret $\mathbf{s}$. The garbler sends public data $\text{pd}, \text{pk}, \llbracket \text{ct}_x \rrbracket$ to the evaluator.

- **Garbling:** Define the circuit $C'_{\text{fhe}} = \text{Bits}(\overline{\text{FHEEval}(C)})$ - in other words, outputs the bit representation of the first n rows of the evaluation of C under homomorphic encryption. The garbler and evaluator run $\Pi_{\text{PF-Garble}}$ on circuit C'_{fhe} and input (encoding) $\llbracket \text{Bits}(\text{ct}_x) \rrbracket$.
- **Output Phase:** Let $\mathbf{X}_{\text{out}} \in \mathbb{Z}_q^{n+1 \times m}$ be the homomorphic evaluation of $\text{FHEEval}(C)$ on input ct_x . The privacy-free garbling will produce encodings $\llbracket \text{Bits}(\bar{\mathbf{X}}_{\text{out}}) \rrbracket$ which contain shares of the form:

$$\llbracket \text{Bits}(\bar{\mathbf{X}}_{\text{out}}) \rrbracket_G = \mathbf{B}_{\text{out}} \in \mathbb{Z}_q^{n \times (m^2 \cdot n \cdot \log q)} \quad , \quad \llbracket \text{Bits}(\bar{\mathbf{X}}_{\text{out}}) \rrbracket_E = (\text{Bits}(\bar{\mathbf{X}}_{\text{out}}), \mathbf{B}_{\text{out}}, \mathbf{C}_{\text{out}})$$

- The garbler outputs authentication key as $\Pi_{\text{PF-Garble}}$, but with final entry decoding key as

$$\text{dk} = \left[(\mathbf{s}^\top \cdot (\mathbf{B}_{\text{Pack}} - \bar{\mathbf{B}}_{\text{fhe}} \cdot \mathbf{R}_C) + \mathbf{e}_{\text{large}}^\top) \cdot \mathbf{G}^{-1}(\mathbf{u}) + \delta \right]_p$$

where $\mathbf{B}_{\text{Pack}} \leftarrow \text{Pack}(\mathbf{B}_{\text{out}})$ and $\mathbf{R}_C$ is the matrix corresponding to FHE homomorphism relation.

- The evaluator computes all but the last entry of the output label as $\Pi_{\text{PF-Garble}}$, which is computed as

$$\left[\mathbf{C}_{\text{out}} \cdot \mathbf{H}_{\text{Pack}} \cdot \mathbf{G}^{-1}(\mathbf{u}) + \delta \right]_p$$

where $\mathbf{H}_{\text{Pack}}$ is computed through $\mathbf{H}_{\text{Pack}} \leftarrow \text{PackE}(1^n, 1^m, q)$.

8.3 Garbler Depth Complexity

We can separate the garbler's computation into two components:

- **FHE-related Computation:** The garbler computes the FHE KeyGen, Encrypt, FHEEval functions. Similar to [Section 7.3](#), we simply observe these can all be implemented from elementary arithmetic/matrix operations over a poly-size modulus, which can be computed in NC^1 . This includes the modified decoding phase.
- **Privacy-free Garbling Computation:** Since the circuit and inputs of our privacy-free garbling are polynomial, this follows directly from the NC^1 depth of $\Pi_{\text{PF-Garble}}$.

8.4 Succinctness and Decomposability

Observe the garbling material consists of garbling material for $\Pi_{\text{PF-Garble}}$ run on FHE ciphertext input bits ct_x and FHE homomorphic evaluation circuit C'_{fhe} , as well as the FHE ciphertext and public key. We can observe directly from our parameter choices that the FHE public key and ciphertext are of size $\text{poly}(\lambda, d)$. From the succinctness of $\Pi_{\text{PF-Garble}}$, the garbling material for $\Pi_{\text{PF-Garble}}$ is of size $\text{poly}(\lambda, d_{\text{fhe}})$, which we recall from the properties of our FHE scheme is $\text{poly}(\lambda, d \cdot O(\log m + \log \log q))$. From inspection, we can see parameters $m, \log q$ are both $\text{poly}(\lambda, d)$, which gives our the garbling material for $\Pi_{\text{PF-Garble}}$ to be $\text{poly}(\lambda, d)$ -succinct.

Note that the labels of our scheme correspond to labels of bits of the FHE ciphertext under the privacy-free garbling scheme. Thus decomposability follows directly from decomposability of the FHE scheme and $\Pi_{\text{PF-Garble}}$.

8.5 Correctness

To show correctness of our scheme, it suffices to show it conforms to our notion of algebraic labeling. Since all but the last term of the evaluator's label are directly generated as the privacy-free garbling label, the correctness of these follow directly from the correctness of the privacy-free scheme. For the decoding key/final entry of the authentication key, recall from the analysis of privacy-free garbling that we can bound the privacy-free output labels

$$\|C_{\text{out}} - \mathbf{s}^T(\mathbf{B}_{\text{out}} - \text{Bits}(\overline{\mathbf{X}}_{\text{out}}) \otimes \mathbf{G})\| \leq B^{d_{\text{fhe}}+2} \cdot \sigma$$

We can rewrite this as

$$\begin{aligned} C_{\text{out}} &= \mathbf{s}^T(\mathbf{B}_{\text{out}} - \text{Bits}(\overline{\mathbf{X}}_{\text{out}}) \otimes \mathbf{G}) + \mathbf{e}^T \\ \Leftrightarrow C_{\text{out}} \cdot \mathbf{H}_{\text{Pack}} &= \mathbf{s}^T \cdot (\mathbf{B}_{\text{Pack}} - \overline{\mathbf{X}}_{\text{out}}) + \mathbf{e}^T \cdot \mathbf{H}_{\text{Pack}} \\ &= \mathbf{s}^T \cdot (\mathbf{B}_{\text{Pack}} - \overline{\mathbf{B}}_{\text{fhe}} \cdot \mathbf{R}_C - C(x) \cdot \mathbf{G}) + \mathbf{e}^T \cdot \mathbf{H}_{\text{Pack}} \\ \Leftrightarrow C_{\text{out}} \cdot \mathbf{H}_{\text{Pack}} \cdot \mathbf{G}^{-1}(\mathbf{u}) &= \mathbf{s}^T \cdot (\mathbf{B}_{\text{Pack}} \cdot \mathbf{G}^{-1}(\mathbf{u}) - \overline{\mathbf{B}}_{\text{fhe}} \cdot \mathbf{R}_C \cdot \mathbf{G}^{-1}(\mathbf{u}) - C(x) \cdot \mathbf{u}) + \mathbf{e}^T \cdot \mathbf{H}_{\text{Pack}} \cdot \mathbf{G}^{-1}(\mathbf{u}) \end{aligned}$$

This tells us the difference between the last label entry and decoding key is once again $\mathbf{s}^T \cdot C(x) \cdot \mathbf{u}$ along with error bounded by $\|\mathbf{e}^T \cdot \mathbf{H}_{\text{Pack}} \cdot \mathbf{G}^{-1}(\mathbf{u})\| \leq B^{d_{\text{fhe}}+2} \cdot \sigma \cdot \text{poly}(\lambda)$ (as $\mathbf{H}_{\text{Pack}}, \mathbf{G}^{-1}(\mathbf{u})$ are both binary matrices). Thus, we can observe again that because Δ is superpolynomially larger than this error, our scheme is correct with overwhelming probability.

8.6 Security

Analogous to the prior section, we will prove the semantic security of [Construction 8.4](#), which by [Lemma 3.4](#) suffices to show input-hiding simulation security.

Theorem 8.5 (Semantic Security). *Assuming decomposed LWE, garbling scheme [Construction 8.4](#) satisfies semantic security as an algebraic garbling scheme.*

Proof. The security proof follows a fairly similar structure to [Theorem 7.14](#), where we incrementally remove trapdoors and sample all matrices with respect to $\mathbf{Z}, \mathbf{C}_Z$, which allows us to replace the latter with a uniform random vector through decomposed LWE. Consider the following sequence of hybrids:

- $\text{Hyb}_{\text{real}}^{(b)}$: This is the real selective semantic security experiment instantiated with [Construction 8.4](#). We highlight the instances this differs from [Theorem 7.14](#) (primarily where FHE is used).

1. The adversary $\mathcal{A}$ specifies circuit C and inputs $x_0, x_1 \in \{0, 1\}^\ell$ to the challenger.
2. The challenger samples the following:

- (a) Setup material $(\text{pp}, \mathbf{Z}) \leftarrow \text{RSetup}(1^\lambda)$.
- (b) Randomizing binary matrices:

$$\mathbf{R}_A \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m}, \mathbf{R}_B \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m \cdot k}, \mathbf{R}_{\text{inp}} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m \cdot \ell}, \mathbf{R}_{\text{fhe}} \xleftarrow{\mathbb{R}} \{0, 1\}^{m \times \ell \cdot m}, \mathbf{R}_{\text{enc}} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m}$$

- (c) Trapdoor matrix $(\mathbf{A}, \text{td}_A) \leftarrow \text{TrapGen}(1^n, 1^m, q)$.
- (d) Encoding matrices $\mathbf{B} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}$, $\mathbf{B}_{\text{inp}} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times m \cdot \ell}$, **FHE matrix** $\bar{\mathbf{B}}_{\text{fhe}} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times m}$ and decoding vector $\mathbf{u} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n$.
- (e) LWE secret and error vectors :

$$\mathbf{s} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n, \quad \mathbf{e}_0 \xleftarrow{\mathbb{R}} \mathcal{D}_{\sigma}^t, \quad \mathbf{e}_{\text{large}} \xleftarrow{\mathbb{R}} \mathcal{D}_{\sigma_{\text{large}}}^{m \cdot k}$$

$$\mathbf{e}_1^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_A, \quad \mathbf{e}_2^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_B, \quad \mathbf{e}_3^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_{\text{inp}}, \quad \mathbf{e}_{\text{fhe}}^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_{\text{enc}}.$$

- (f) Rekeying material for each $i \in [d_{\text{fhe}}]$:

$$\mathbf{B}^{(0)} \leftarrow \text{RWrite}(\text{pp}, \mathbf{B}, [\ell], \mathbf{B}_{\text{inp}}), \quad \mathbf{B}^{(i)} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}$$

$$\mathbf{B}_{C_i}^{(i-1)} \leftarrow \text{SEvalF}(\text{pp}, [\mathbf{B}^{(0)} \mid \dots \mid \mathbf{B}^{(i-1)}], C_i), \quad \text{rk}^{(i)} \leftarrow \text{SamplePre}\left([\mathbf{A} \mid \mathbf{Z}], \begin{bmatrix} \text{td}_A \\ 0 \end{bmatrix}, \mathbf{B}^{(i)} - \mathbf{B}_{C_i}^{(i-1)}, 1\right)$$

- (g) **FHE public key and ciphertext:**

$$\mathbf{B}_{\text{fhe}} = \begin{bmatrix} \bar{\mathbf{B}}_{\text{fhe}} \\ \mathbf{s}^\top \bar{\mathbf{B}}_{\text{fhe}} + \mathbf{e}_{\text{fhe}}^\top \end{bmatrix}, \quad \mathbf{X} = \mathbf{B}_{\text{fhe}} \cdot \mathbf{R}_{\text{fhe}} - x_b \otimes \mathbf{G}$$

3. The challenger computes the adversary's view as follows:

- (a) Set public data $\text{pd} = (\text{pp}, \mathbf{Z}, \mathbf{B})$.
- (b) Set garbled circuit and public garbling key $\hat{C} = \text{gk} = \{\text{rk}^{(i)}\}_{i \in [d]}$.
- (c) Set garbled input label $L_x = (\mathbf{C}_A, \mathbf{C}_Z, \mathbf{C}_B, \mathbf{C}_x, \mathbf{B}_{\text{inp}})$ where

$$\mathbf{C}_Z = \mathbf{s}^\top \cdot \mathbf{Z} + \mathbf{e}_0^\top, \quad \mathbf{C}_A = \mathbf{s}^\top \cdot \mathbf{A} + \mathbf{e}_1^\top$$

$$\mathbf{C}_B = \mathbf{s}^\top \cdot (\mathbf{B} - \text{RHash}(0) \otimes \mathbf{G}) + \mathbf{e}_2^\top, \quad \mathbf{C}_x = \mathbf{s}^\top \cdot (\mathbf{B}_{\text{inp}} - \text{Bits}(\mathbf{X}) \otimes \mathbf{G}) + \mathbf{e}_3^\top.$$

4. The adversary outputs a bit b' , which is the output of the experiment.

- $\text{Hyb}_{1,i}^{(b)}$: In this experiment, the challenger samples the rekeying keys without using td_A :

- 2.(f) Rekeying material for each $i \in [d]$:

$$\mathbf{B}^{(0)} \leftarrow \text{RWrite}(\text{pp}, \mathbf{B}, [\ell], \mathbf{B}_{\text{inp}}), \quad \text{rk}^{(i)} \xleftarrow{\mathbb{R}} \mathcal{D}_B^{(m+t) \times k \cdot m}$$

$$\mathbf{B}_{C_i}^{(i-1)} \leftarrow \text{SEvalF}(\text{pp}, [\mathbf{B}^{(0)} \mid \dots \mid \mathbf{B}^{(i-1)}], C_i), \quad \mathbf{B}^{(i)} = [\mathbf{A} \mid \mathbf{Z}] \cdot \text{rk}^{(i)} + \mathbf{B}_{C_i}^{(i-1)}$$

- $\text{Hyb}_2^{(b)}$: In this experiment, the challenger samples $\mathbf{A}$ as a random matrix.

- 2.(c) ~~Trapdoor~~ matrix $\mathbf{A} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times m}$.

- $\text{Hyb}_3^{(b)}$: In this experiment, the challenger derives matrices $\mathbf{A}, \mathbf{B}, \mathbf{B}_{\text{inp}}$ and $\mathbf{B}_{\text{fhe}}$ from $\mathbf{Z}$.
 - 2.(c) Matrix $\mathbf{A} = \mathbf{Z} \cdot \mathbf{R}_A$.
 - 2.(d) Encoding matrices $\mathbf{B} = \mathbf{Z} \cdot \mathbf{R}_B + \text{RHash}(0) \otimes \mathbf{G}, \mathbf{B}_{\text{inp}} = \mathbf{Z} \cdot \mathbf{R}_{\text{inp}} + \text{Bits}(\mathbf{X}) \otimes \mathbf{G}$, FHE matrix $\bar{\mathbf{B}}_{\text{fhe}} = \mathbf{Z} \cdot \mathbf{R}_{\text{enc}}$ and decoding vector $\mathbf{u} = \mathbf{Z} \cdot \mathbf{r}_u$ for $\mathbf{r}_u \xleftarrow{\mathbb{R}} \{0, 1\}^m$.

- $\text{Hyb}_4^{(b)}$: In this experiment, the challenger generates $\mathbf{C}_A, \mathbf{C}_B, \mathbf{C}_x$ and $\mathbf{B}_{\text{fhe}}$ from $\mathbf{C}_Z$.
 - 2.(g) FHE public key and ciphertext:

$$\mathbf{B}_{\text{fhe}} = \begin{bmatrix} \bar{\mathbf{B}}_{\text{fhe}} \\ \mathbf{C}_Z \cdot \mathbf{R}_{\text{enc}} \end{bmatrix}, \quad \mathbf{X} = \mathbf{B}_{\text{fhe}} \cdot \mathbf{R}_{\text{fhe}} - x_b \otimes \mathbf{G}$$

- 3.(c) Set garbled input label $L_x = (\mathbf{C}_A, \mathbf{C}_Z, \mathbf{C}_B, \mathbf{C}_x, \mathbf{B}_{\text{inp}})$ where

$$\begin{aligned} \mathbf{C}_Z &= \mathbf{s}^\top \cdot \mathbf{Z} + \mathbf{e}_0^\top, & \mathbf{C}_A &= \mathbf{C}_Z \cdot \mathbf{R}_A \\ \mathbf{C}_B &= \mathbf{C}_Z \cdot \mathbf{R}_B, & \mathbf{C}_x &= \mathbf{C}_Z \cdot \mathbf{R}_x. \end{aligned}$$

- $\text{Hyb}_5^{(b)}$: In this experiment, $\mathbf{C}_Z$ is replaced with a uniform random vector.

- 3.(c) Set garbled input label $L_x = (\mathbf{C}_A, \mathbf{C}_Z, \mathbf{C}_B, \mathbf{C}_x, \mathbf{B}_{\text{inp}})$ where

$$\begin{aligned} \mathbf{C}_Z &\xleftarrow{\mathbb{R}} \mathbb{Z}_q^t, & \mathbf{C}_A &= \mathbf{C}_Z \cdot \mathbf{R}_A \\ \mathbf{C}_B &= \mathbf{C}_Z \cdot \mathbf{R}_B, & \mathbf{C}_x &= \mathbf{C}_Z \cdot \mathbf{R}_x. \end{aligned}$$

- $\text{Hyb}_6^{(b)}$: In this experiment, the FHE public key $\mathbf{B}_{\text{fhe}}$ is replaced with a uniform matrix.

- 2.(d) Encoding matrices $\mathbf{B} = \mathbf{Z} \cdot \mathbf{R}_B + \text{RHash}(0) \otimes \mathbf{G}, \mathbf{B}_{\text{inp}} = \mathbf{Z} \cdot \mathbf{R}_{\text{inp}} + \text{Bits}(\mathbf{X}) \otimes \mathbf{G}$, FHE matrix $\bar{\mathbf{B}}_{\text{fhe}} = \mathbf{Z} \cdot \mathbf{R}_{\text{enc}}$ and decoding vector $\mathbf{u} = \mathbf{Z} \cdot \mathbf{r}_u$ for $\mathbf{r}_u \xleftarrow{\mathbb{R}} \{0, 1\}^m$.

- 2.(g) FHE public key and ciphertext:

$$\mathbf{B}_{\text{fhe}} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{(n+1) \times m}, \quad \mathbf{X} = \mathbf{B}_{\text{fhe}} \cdot \mathbf{R}_{\text{fhe}} - x_b \otimes \mathbf{G}$$

- $\text{Hyb}_7^{(b)}$: In this experiment, the FHE ciphertext $\mathbf{X}$ is replaced with a uniform matrix.

- 2.(g) FHE public key and ciphertext:

$$\mathbf{B}_{\text{fhe}} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{(n+1) \times m}, \quad \mathbf{X} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{(n+1) \times (m \cdot \ell)}$$

The proof of semantic security proceeds essentially identically to that of [Theorem 7.14](#), where the additional FHE public key component $\bar{\mathbf{B}}_{\text{fhe}}$ is simulated as just as matrices $\mathbf{A}, \mathbf{B}$, through $\mathbf{Z}$. The only difference is once $\mathbf{C}_Z$ is replaced with uniform in $\text{Hyb}_5^{(b)}$, we subsequently rely on LHL (with randomness $\mathbf{B}_{\text{enc}}, \mathbf{B}_{\text{fhe}}$) in $\text{Hyb}_6^{(b)}, \text{Hyb}_7^{(b)}$ to replace the FHE ciphertext with a random matrix, thus hiding the input. $\square$

We note that a similar argument as above also suffices to show this *private* garbling scheme also satisfies authenticity/one-wayness.

9 Attribute-Based Encryption

Our techniques in [Section 7](#) naturally extend to yield a key-policy attribute-based encryption scheme for circuits which enjoys succinct secret keys and NC^1 key generation. We note this in turn can be used to recover a (somewhat less direct) construction of reusable garbling via [\[GKP⁺13\]](#) in combination with any FHE and (single use) garbling scheme.

9.1 Definitions

We recall the definition of attribute-based encryption below:

Definition 9.1 (Key-Policy Attribute-Based Encryption). Let λ be a security parameter, and $\mathcal{M} = \{\mathcal{M}_\lambda\}$ be a message space, and $\mathcal{F} = \mathcal{F}_{\lambda,z}$ be a class of functions $f : \{0, 1\}^\ell \rightarrow \{0, 1\}$ for some $\ell = \ell(\lambda, z)$. Then a key-policy ABE scheme Π_{ABE} is a tuple of efficient algorithms with the following syntax.

- $\text{Setup}(1^\lambda, z) \rightarrow (\text{mpk}, \text{msk})$: On input security parameter λ and functionality index z , the setup algorithm outputs a master public key mpk and master secret key msk .
- $\text{KeyGen}(\text{msk}, f) \rightarrow \text{sk}_f$: On input master secret key msk and function $f \in \mathcal{F}$, the key generation algorithm outputs a function secret key sk_f .
- $\text{Encrypt}(\text{mpk}, x, \mu) \rightarrow \text{ct}_x$: On input master public key mpk , input $x \in \{0, 1\}^\ell$, and message $\mu \in \mathcal{M}$, the encryption algorithm outputs a ciphertext ct_x .
- $\text{Decrypt}((\text{sk}_f, f), \text{ct}_x) \rightarrow \mu \cup \perp$: On input function secret key sk_f and ciphertext ct_x , the decryption algorithm outputs message μ or $\perp$ indicating decryption failure. Since we are in the succinct setting where sk_f is potentially shorter than the description of f , the decryption algorithm will also explicitly get f .

Moreover, the algorithms should satisfy the following properties:

- **Correctness:** There exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, all z , all messages $\mu \in \mathcal{M}$, functions $f \in \mathcal{F}$ and inputs $x \in \{0, 1\}^\ell$ where $f(x) = 0$,

$$\Pr \left[\begin{array}{l} (\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, z) \\ \text{sk}_f \leftarrow \text{KeyGen}(\text{msk}, f) \\ \text{ct}_x \leftarrow \text{Encrypt}(\text{mpk}, x, \mu) \\ \mu' \leftarrow \text{Decrypt}((\text{sk}_f, f), \text{ct}_x) \end{array} \right] = \text{negl}(\lambda)$$

- **(Selective) Security:** For a bit $b \in \{0, 1\}$ and an adversary $\mathcal{A}$, define the following security game $\text{ExptABE}_{\mathcal{A}}$ between $\mathcal{A}$ and a challenger:

1. **Setup Phase:** The adversary $\mathcal{A}$ specifies an input x^* and functionality index z . The challenger computes $(\text{mpk}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, z)$ and returns mpk to the adversary.
2. **Keygen Phase:** As many times as desired, the adversary $\mathcal{A}$ submits a function $f \in \mathcal{F}$ subject to $f(x) = 1$. The challenger replies with $\text{sk}_f \leftarrow \text{KeyGen}(\text{msk}, f)$.
3. **Challenge Phase:** The adversary $\mathcal{A}$ submits two messages μ_0, μ_1 . The challenger responds with $\text{ct}^* \leftarrow \text{Encrypt}(\text{mpk}, x^*, \mu_b)$.
4. **(Post-Challenge) Keygen Phase:** The adversary $\mathcal{A}$ can once again submit functions $f \in \mathcal{F}$ subject to $f(x) = 1$ and receive $\text{sk}_f \leftarrow \text{KeyGen}(\text{msk}, f)$.
5. **Output Phase:** Adversary $\mathcal{A}$ submits a bit b' , which is the output of the experiment.

We say a scheme Π_{ABE} satisfies selective security if for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[\text{ExptABE}_{\mathcal{A}}^{(1)}(\lambda) = 1 \right] - \Pr \left[\text{ExptABE}_{\mathcal{A}}^{(0)}(\lambda) = 1 \right] \right| = \text{negl}(\lambda).$$

9.2 Construction

We present our construction of KP-ABE as follows:

Construction 9.2 (Key-Policy Attribute-Based Encryption for Circuits). Let λ be the security parameter and d be a depth bound. Our scheme relies on the same lattice ingredients as our construction of privacy-free garbling:

- A lattice gadget trapdoor sampler (TrapGen, SamplePre) per [Definition 7.6](#).
- Lattice-based homomorphic evaluation (EvalF, EvalFX) per [Definition 7.8](#).
- Key-homomorphic RAM encoding ((RSetup, RHash, RRead, RXRead, RWrite, RXWrite)) per [Definition 7.9](#).

Similar to the analysis of [Section 7](#), we will directly use the composition of lattice and RAM homomorphism (SEvalF, SEvalFX) as described in [Remark 7.13](#). We parameterize our scheme as follows:

$$n \geq \lambda \quad , \quad m = 3n \log q \quad , \quad B = \lambda^{\log(\lambda)} \quad , \quad \sigma_i = B^i \text{ for } i \in [d] \quad , \quad \sigma_{\text{out}} = B^{d+1} \quad , \quad q \geq B^{2d+2} \cdot \sigma$$

where the dLWE $_{n,m,q,2m^2\lambda,\chi,\sigma}$ is hard with respect to security parameter λ , which in turn fix parameters t, k of the RAM encoding. We can now present our scheme as follows:

- **Setup**($1^\lambda, 1^d$) : On input security parameter λ and depth parameter d , the setup algorithm proceeds as follows:
 - Sample RAM encoding setup $(\text{pp}, \mathbf{Z}) \leftarrow \text{RSetup}(1^\lambda)$.
 - Sample trapdoored matrix $(\mathbf{A}, \text{td}_{\mathbf{A}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$.
 - Sample matrix $\mathbf{B}^{(0)} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}$
 - Sample vector $\mathbf{p} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n$.
 - Output $(\text{mpk} = (\text{pp}, \mathbf{Z}, \mathbf{A}, \mathbf{B}^{(0)}, \mathbf{p}), \text{msk} = (\text{td}_{\mathbf{A}}, \text{mpk}))$.
- **KeyGen**(msk, C) : On input master secret key $\text{msk} = \text{td}_{\mathbf{A}}$ and circuit C , define circuit C_i to be the depth 1 taking in all depth $\leq i-1$ wires and outputting all depth $\leq i$ wires¹⁸, the keygen algorithm:
 - For $i \in [d]$, sample base matrices $\mathbf{B}^{(i)} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}$
 - For $i \in [d]$, set $\mathbf{B}_{C_i}^{(i-1)} \leftarrow \text{SEvalF}(\text{pp}, \mathbf{B}^{(i-1)}, C_i)$.
 - For $i \in [d]$, sample $\text{rk}^{(i)} \leftarrow \text{SamplePre}\left(\left[\mathbf{A} \mid \mathbf{Z}\right], \begin{bmatrix} \text{td}_{\mathbf{A}} \\ \mathbf{0} \end{bmatrix}, \mathbf{B}_{C_i}^{(i-1)} - \mathbf{B}^{(i)}, \sigma_i\right)$.
 - Set $\mathbf{B}_{\text{out}}^{(d)} \leftarrow \text{RRead}(\text{pp}, \mathbf{B}^{(d)}, |C|)$.
 - Sample decryption component $\mathbf{k}_C \leftarrow \text{SamplePre}\left(\left[\mathbf{A} \mid \mathbf{Z} \mid \mathbf{B}_{\text{out}}^{(d)}\right], \begin{bmatrix} \text{td}_{\mathbf{A}} \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \mathbf{p}, \sigma_{\text{out}}\right)$.
 - Return $\text{sk}_C = (\{\text{rk}^{(i)}\}_{i \in [d]}, \mathbf{k}_C, \text{mpk})$.
- **Encrypt**(mpk, x, μ) : On input master public key mpk , attribute set x and message $\mu \in \{0, 1\}$, the encryption algorithm computes the following:
 - Sample encryption randomness $\mathbf{s} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^m$.
 - Sample noise vectors $\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ as:

$$\mathbf{e}_0 \xleftarrow{\mathbb{R}} \mathcal{D}_\sigma^t \quad , \quad \mathbf{R}_{\mathbf{A}} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times m} \quad , \quad \mathbf{R}_{\mathbf{B}} \xleftarrow{\mathbb{R}} \{0, 1\}^{m+t \times m \cdot k} \quad , \quad \mathbf{R}_{\mathbf{p}} \xleftarrow{\mathbb{R}} \{0, 1\}^{t \times 1}$$

$$\mathbf{e}_1^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_{\mathbf{A}} \quad , \quad \mathbf{e}_2^\top = [\mathbf{e}_1 \mid \mathbf{e}_0^\top] \cdot \mathbf{R}_{\mathbf{B}} \quad , \quad \mathbf{e}_3^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_{\mathbf{p}}.$$
 - Set ciphertext components

$$\mathbf{C}_{\mathbf{Z}} = \mathbf{s}^\top \mathbf{Z} + \mathbf{e}_0^\top \quad , \quad \mathbf{C}_{\mathbf{A}} = \mathbf{s}^\top \mathbf{A} + \mathbf{e}_1^\top$$

$$\mathbf{C}_x = \mathbf{s}^\top (\mathbf{B} - \text{RHash}(x) \otimes \mathbf{G}) + \mathbf{e}_2^\top \quad , \quad \mathbf{C}_\mu = \mathbf{s}^\top \mathbf{p} + \mathbf{e}_3^\top + \mu \cdot \left\lceil \frac{q}{2} \right\rceil.$$
 - Output ciphertext $\text{ct} = (x, \mathbf{C}_{\mathbf{A}}, \mathbf{C}_{\mathbf{Z}}, \mathbf{C}_x, \mathbf{C}_\mu)$.

¹⁸Note this departs slightly different from the notation in prior sections, where C_i only computes depth exactly i wires

- $\text{Decrypt}(C, \text{sk}_C, \text{ct}_x)$: On input secret key sk and ciphertext ct , the decryption algorithm Decrypt
 - Parse $\text{sk}_C = (\{\text{rk}^{(i)}\}_{i \in [d]}, \mathbf{k}_C, \text{mpk})$ and $\text{ct} = (x, \mathbf{C}_A, \mathbf{C}_Z, \mathbf{C}_x, \mathbf{C}_\mu)$ where $\text{mpk} = (\text{pp}, \mathbf{Z}, \mathbf{A}, \mathbf{B}^{(0)}, \mathbf{p})$.
 - Set $h^{(0)} = \text{RHash}(x)$ and $\mathbf{C}^{(0)} = \mathbf{C}_x$.
 - For $i \in [d]$, let $x^{(i)}$ be all of the layer $\leq i$ wire values of $C(x)$ and let $h^{(i)} = \text{RHash}(x^{(i)})$. Compute $(\mathbf{B}^{(i)}, \mathbf{C}^{(i)})$ as follows:

- * Compute homomorphic evaluation

$$\mathbf{B}_{C_i}^{(i-1)} = \text{SEvalF}(\text{pp}, \mathbf{B}^{(i-1)}, C_i) \quad , \quad \mathbf{C}_{C_i}^{(i-1)} = [\mathbf{C}^{(i-1)} \mid \mathbf{C}_Z] \cdot \mathbf{H}_{C_i}$$

where $\mathbf{H}_{C_i} \leftarrow \text{SEvalFX}(\text{pp}, \mathbf{B}^{(i-1)}, C_i, x^{(i)})$.

- * Apply rekey

$$\mathbf{B}^{(i)} = \mathbf{B}_{C_i}^{(i-1)} + [\mathbf{A} \mid \mathbf{Z}] \cdot \text{rk}^{(i)} \quad , \quad \mathbf{C}^{(i)} = \mathbf{C}_{C_i}^{(i-1)} + [\mathbf{C}_A \mid \mathbf{C}_Z] \cdot \text{rk}^{(i)}.$$

- Compute $\mathbf{C}_{\text{out}} = \mathbf{C}^{(d)} \cdot \mathbf{H}_{\text{out}}^{(d)}$ where $\mathbf{H}_{\text{out}}^{(d)} = \text{RXRead}(\text{pp}, \mathbf{B}^{(d)}, |C|, x^{(d)})$.
- Return message as $[\mathbf{C}_\mu - [\mathbf{C}_A \mid \mathbf{C}_Z \mid \mathbf{C}_{\text{out}}] \cdot \mathbf{k}_C]_2$.

9.3 Key Succinctness and Generation Depth

From the succinctness of [Definition 7.9](#) RAM encodings, we can directly observe that the dimensions of our secret keys along with our modulus depend only on the depth of the circuit C on not its size.

The analysis of KeyGen depth follows from an essentially identical argument to that of [Construction 7.12](#), as they utilize the same underlying lattice ingredients and basic structure.

9.4 Correctness

Theorem 9.3 (Correctness). *[Construction 9.2](#) is a correct attribute-based encryption scheme.*

Proof. The argument follows analogous to [Section 7.5](#), where we inductively bound the distance $\|\mathbf{C}^{(i)} - \mathbf{s}^\top \cdot (\mathbf{B}^{(i)} - h^{(i)} \otimes \mathbf{G})\|$, which can in turn be used to bound the difference

$$\left(\mathbf{C}_\mu - \mu \cdot \left\lceil \frac{q}{2} \right\rceil \right) - ([\mathbf{C}_A \mid \mathbf{C}_Z \mid \mathbf{C}_{\text{out}}] \cdot \mathbf{k}_C)$$

Since decryption simply rounds $\mathbf{C}_\mu - [\mathbf{C}_A \mid \mathbf{C}_Z \mid \mathbf{C}_{\text{out}}] \cdot \mathbf{k}_C$ to $\mathbb{Z}_2$, it suffices to bound the above error to be $< (\frac{q}{2})$.

In this case, we will inductively argue that for $i \in [d]$

$$\|\mathbf{C}^{(i)} - \mathbf{s}^\top \cdot (\mathbf{B}^{(i)} - h^{(i)} \otimes \mathbf{G})\| \leq \sigma \cdot B^{i+2}$$

Our base case, for $\mathbf{C}^{(0)} = \mathbf{C}_x$, follows from construction, as $\mathbf{C}_x$ is computed as

$$\mathbf{s}^\top (\mathbf{B} - h^{(0)} \otimes \mathbf{G}) + \mathbf{e}_0^\top \cdot \mathbf{R}_B.$$

Since $\|\mathbf{e}_0\| \leq \sigma$ and $\|\mathbf{R}_B\| \leq 1$, we can bound the total difference

$$\|\mathbf{C}^{(0)} - \mathbf{s}^\top \cdot (\mathbf{B}^{(0)} - h^{(0)} \otimes \mathbf{G})\| \leq \sigma \cdot \text{poly}(\lambda) \leq \sigma \cdot B$$

Proceeding onto our inductive step, setting

$$\mathbf{C}^{(i-1)} = \mathbf{s}^\top \cdot (\mathbf{B}^{(i-1)} - h^{(i-1)} \otimes \mathbf{G}) + \mathbf{e}_{(i-1)}^\top$$

for some bounded error $\|\mathbf{e}_{(i-1)}\| \leq \sigma \cdot B^{i+1}$, we can write

$$\begin{aligned} \mathbf{C}_{C_i}^{(i-1)} &= [\mathbf{C}^{(i-1)} \mid \mathbf{C}_Z] \cdot \mathbf{H}_{C_i} = \mathbf{s}^\top [(\mathbf{B}^{(i-1)} - h^{(i-1)} \otimes \mathbf{G}) \mid \mathbf{Z}] \cdot \mathbf{H}_{C_i} + [\mathbf{e}_{(i-1)}^\top \mid \mathbf{e}_0^\top] \cdot \mathbf{H}_{C_i} \\ &= \mathbf{s}^\top \cdot (\mathbf{B}_{C_i}^{(i-1)} - h^{(i)} \otimes \mathbf{G}) + \mathbf{e}_{(i-1, C_i)}^\top \end{aligned}$$

where $\mathbf{e}_{(i-1, C_i)}^\top = [\mathbf{e}_{(i-1)}^\top \mid \mathbf{e}_0^\top] \cdot \mathbf{H}_{C_i}$, which then gives

$$\begin{aligned} \mathbf{C}^{(i)} &= \mathbf{C}_{C_i}^{(i-1)} + [\mathbf{C}_A \mid \mathbf{C}_Z] \cdot \mathbf{rk}^{(i)} \\ &= \mathbf{s}^\top \cdot (\mathbf{B}_{C_i}^{(i-1)} - h^{(i)} \otimes \mathbf{G}) + \mathbf{e}_{(i-1, C_i)}^\top + (\mathbf{s}^\top \cdot [\mathbf{A} \mid \mathbf{Z}] + [\mathbf{e}_1^\top \mid \mathbf{e}_0^\top]) \cdot \mathbf{rk}^{(i)} \\ &= \mathbf{s}^\top \cdot \left(\mathbf{B}_{C_i}^{(i-1)} - h^{(i)} \otimes \mathbf{G} + \left(\mathbf{B}^{(i)} - \mathbf{B}_{C_i}^{(i-1)} \right) \right) + \left(\mathbf{e}_{(i-1, C_i)}^\top + [\mathbf{e}_1^\top \mid \mathbf{e}_0^\top] \cdot \mathbf{rk}^{(i)} \right) \end{aligned}$$

Observe that we can bound $\|\mathbf{H}_{C_i}\| \leq \text{poly}(\lambda)$, $\|\mathbf{e}_0\|, \|\mathbf{e}_1\| \leq \sigma \cdot \text{poly}(\lambda)$, and $\|\mathbf{rk}^{(i)}\| \leq \sigma_i \cdot B$. Combining this with the inductive hypothesis gives

$$\left\| \left(\mathbf{e}_{(i-1, C_i)}^\top + [\mathbf{e}_1^\top \mid \mathbf{e}_0^\top] \cdot \mathbf{rk}^{(i)} \right) \right\| \leq (\sigma_i \cdot B + \sigma \cdot B^{i+1}) \cdot \text{poly}(\lambda) \leq (\sigma \cdot B^{i+2}).$$

Which is sufficient to complete our induction, which lets us bound

$$\|\mathbf{C}^{(i)} - \mathbf{s}^\top \cdot (\mathbf{B}^{(i)} - h^{(i)} \otimes \mathbf{G})\| \leq \sigma \cdot B^{d+2}$$

Now, to bound the final error on $[\mathbf{C}_A \mid \mathbf{C}_Z \mid \mathbf{C}_{\text{out}}] \cdot \mathbf{k}_C$, it remains is to apply the read homomorphism on the output wire $\mathbf{H}_{\text{out}}^{(d)}$ and the decryption component $\mathbf{k}_C$, which follow from similar arguments. The error growth here is dominated by $\|\mathbf{k}_C\| \leq \sigma_{\text{out}} = B^{d+1}$, giving us a final error of $\sigma \cdot B^{d+2} \cdot B^{d+1} \cdot \text{poly}(\lambda)$. Since we chose our modulus to superpolynomially larger $\geq \sigma \cdot B^{2d+2}$, correctness holds with overwhelming probability. $\square$

9.5 Security

Theorem 9.4 (Selective Security). *Assuming decomposed LWE, [Construction 9.2](#) satisfies selective security as an attribute-based encryption scheme.*

Proof. Consider the following series of hybrids:

- $\text{Hyb}_{\text{real}}^{(b)}$: This is the real (selective) security game instantiated with [Construction 9.2](#).
- 1. **Setup Phase:** The adversary $\mathcal{A}$ specifies an input x^* and bound on circuit depth. The challenger samples the following:

(a) Sample RAM encoding setup $(\text{pp}, \mathbf{Z}) \leftarrow \text{RSetup}(1^\lambda)$.

(b) Sample noise vectors $\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ as:

$$\mathbf{e}_0 \xleftarrow{\mathcal{R}} \mathcal{D}_\sigma^t, \quad \mathbf{R}_A \xleftarrow{\mathcal{R}} \{0, 1\}^{t \times m}, \quad \mathbf{R}_B \xleftarrow{\mathcal{R}} \{0, 1\}^{m+t \times m \cdot k}, \quad \mathbf{R}_p \xleftarrow{\mathcal{R}} \{0, 1\}^{t \times 1}$$

$$\mathbf{e}_1^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_A, \quad \mathbf{e}_2^\top = [\mathbf{e}_1 \mid \mathbf{e}_0^\top] \cdot \mathbf{R}_B, \quad \mathbf{e}_3^\top = \mathbf{e}_0^\top \cdot \mathbf{R}_p.$$

(c) Sample trapdoored matrix $(\mathbf{A}, \text{td}_A) \leftarrow \text{TrapGen}(1^n, 1^m, q)$.

(d) Sample attribute encoding matrix $\mathbf{B}^{(0)} \xleftarrow{\mathcal{R}} \mathbb{Z}_q^{n \times k \cdot m}$ and message encoding vector $\mathbf{p} \xleftarrow{\mathcal{R}} \mathbb{Z}_q^n$.

(e) Set ciphertext components

$$\mathbf{C}_Z = \mathbf{s}^\top \mathbf{Z} + \mathbf{e}_0^\top, \quad \mathbf{C}_A = \mathbf{s}^\top \mathbf{A} + \mathbf{e}_1^\top$$

$$\mathbf{C}_x = \mathbf{s}^\top (\mathbf{B} - \text{RHash}(x) \otimes \mathbf{G}) + \mathbf{e}_2^\top, \quad \mathbf{C}_\mu = \mathbf{s}^\top \mathbf{p} + \mathbf{e}_3^\top + b \cdot \left\lceil \frac{q}{2} \right\rceil.$$

(f) The challenger returns $(\text{pp}, \mathbf{Z}, \mathbf{A}, \mathbf{B}^{(0)}, \mathbf{p})$ to the adversary.

2. **Keygen Phase:** As many times as desired, adversary $\mathcal{A}$ submits a circuit C of depth d where $C(x^*) = 1$. The challenger responds with as follows:

- (a) For $i \in [d]$, sample base matrices $\mathbf{B}^{(i)} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}$
- (b) For $i \in [d]$, set $\mathbf{B}_{C_i}^{(i-1)} \leftarrow \text{SEvalF}(\text{pp}, \mathbf{B}^{(i-1)}, C_i)$.
- (c) For $i \in [d]$, sample $\text{rk}^{(i)} \leftarrow \text{SamplePre}\left([A \mid Z], \begin{bmatrix} \text{td}_A \\ \mathbf{0} \end{bmatrix}, \mathbf{B}_{C_i}^{(i-1)} - \mathbf{B}^{(i)}, \sigma_i\right)$.
- (d) Set $\mathbf{B}_{\text{out}}^{(d)} \leftarrow \text{RRead}(\text{pp}, \mathbf{B}^{(d)}, |C|)$.
- (e) Sample decryption component $\mathbf{k}_C \leftarrow \text{SamplePre}\left([A \mid Z \mid \mathbf{B}_{\text{out}}^{(d)}], \begin{bmatrix} \text{td}_A \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \mathbf{p}, \sigma_{\text{out}}\right)$.
- (f) The challenger returns $\text{sk}_C = (\{\text{rk}^{(i)}\}_{i \in [d]}, \mathbf{k}_C, \text{mpk})$

3. **Challenge Phase:**¹⁹ The challenger sends ciphertext $(x^*, C_Z, C_A, C_x, C_\mu)$.

4. **(Post-Challenge) Keygen Phase:** Same as pre-challenge

5. **Output Phase:** Adversary $\mathcal{A}$ submits a bit b' , which is the output of the experiment.

• $\text{Hyb}_1^{(b)}$: In this experiment, the challenger sets

- 1.(d) Sample attribute encoding matrix $\mathbf{B}^{(0)} = [A \mid Z] \cdot \mathbf{R}_B + \text{RHash}(x^*) \otimes \mathbf{G}$ and message encoding vector $\mathbf{p} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n$.

• $\text{Hyb}_{2,i^*}^{(b)}$: For $i^* \in [d]$, in this experiment, for all keygen queries, the challenger samples

- 2.(a) For $i \in [d]$ where $i \leq i^*$ sample $\mathbf{R}^{(i)} \xleftarrow{\mathbb{R}} \mathcal{D}_{\sigma_i \cdot \mathbf{B}}^{(m+t) \times m}$ and set $\mathbf{B}^{(i)} = [A \mid Z] \cdot \mathbf{R}^{(i)} + h^{(i)} \otimes \mathbf{G}$. Otherwise, sample $\mathbf{B}^{(i)} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times k \cdot m}$.
- 2.(c) For $i \in [d]$ where $i \leq i^*$ set $\text{rk}^{(i)} = \mathbf{R}^{(i-1)} \cdot \mathbf{H}_{C_i} + \mathbf{R}^{(i)}$. Otherwise, sample

$$\text{rk}^{(i)} \leftarrow \text{SamplePre}\left([A \mid Z], \begin{bmatrix} \text{td}_A \\ \mathbf{0} \end{bmatrix}, \mathbf{B}_{C_i}^{(i-1)} - \mathbf{B}^{(i)}, \sigma_i\right).$$

where $\mathbf{R}^{(0)} = \mathbf{R}_B$.

• $\text{Hyb}_3^{(b)}$: In this experiment, for all keygen queries, the challenger samples

- 2.(e) Sample decryption component

$$\mathbf{k}_C \leftarrow \text{SamplePre}\left([A \mid Z \mid \mathbf{B}_{\text{out}}^{(d)}], \begin{bmatrix} -\mathbf{R}_{\text{out}}^{(d)} \\ \mathbf{I}_m \end{bmatrix}, \mathbf{p}, \sigma_{\text{out}}\right)$$

where

$$\mathbf{R}_{\text{out}}^{(d)} = \left[\mathbf{R}^{(d)} \mid \begin{bmatrix} \mathbf{0} \\ \mathbf{I}_t \end{bmatrix} \right] \cdot \mathbf{H}_{\text{out}}^{(d)}$$

(recall from our decryption relation $\mathbf{H}_{\text{out}}^{(d)} \leftarrow \text{RXRead}(\text{pp}, \mathbf{B}^{(d)}, |C|, x^{(d)})$).

• $\text{Hyb}_4^{(b)}$: In this experiment, the challenger samples $\mathbf{A}$ uniformly.

- 1.(c) Sample ~~trapdoor~~ matrix $\mathbf{A} \xleftarrow{\mathbb{R}} \mathbb{Z}_q^{n \times m}$.

• $\text{Hyb}_5^{(b)}$: In this experiment, the challenger computes $\mathbf{A}, \mathbf{p}$ from $\mathbf{Z}$.

¹⁹Since we present our scheme as a bit-encryption, the adversary is simply tasked with distinguishing encryptions of 0 from 1

1.(c) Sample matrix $\mathbf{A} = \mathbf{Z} \cdot \mathbf{R}_A$.

1.(d) Sample attribute encoding matrix $\mathbf{B}^{(0)} = [\mathbf{A} \mid \mathbf{Z}] \cdot \mathbf{R}_B + \text{RHash}(x^*) \otimes \mathbf{G}$ and message encoding vector $\mathbf{p} = \mathbf{Z} \cdot \mathbf{R}_p$.

- $\text{Hyb}_6^{(b)}$: In this experiment, the challenger computes $\mathbf{C}_A, \mathbf{C}_x, \mathbf{C}_\mu$ from $\mathbf{C}_Z$.

1.(e) Set ciphertext components

$$\mathbf{C}_Z = \mathbf{s}^\top \mathbf{Z} + \mathbf{e}_0^\top, \quad \mathbf{C}_A = \mathbf{C}_Z \cdot \mathbf{R}_A$$

$$\mathbf{C}_x = [\mathbf{C}_A \mid \mathbf{C}_Z] \cdot \mathbf{R}_B + \mathbf{e}_2^\top, \quad \mathbf{C}_\mu = \mathbf{C}_Z \cdot \mathbf{R}_p + \mu \cdot \left\lceil \frac{q}{2} \right\rceil.$$

- $\text{Hyb}_7^{(b)}$: In this experiment, the challenger samples $\mathbf{C}_Z \xleftarrow{\mathbb{R}} \mathbb{Z}_q^t$.

1.(e) Set ciphertext components

$$\mathbf{C}_Z \xleftarrow{\mathbb{R}} \mathbb{Z}_q^t, \quad \mathbf{C}_A = \mathbf{C}_Z \cdot \mathbf{R}_A$$

$$\mathbf{C}_x = [\mathbf{C}_A \mid \mathbf{C}_Z] \cdot \mathbf{R}_B + \mathbf{e}_2^\top, \quad \mathbf{C}_\mu = \mathbf{C}_Z \cdot \mathbf{R}_p + \mu \cdot \left\lceil \frac{q}{2} \right\rceil.$$

- $\text{Hyb}_8^{(b)}$: In this experiment, the challenger samples $\mathbf{C}_\mu \xleftarrow{\mathbb{R}} \mathbb{Z}_q$.

1.(e) Set ciphertext components

$$\mathbf{C}_Z \xleftarrow{\mathbb{R}} \mathbb{Z}_q^t, \quad \mathbf{C}_A = \mathbf{C}_Z \cdot \mathbf{R}_A$$

$$\mathbf{C}_x = [\mathbf{C}_A \mid \mathbf{C}_Z] \cdot \mathbf{R}_B + \mathbf{e}_2^\top, \quad \mathbf{C}_\mu \xleftarrow{\mathbb{R}} \mathbb{Z}_q.$$

Lemma 9.5. For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that

$$\left| \Pr[\text{Hyb}_{\text{real}}^{(b)}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_1^{(b)}(\mathcal{A}, \lambda) = 1] \right| = \text{negl}(\lambda).$$

Proof. This follows from a straightforward application of the leftover hash lemma. □

Lemma 9.6. For all adversaries $\mathcal{A}$,

$$\Pr[\text{Hyb}_1^{(b)}(\mathcal{A}, \lambda) = 1] = \Pr[\text{Hyb}_{2,0}^{(b)}(\mathcal{A}, \lambda) = 1].$$

Proof. These hybrids are identical. □

Lemma 9.7. For all adversaries $\mathcal{A}$ and $i \in [d]$, there exists a negligible function $\text{negl}(\lambda)$ such that

$$\left| \Pr[\text{Hyb}_{2,i-1}^{(b)}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_{2,i}^{(b)}(\mathcal{A}, \lambda) = 1] \right| = \text{negl}(\lambda).$$

Proof. We can further break this hybrid transition down into a series of subhybrids:

- sHyb_1 : In this experiment, we explicitly sample the difference $(\mathbf{B}^{(i)} - \mathbf{B}_{C_i}^{(i-1)})$ as $\mathbf{T}^{(i)} \leftarrow \mathbb{Z}_q^{n \times k \cdot m}$ and set $\mathbf{B}^{(i)} = \mathbf{T}^{(i)} + \mathbf{B}_{C_i}^{(i-1)}$. Since $\mathbf{B}^{(i)}$ is sampled uniformly in $\text{Hyb}_{2,i-1}$, these distributions are identical.
- sHyb_2 : In this experiment, we sample $\text{rk}^{(i)} \xleftarrow{\mathbb{R}} \mathcal{D}_{B \cdot \sigma_i}^{(m+t) \times (k \cdot m)}$ and set $\mathbf{T}^{(i)} = [\mathbf{A} \mid \mathbf{Z}] \cdot \text{rk}^{(i)}$. This follows from the preimage-distribution property of the gadget trapdoor.
- sHyb_3 : In this experiment, we sample $\mathbf{R}^{(i)} \xleftarrow{\mathbb{R}} \mathcal{D}_{B \cdot \sigma_i}^{(m+t) \times (k \cdot m)}$ and set $\text{rk}^{(i)} = \mathbf{R}^{(i-1)} \cdot \mathbf{H}_{C_i} + \mathbf{R}^{(i)}$. Since $B \cdot \sigma_i \gg \|\mathbf{R}^{(i-1)} \cdot \mathbf{H}_{C_i}\| = B \cdot \sigma_{i-1} \cdot \text{poly}(\lambda)$, this follows by noise smudging (recall as a base case $\sigma_0 = 1$, corresponding to the binary $\mathbf{R}^{(0)} = \mathbf{R}_B$).

Since each KeyGen query samples its own set of $\mathbf{B}^{(i)}, \mathbf{rk}^{(i)}$, we iteratively apply this series of subhybrids to the output of every KeyGen query. $\square$

Lemma 9.8. *For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$\left| \Pr[\text{Hyb}_{2,d}^{(b)}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_3^{(b)}(\mathcal{A}, \lambda) = 1] \right| = \text{negl}(\lambda).$$

Proof. Recall by $\text{Hyb}_{2,d}$, we compute

$$\mathbf{B}^{(d)} = [\mathbf{A} \mid \mathbf{Z}] \cdot \mathbf{R}^{(d)} + h^{(d)} \otimes \mathbf{G}$$

Thus, by the homomorphism of the RAM encoding we have

$$\begin{aligned} & [\mathbf{B}^{(d)} - h^{(d)} \otimes \mathbf{G} \mid \mathbf{Z}] \cdot \mathbf{H}_{\text{out}} = \mathbf{B}_{\text{out}}^{(d)} - 1 \cdot \mathbf{G} \\ \Leftrightarrow & [[\mathbf{A} \mid \mathbf{Z}] \cdot \mathbf{R}^{(d)} \mid \mathbf{Z}] \cdot \mathbf{H}_{\text{out}} = \mathbf{B}_{\text{out}}^{(d)} - \mathbf{G} \\ \Leftrightarrow & [\mathbf{A} \mid \mathbf{Z}] \cdot \left[\mathbf{R}^{(d)} \mid \begin{bmatrix} \mathbf{0} \\ \mathbf{I}_t \end{bmatrix} \right] \cdot \mathbf{H}_{\text{out}} - \mathbf{B}_{\text{out}}^{(d)} = -\mathbf{G} \\ & [\mathbf{A} \mid \mathbf{Z} \mid \mathbf{B}_{\text{out}}^{(d)}] \cdot \begin{bmatrix} -\mathbf{R}_{\text{out}}^{(d)} \\ \mathbf{I}_m \end{bmatrix} = \mathbf{G} \end{aligned}$$

Since $\begin{bmatrix} -\mathbf{R}_{\text{out}}^{(d)} \\ \mathbf{I}_m \end{bmatrix}$ is indeed a gadget trapdoor with entries bounded by $\|\mathbf{R}^{(d)}\| \cdot \|\mathbf{H}_{\text{out}}\| \cdot \text{poly}(\lambda) \ll \|\mathbf{R}^{(d)}\| \cdot B = \sigma_{\text{out}}$, the distribution of $\mathbf{k}_C$ is both $\text{Hyb}_{2,d}$, Hyb_3 are both uniform preimages drawn from $\mathcal{D}_{\sigma_{\text{out}} \cdot B}$ of a random $\mathbf{p}$ and thus identically distributed. As with [Lemma 9.7](#), we iteratively apply this argument to the $\mathbf{k}_C$ output by every KeyGen query. $\square$

Lemma 9.9. *For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$\left| \Pr[\text{Hyb}_3^{(b)}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_4^{(b)}(\mathcal{A}, \lambda) = 1] \right| = \text{negl}(\lambda).$$

Proof. Now that td_A is no longer being used, so by trapgen uniformity the distribution of $\mathbf{A}$ is uniform. $\square$

Lemma 9.10. *For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$\left| \Pr[\text{Hyb}_4^{(b)}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_5^{(b)}(\mathcal{A}, \lambda) = 1] \right| = \text{negl}(\lambda).$$

Proof. This follows from a straightforward application of the leftover hash lemma (via the LHL compatibility of [Definition 7.9](#)). $\square$

Lemma 9.11. *For all adversaries $\mathcal{A}$,*

$$\Pr[\text{Hyb}_5^{(b)}(\mathcal{A}, \lambda) = 1] = \Pr[\text{Hyb}_6^{(b)}(\mathcal{A}, \lambda) = 1].$$

Proof. Observe the differences between these hybrids are entirely syntactic, and can be rewritten (See [Lemma 7.20](#)). $\square$

Lemma 9.12. *Assuming decomposed LWE, for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$\left| \Pr[\text{Hyb}_6^{(b)}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_7^{(b)}(\mathcal{A}, \lambda) = 1] \right| = \text{negl}(\lambda).$$

Proof. Specifically, this will follow from the LWE indistinguishability property of the key-homomorphic RAM encoding, where

$$\{\text{pp}, \mathbf{Z}, \mathbf{s}^\top \cdot \mathbf{Z} + \mathbf{e}^\top\} \approx \{\text{pp}, \mathbf{Z}, \mathbf{t}^\top\}.$$

Observe at by $\text{Hyb}_6^{(b)}$, all the elements in the view of the adversary dependent on $\mathbf{s}$ can be derived from $\mathbf{C}_Z = \mathbf{s}^\top \cdot \mathbf{Z} + \mathbf{e}_0^\top$. This can be directly replaced with the (decomposed) LWE challenge vector. $\square$

Lemma 9.13. *For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$\left| \Pr[\text{Hyb}_7^{(b)}(\mathcal{A}, \lambda) = 1] - \Pr[\text{Hyb}_8^{(b)}(\mathcal{A}, \lambda) = 1] \right| = \text{negl}(\lambda).$$

Proof. Since C_Z is now a uniformly random vector,

$$(C_Z \cdot R_p, e_0^\top \cdot R_p) \approx_s (r \xleftarrow{\mathbb{R}} \mathbb{Z}_q^n, e_0^\top \cdot R_p)$$

follows from the leftover hash lemma. □

Lemma 9.14. *For all adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that*

$$\Pr[\text{Hyb}_8^{(0)}(\mathcal{A}, \lambda) = 1] = \Pr[\text{Hyb}_8^{(1)}(\mathcal{A}, \lambda) = 1].$$

Proof. Since C_p is uniformly random, the adversary's view is now independent of the challenge bit b . □

Taking together [Lemmas 9.5](#) to [9.14](#) gives us the proof of [Theorem 9.4](#) □

10 Acknowledgments

Huijia Lin is supported by a Simons collaboration grant for algorithmic fairness and Amazon Research Award. We thank Lawrence Roy for pointing out an error the group-based garbling schemes in an earlier version of this work.

References

- [ABB10] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 553–572, French Riviera, May 30 – June 3, 2010. Springer Berlin Heidelberg, Germany.
- [ADOS22] Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 421–452, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [AIK04] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. Cryptography in NC^0 . In *45th FOCS*, pages 166–175, Rome, Italy, October 17–19, 2004. IEEE Computer Society Press.
- [AIK06] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. Computationally private randomizing polynomials and their applications. *computational complexity*, 15(2):115–162, 2006.
- [AIK10] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. From secrecy to soundness: Efficient verification via secure computation. In Samson Abramsky, Cyril Gavoille, Claude Kirchner, Friedhelm Meyer auf der Heide, and Paul G. Spirakis, editors, *ICALP 2010, Part I*, volume 6198 of *LNCS*, pages 152–163, Bordeaux, France, July 6–10, 2010. Springer Berlin Heidelberg, Germany.
- [AJL⁺12] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 483–501, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.
- [Ajt96] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *28th ACM STOC*, pages 99–108, Philadelphia, PA, USA, May 22–24, 1996. ACM Press.

- [AMR25] Damiano Abram, Giulio Malavolta, and Lawrence Roy. Key-homomorphic computations for ram: Fully succinct randomised encodings and more. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, 2025.
- [App17] Benny Applebaum. Garbled circuits as randomized encodings of functions: a primer. In *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich*, pages 1–44. Springer, 2017.
- [BCG⁺18] Nir Bitansky, Ran Canetti, Sanjam Garg, Justin Holmgren, Abhishek Jain, Huijia Lin, Rafael Pass, Sidharth Telang, and Vinod Vaikuntanathan. Indistinguishability obfuscation for ram programs and succinct randomized encodings. *SIAM Journal on Computing*, 47(3):1123–1210, 2018.
- [BGG⁺14] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 533–556, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 509–539, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.
- [BGI17] Elette Boyle, Niv Gilboa, and Yuval Ishai. Group-based secure computation: Optimizing rounds, communication, and computation. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part II*, volume 10211 of *LNCS*, pages 163–193, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland.
- [BHHO08] Dan Boneh, Shai Halevi, Michael Hamburg, and Rafail Ostrovsky. Circular-secure encryption from decision Diffie-Hellman. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 108–125, Santa Barbara, CA, USA, August 17–21, 2008. Springer Berlin Heidelberg, Germany.
- [BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 784–796, Raleigh, NC, USA, October 16–18, 2012. ACM Press.
- [BKS19] Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without FHE. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 3–33, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.
- [BL18] Fabrice Benhamouda and Huijia Lin. k-round multiparty computation from k-round oblivious transfer via garbled interactive circuits. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 500–532, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.
- [BLLL23] Marshall Ball, Hanjun Li, Huijia Lin, and Tianren Liu. New ways to garble arithmetic circuits. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 3–34, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [BLMR13] Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 410–428, Santa Barbara, CA, USA, August 18–22, 2013. Springer Berlin Heidelberg, Germany.
- [BMR90] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd ACM STOC*, pages 503–513, Baltimore, MD, USA, May 14–16, 1990. ACM Press.

- [BTWV17] Zvika Brakerski, Rotem Tsabary, Vinod Vaikuntanathan, and Hoeteck Wee. Private constrained PRFs (and more) from LWE. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 264–302, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland.
- [CDL01] Andrew Chiu, George Davida, and Bruce Litow. Division in logspace-uniform nc1. *RAIRO-Theoretical Informatics and Applications*, 35(3):259–275, 2001.
- [DHRW16] Yevgeniy Dodis, Shai Halevi, Ron D. Rothblum, and Daniel Wichs. Spooky encryption and its applications. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 93–122, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.
- [DJ01] Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier’s probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001*, volume 1992 of *LNCS*, pages 119–136, Cheju Island, South Korea, February 13–15, 2001. Springer Berlin Heidelberg, Germany.
- [DKK18] Itai Dinur, Nathan Keller, and Ohad Klein. An optimal distributed discrete log protocol with applications to homomorphic secret sharing. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 213–242, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [FNO15] Tore Kasper Frederiksen, Jesper Buus Nielsen, and Claudio Orlandi. Privacy-free garbled circuits with applications to efficient zero-knowledge. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 191–219, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.
- [GGP10] Rosario Gennaro, Craig Gentry, and Bryan Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 465–482, Santa Barbara, CA, USA, August 15–19, 2010. Springer Berlin Heidelberg, Germany.
- [GKP⁺13] Shafi Goldwasser, Yael Tauman Kalai, Raluca A. Popa, Vinod Vaikuntanathan, and Nickolai Zeldovich. Reusable garbled circuits and succinct functional encryption. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 555–564, Palo Alto, CA, USA, June 1–4, 2013. ACM Press.
- [GLNP15] Shay Gueron, Yehuda Lindell, Ariel Nof, and Benny Pinkas. Fast garbling of circuits under standard assumptions. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 567–578, Denver, CO, USA, October 12–16, 2015. ACM Press.
- [Gol00] Oded Goldreich. Candidate one-way functions based on expander graphs. In *Elect. Colloq. on Computational Complexity (ECCC)*, volume 7, 2000.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206, Victoria, BC, Canada, May 17–20, 2008. ACM Press.
- [GS17] Sanjam Garg and Akshayaram Srinivasan. Garbled protocols and two-round MPC from bilinear maps. In Chris Umans, editor, *58th FOCS*, pages 588–599, Berkeley, CA, USA, October 15–17, 2017. IEEE Computer Society Press.
- [GS18] Sanjam Garg and Akshayaram Srinivasan. Two-round multiparty secure computation from minimal assumptions. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 468–499, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.
- [GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 75–92, Santa Barbara, CA, USA, August 18–22, 2013. Springer Berlin Heidelberg, Germany.

- [GVW15] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Predicate encryption for circuits from LWE. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 503–523, Santa Barbara, CA, USA, August 16–20, 2015. Springer Berlin Heidelberg, Germany.
- [HK21] David Heath and Vladimir Kolesnikov. One hot garbling. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 574–593, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.
- [HKO22] David Heath, Vladimir Kolesnikov, and Rafail Ostrovsky. EpiGRAM: Practical garbled RAM. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 3–33, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [HLL23] Yao-Ching Hsieh, Huijia Lin, and Ji Luo. Attribute-based encryption for circuits of unbounded depth from lattices. In *64th FOCS*, pages 415–434, Santa Cruz, CA, USA, November 6–9, 2023. IEEE Computer Society Press.
- [IK02] Yuval Ishai and Eyal Kushilevitz. Perfect constant-round secure computation via perfect randomizing polynomials. In Peter Widmayer, Francisco Triguero Ruiz, Rafael Morales Bueno, Matthew Hennessy, Stephan Eidenbenz, and Ricardo Conejo, editors, *ICALP 2002*, volume 2380 of *LNCS*, pages 244–256, Malaga, Spain, July 8–13, 2002. Springer Berlin Heidelberg, Germany.
- [ILL25a] Yuval Ishai, Hanjun Li, and Huijia Lin. Succinct homomorphic macs from groups and applications. In Ran Raz and Rotem Oshman, editors, *In 66th FOCS*, 2025.
- [ILL25b] Yuval Ishai, Hanjun Li, and Huijia Lin. A unified framework for succinct garbling from homomorphic secret sharing. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, 2025.
- [ILL25c] Yuval Ishai, Hanjun Li, and Huijia Lin. A unified framework for succinct garbling from homomorphic secret sharing. Cryptology ePrint Archive, Report 2025/442, 2025.
- [IPS08] Yuval Ishai, Manoj Prabhakaran, and Amit Sahai. Founding cryptography on oblivious transfer - efficiently. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 572–591, Santa Barbara, CA, USA, August 17–21, 2008. Springer Berlin Heidelberg, Germany.
- [IW14] Yuval Ishai and Hoeteck Wee. Partial garbling schemes and their applications. In Javier Esparza, Pierre Fraigniaud, Thore Husfeldt, and Elias Koutsoupias, editors, *ICALP 2014, Part I*, volume 8572 of *LNCS*, pages 650–662, Copenhagen, Denmark, July 8–11, 2014. Springer Berlin Heidelberg, Germany.
- [JKO13] Marek Jawurek, Florian Kerschbaum, and Claudio Orlandi. Zero-knowledge using garbled circuits: how to prove non-algebraic statements efficiently. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 955–966, Berlin, Germany, November 4–8, 2013. ACM Press.
- [KLW15] Venkata Koppula, Allison Bishop Lewko, and Brent Waters. Indistinguishability obfuscation for Turing machines with unbounded memory. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 419–428, Portland, OR, USA, June 14–17, 2015. ACM Press.
- [KMR14] Vladimir Kolesnikov, Payman Mohassel, and Mike Rosulek. FleXOR: Flexible garbling for XOR gates that beats free-XOR. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 440–457, Santa Barbara, CA, USA, August 17–21, 2014. Springer Berlin Heidelberg, Germany.
- [KS08] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498, Reykjavik, Iceland, July 7–11, 2008. Springer Berlin Heidelberg, Germany.

- [Lin25] Robin Linus. Bitvm 3s – garbled circuits for efficient computation on bitcoin. Technical report, BitVM Alliance, 2025.
- [LPR10] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23, French Riviera, May 30 – June 3, 2010. Springer Berlin Heidelberg, Germany.
- [LWYY25] Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. BitGC: Garbled circuits with 1 bit per gate. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VII*, volume 15607 of *LNCS*, pages 437–466, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.
- [MORS24] Pierre Meyer, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Rate-1 arithmetic garbling from homomorphic secret sharing. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part IV*, volume 15367 of *LNCS*, pages 71–97, Milan, Italy, December 2–6, 2024. Springer, Cham, Switzerland.
- [MORS25a] Pierre Meyer, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Computationally succinct authentication from dcr: Attribute-based laconic function evaluation and more. Manuscript, 2025.
- [MORS25b] Pierre Meyer, Claudio Orlandi, Lawrence Roy, and Peter Scholl. Silent circuit relinearisation: Sublinear-size (boolean and arithmetic) garbled circuits from dcr. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, 2025.
- [MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany.
- [NPS99] Moni Naor, Benny Pinkas, and Reuban Sumner. Privacy preserving auctions and mechanism design. In *Proceedings of the 1st ACM Conference on Electronic Commerce*, pages 129–139, 1999.
- [NR97] Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. In *38th FOCS*, pages 458–467, Miami Beach, Florida, October 19–22, 1997. IEEE Computer Society Press.
- [OSY21] Claudio Orlandi, Peter Scholl, and Sophia Yakubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 678–708, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- [Pai99] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT’99*, volume 1592 of *LNCS*, pages 223–238, Prague, Czech Republic, May 2–6, 1999. Springer Berlin Heidelberg, Germany.
- [PSSW09] Benny Pinkas, Thomas Schneider, Nigel P. Smart, and Stephen C. Williams. Secure two-party computation is practical. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 250–267, Tokyo, Japan, December 6–10, 2009. Springer Berlin Heidelberg, Germany.
- [QWW18] Willy Quach, Hoeteck Wee, and Daniel Wichs. Laconic function evaluation and applications. In Mikkel Thorup, editor, *59th FOCS*, pages 859–870, Paris, France, October 7–9, 2018. IEEE Computer Society Press.
- [RR21] Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 94–124, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
- [RS21] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 687–717, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

- [Wee24] Hoeteck Wee. Circuit ABE with $\text{poly}(\text{depth}, \lambda)$ -sized ciphertexts and keys from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 178–209, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
- [Wee25] Hoeteck Wee. Almost optimal KP and CP-ABE for circuits from succinct LWE. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 34–62, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.
- [Yao82] Andrew Chi-Chih Yao. Protocols for secure computations (extended abstract). In *23rd FOCS*, pages 160–164, Chicago, Illinois, November 3–5, 1982. IEEE Computer Society Press.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *27th annual symposium on foundations of computer science (Sfcs 1986)*, pages 162–167. IEEE, 1986.
- [ZRE15] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.